SYSTEM DESIGN

Complete PDF Collection

Organized by topic • Original materials preserved

19 source PDFs | 230 source pages

Networking → APIs → Security → Scalability → Performance

A compiled reference with table of contents and PDF bookmarks.
All source PDF pages appear unchanged.

Contents

01 Networking Foundations 3

02 Client-Server Architecture 69

03 IP Addressing and NAT 75

04 DNS and Name Resolution 81

05 TCP versus UDP 84

06 Forward and Reverse Proxies 89

07 REST API Design 95

08 Beyond REST: GraphQL and gRPC 101

09 Real-Time Communication 107

10 CORS and Web Security 113

11 API Gateway 118

12 Content Delivery Networks 123

13 Scalability Foundations 129

14 Scaling Strategies 159

15 Introduction to Load Balancing 163

16 Load Balancing Deep Dive 167

17 Autoscaling and Cloud Practices 173

18 System Performance Foundations 176

19 Caching for Speed Optimization 229

Page numbers are PDF viewer pages (including the front cover and contents).

Introduction to Networking in System Design

Networking & Communication (System Design Fundamentals)

Mastering System Design: From Basics to Cracking Interviews

Rahul Rajat Singh



Instructor: Rahul Rajat Singh

Why Networking Matters in System Design?

Every system relies on data exchange
between components.

Networking enables scalability,
reliability, and performance.

Key areas where networking plays
a crucial role:



Communication

Ensuring smooth data transfer between clients, servers, and databases.

Load Balancing

Distributing traffic efficiently to prevent overload on a single server.

Security

Protecting data from unauthorized access and cyber threats.

Efficiency

Optimizing network performance to reduce latency and improve user experience.

How Networking Impacts Large-Scale Systems

Helps handle millions of users concurrently

Enables fast and efficient data exchange

Reduces latency & improves system resilience

Essential for cloud computing & distributed systems



Instructor: Rahul Rajat Singh

Agenda

Key Networking Concepts in System Design

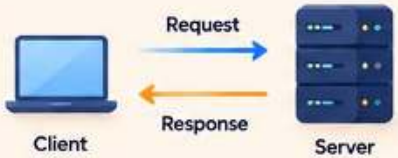
1 Understanding IP Addresses



2 How DNS Works



3 Client-Server Model Explained



4 Forward Proxy vs. Reverse Proxy



5 Introduction to Load Balancing



6 What is an API Gateway?



7 Content Delivery Networks (CDN) in System Design



Instructor: Rahul Rajat Singh

Wrap-Up & What's Next

1. **Networking is the foundation of system design**
   Everything in a distributed system communicates over a network.

2. **Next, we'll deep dive into networking components**
   Understand the building blocks that make communication possible.

3. **Coming up:**
   - IP Addressing & DNS





Next Lesson
IP Addressing & DNS

Understanding IP Addresses in System Design

Networking & Communication (System Design Fundamentals)

Mastering System Design: From Basics to Cracking Interviews





Instructor: Rahul Rajat Singh

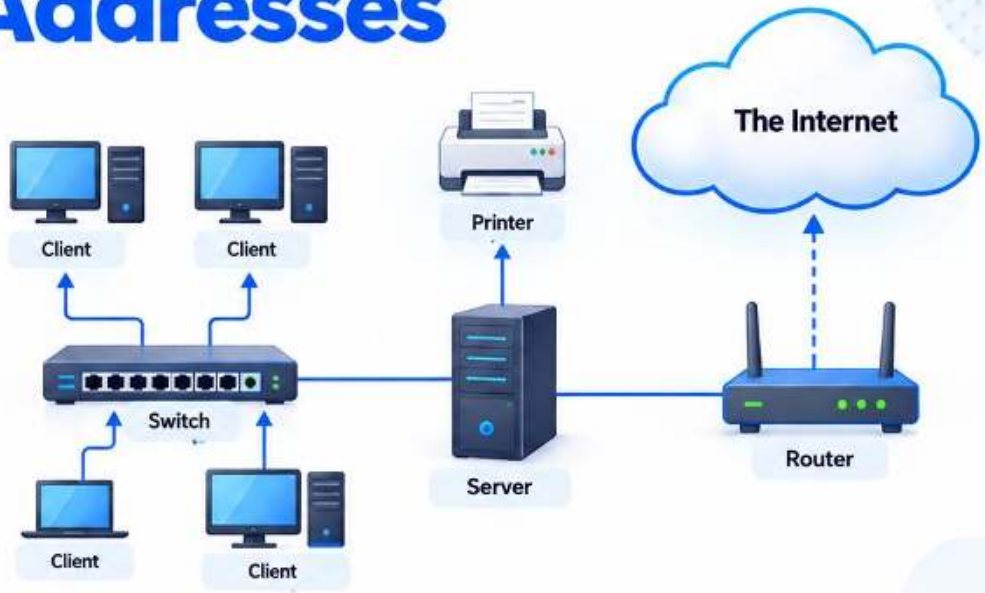
Introduction to IP Addresses

Unique Identifier

IP (Internet Protocol) addresses are unique numerical labels assigned to devices on a network.

Enables Communication

They enable communication between different machines, servers, and services across the internet.



Two Primary Versions

IPv4

- 32-bit address
- e.g. 192.168.1.1
- Widely used

IPv6

- 128-bit address
- e.g. 2001:db8::1
- Massive address space

Two Primary Categories

Public

- Globally unique
- Routable on the internet
- Used to access devices and services from anywhere

Private

- Used within internal networks
- Not directly accessible from the internet
- Common ranges: 10.0.0.0/8, 172.16.0.0/12, 192.168.0.0/16

What is IPv4?

Definition
IPv4 (Internet Protocol Version 4) is the most widely used addressing system.

Address Format
0101
0101

32-bit address format (e.g., 192.168.1.1).

Total Addresses
~4.3 billion addresses ($2^{32} \approx 4,294,967,296$).

Common Uses
Traditional networking, web servers, and most current internet devices.

Challenges
Limited IPs, fragmentation, and security concerns.



Example IPv4 Address

192 . **168** . **1** . **1**

1st Octet	2nd Octet	3rd Octet	4th Octet
(0 – 255)	(0 – 255)	(0 – 255)	(0 – 255)

What is IPv6?

IPv6 (Internet Protocol Version 6)
is the next-generation IP standard.

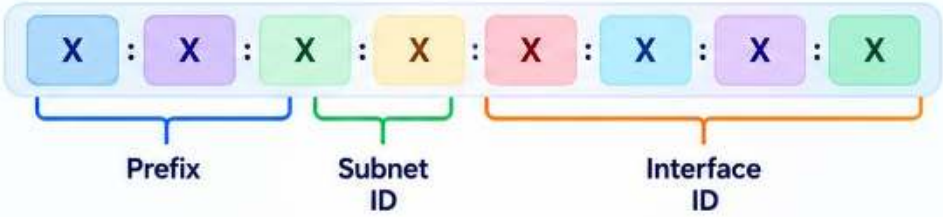
128-bit address format
Example:
`2001:0db8:85a3:0000:0000:8a2e:0370:7334`

Total addresses available
340 undecillion (virtually unlimited).

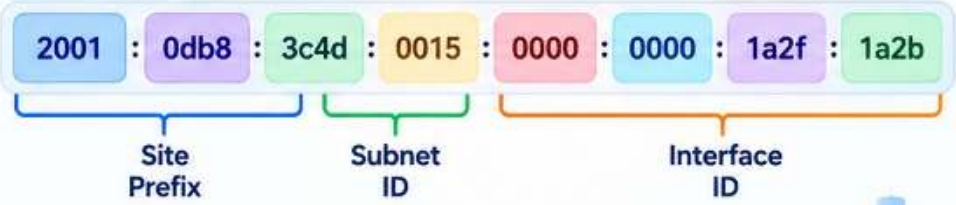
Designed for
IoT, mobile networks, and future scalability.

Key benefits
Larger address space, better security, and improved routing efficiency.

IPv6 Address Structure



Example:





IPv4 vs. IPv6 – Key Differences

IPv4



192.168.1.1



Feature	IPv4	IPv6
Address Size	32-bit	128-bit
Address Format	Decimal (e.g., 192.168.1.1)	Hexadecimal (e.g., 2001:db8::ff00:42:8329)
Address Space	~4.3 billion addresses	Virtually unlimited
Security	Relies on additional protocols	Built-in security (IPSec)
Performance	Limited due to NAT & fragmentation	More efficient routing & handling
Adoption	Still widely used	Slowly being adopted

IPv6



2001:db8::ff00:42:8329



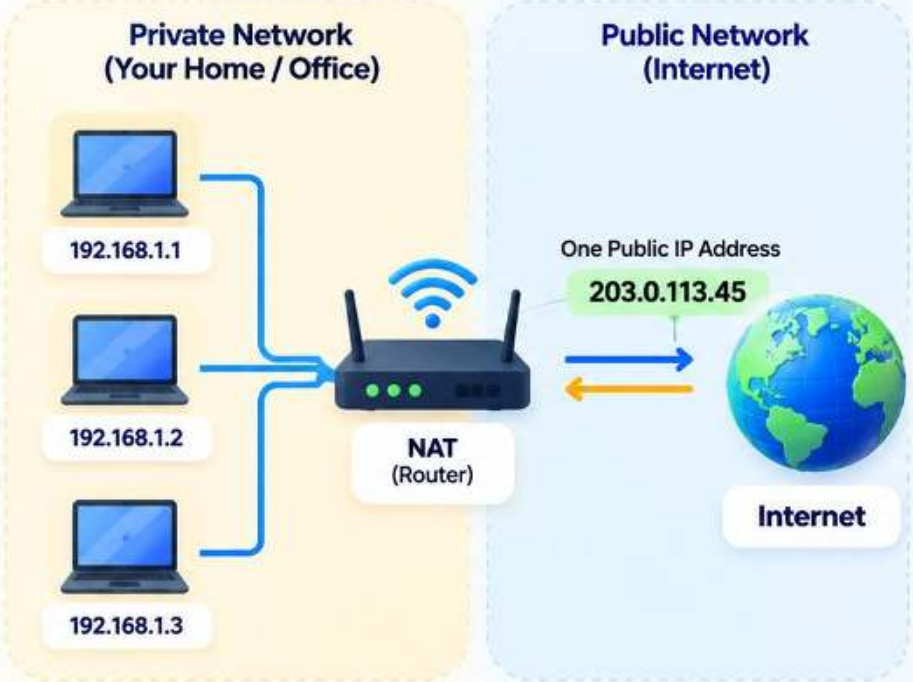
Private vs. Public IPs

Public IPs

- Assigned by ISPs (Internet Service Providers).
- Used to communicate over the internet.
- Unique worldwide.
- Example: 192.203.23.45

Private IPs

- Used within local networks (LANs, enterprises, homes).
- Cannot be accessed directly from the internet.
- Examples:
  - 10.0.0.0 - 10.255.255.255
  - 172.16.0.0 - 172.31.255.255
  - 192.168.0.0 - 192.168.255.255



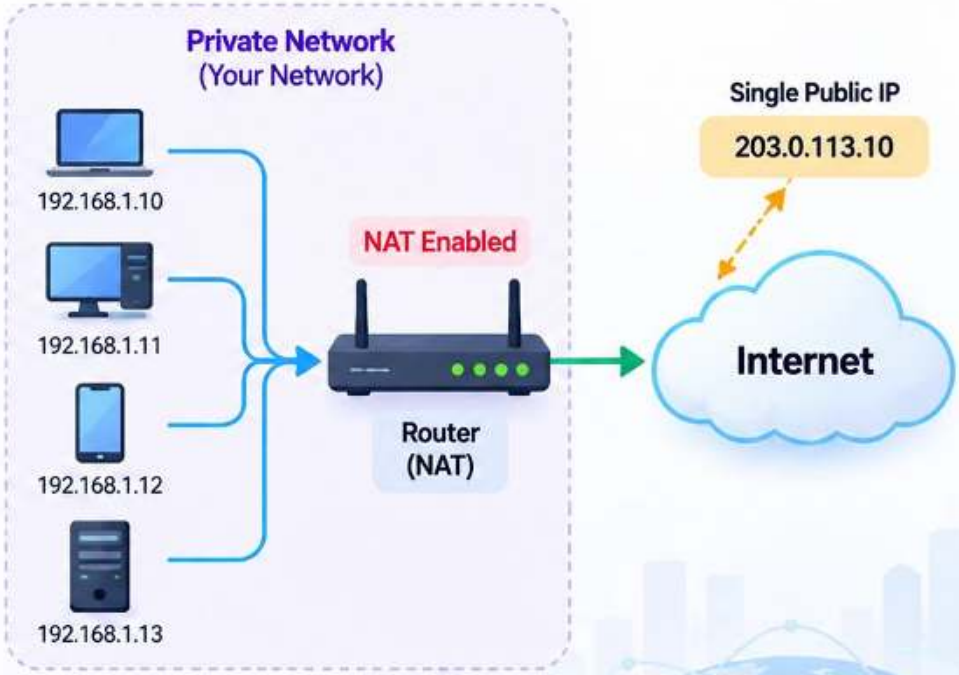
Why Do We Need Private IPs?

Conserves Public IP Addresses
Helps deal with IPv4 address limitation by allowing many devices to share a single public IP.

Enhances Security
Private IPs are not routable over the internet, keeping internal devices hidden from the public.

Enables Network Address Translation (NAT)
Allows multiple devices with private IPs to share a single public IP when accessing the internet.

Widely Used
Common in corporate networks, data centers, and cloud environments.



The Role of IPs in System Design

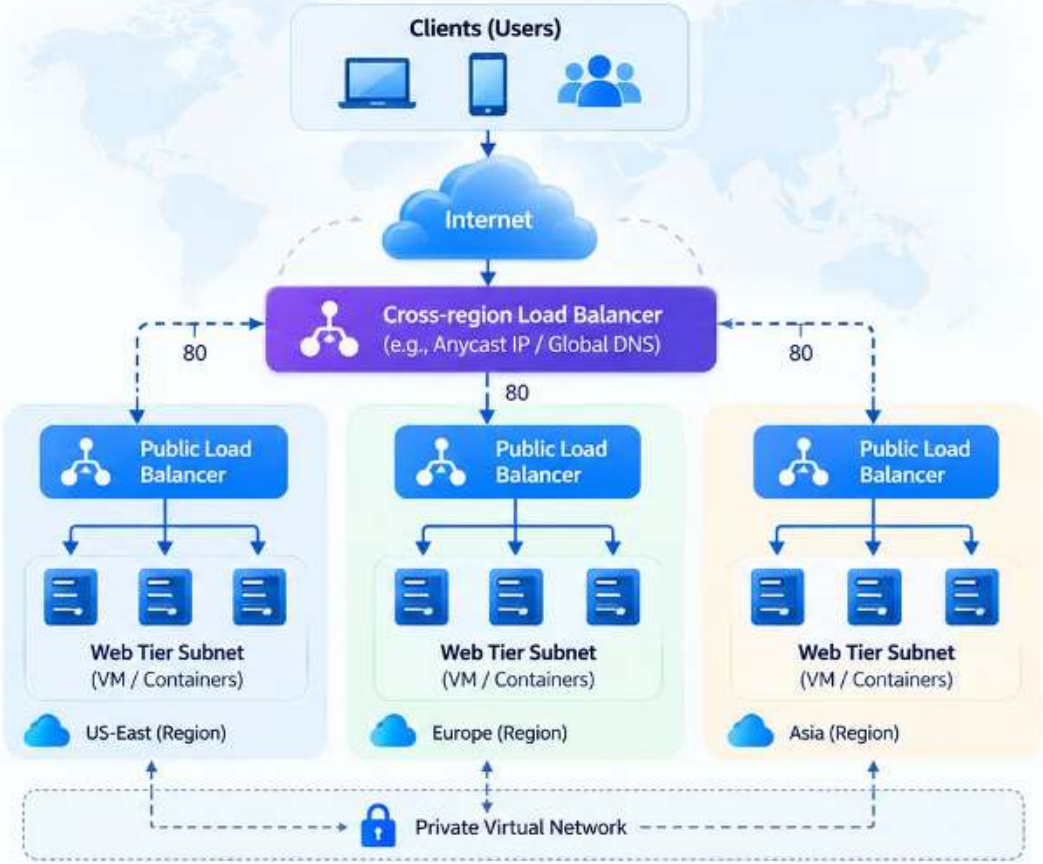
Scalability
Helps in designing distributed, multi-region, and cloud-based architectures.

Security
Enables firewall rules, VPNs, and private networking.

Load Balancing
Uses IP-based traffic distribution (e.g., Round-robin DNS, Anycast IPs).

Cloud Networking
Public, private, and hybrid cloud IP management (AWS, GCP, Azure).

Microservices & Containers
Use internal private IPs for communication.



Common System Design Interview Questions on IPs



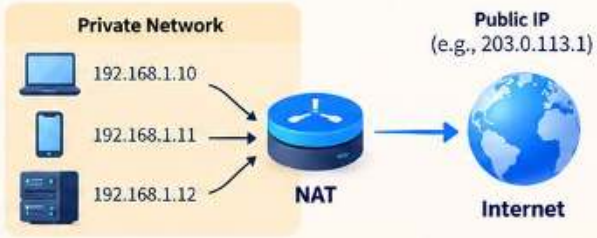
1 How do IPv4 and IPv6 addresses differ?

IPv4	IPv6
192.168.1.1	2001:0db8:85a3::7334
32-bit address (e.g., 192.168.1.1)	**128**-bit address (e.g., 2001:0db8:85a3::7334)

2 Why do we need private IPs in system design?



3 How does NAT help in addressing the IPv4 shortage?



4 Explain how a load balancer distributes traffic using IPs.



5 How does DNS resolve IP addresses in a large-scale system?



Instructor: Rahul Rajat Singh

Summary & What's Next

1. **IPv4 and IPv6 are the two primary internet protocols.**
   IPv4 is widely used, while IPv6 provides a much larger address space.

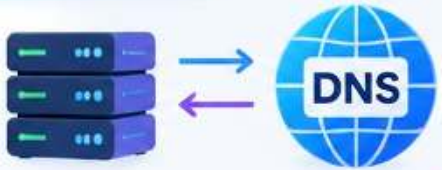
2. **Private vs. Public IPs help manage internal and external networking.**
   Private IPs are used inside networks, while public IPs are used to communicate over the internet.

3. **Networking & IPs are essential for system scalability, security, and performance.**
   They form the foundation of reliable and scalable systems.

4. **Coming up next:**
   - How DNS works and its role in system design.



NEXT LESSON



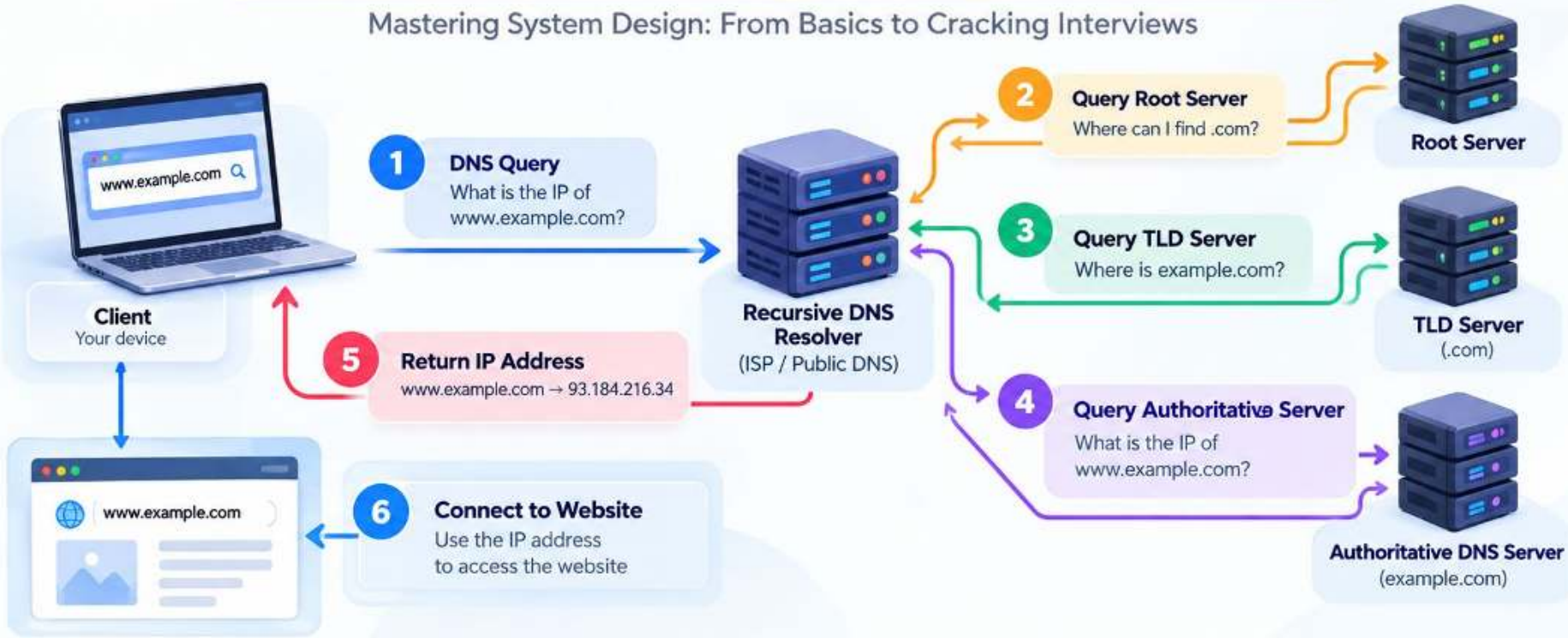
How DNS works and its role in system design.

Instructor: Rahul Rajat Singh

How DNS Works

Networking & Communication (System Design Fundamentals)

Mastering System Design: From Basics to Cracking Interviews



Instructor: Rahul Rajat Singh

Introduction to DNS

What is DNS?

- The "phonebook of the internet."
- Translates human-readable domain names (e.g., google.com) into IP addresses.

Why is DNS Important?

- Enables accessibility of websites.
- Supports distributed, scalable internet architecture.



Instructor: Rahul Rajat Singh

Types of DNS Servers

1 

Root Name Servers

Top-level servers handling .com, .org, etc.

2 

TLD Name Servers

Specific to domains like .com, .net, etc.

3 

Authoritative Name Servers

Store domain-specific records (e.g., IP addresses).

4 

Recursive Resolvers

Handle queries on behalf of users, contacting multiple DNS servers if necessary.



User Request
www.example.com

4 

Recursive Resolver
(ISP or Public DNS)

Returns IP address

1 

Root Name Servers
(Top-Level Servers for .com, .org, etc.)

Refers to TLD servers

2 

TLD Name Servers
(Handles .com, .net, etc.)

Refers to authoritative servers

3 

Authoritative Name Servers
(Stores Domain-Specific Records)



Resolved IP Address Returned
93.184.216.34



Website Loads in Browser
www.example.com

Instructor: Rahul Rajat Singh

DNS Caching and Performance Optimization



1 Why caching matters?

- Reduces latency.
- Reduces load on DNS servers.

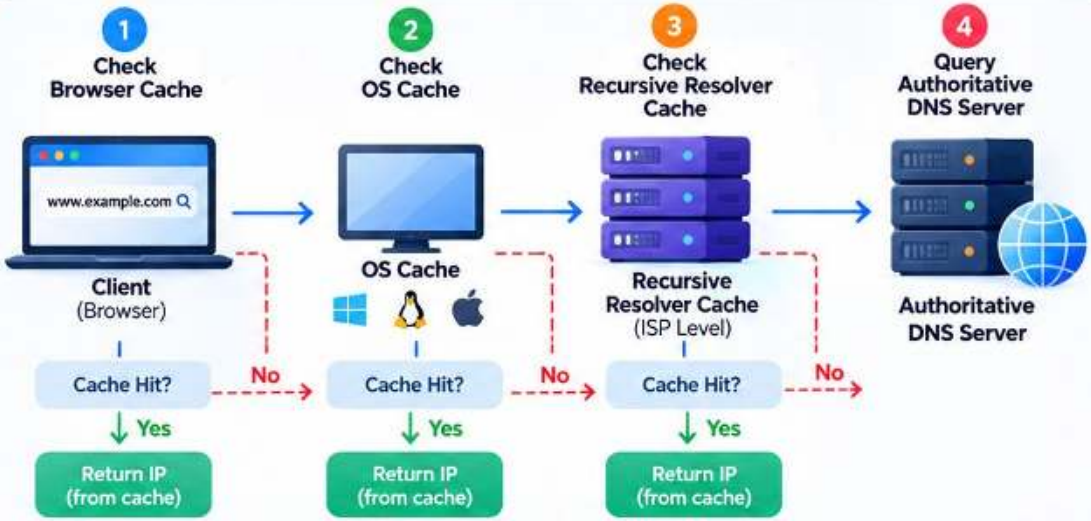
2 Where caching occurs?

- **Browser cache**
  Stores previous lookups.
- **OS cache**
  (etc/hosts, Windows DNS cache).
- **Recursive Resolver cache**
  (ISP level).

3 Time-To-Live (TTL)

Determines how long a cached record is valid.

How DNS Caching Works



TTL (Time-To-Live)
Determines how long a cached record is valid.

Example:

```
example.com IN A 93.184.216.34
```

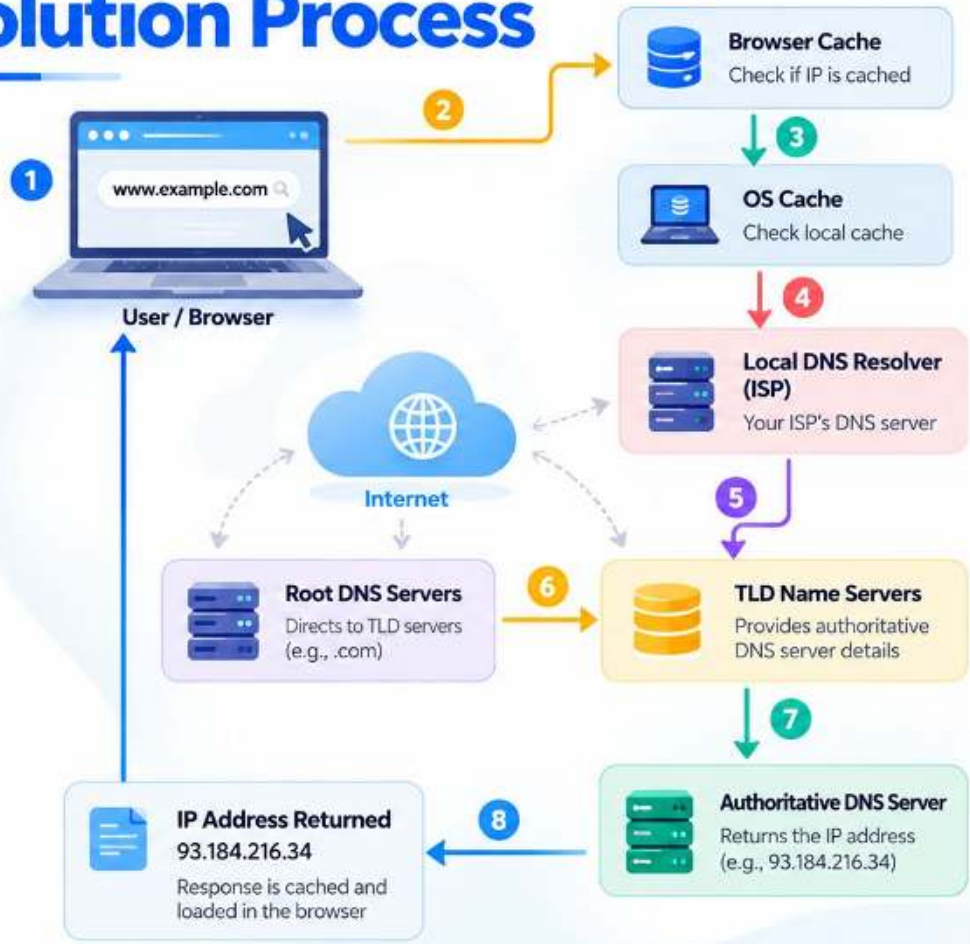
TTL = 300s

The record will be cached for 300 seconds (5 minutes).

Instructor: Rahul Rajat Singh

The Domain Name Resolution Process

1. **User types a domain**
   You enter a domain name in the browser (e.g., www.example.com)
2. **Browser cache check**
   Browser checks if the IP is already cached.
3. **OS cache check**
   Operating System (OS) checks its local cache.
4. **Query to local DNS resolver (ISP)**
   If not found, query goes to the configured DNS server (usually by your ISP).
5. **Query to Root DNS servers**
   If still not found, the resolver queries a Root DNS server.
6. **TLD name servers**
   Root server refers to the TLD (e.g., .com) name servers, which provide the authoritative server details.
7. **Authoritative DNS server**
   The authoritative DNS server for the domain returns the IP address.
8. **IP returned & cached**
   The IP address is returned to the browser and cached for future use.



Instructor: Rahul Rajat Singh

Importance of DNS in Large-Scale Systems

1 Ensuring High Availability

- Load balancing using DNS.
- Anycast DNS for faster global resolution.

2 DNS Failover Strategies

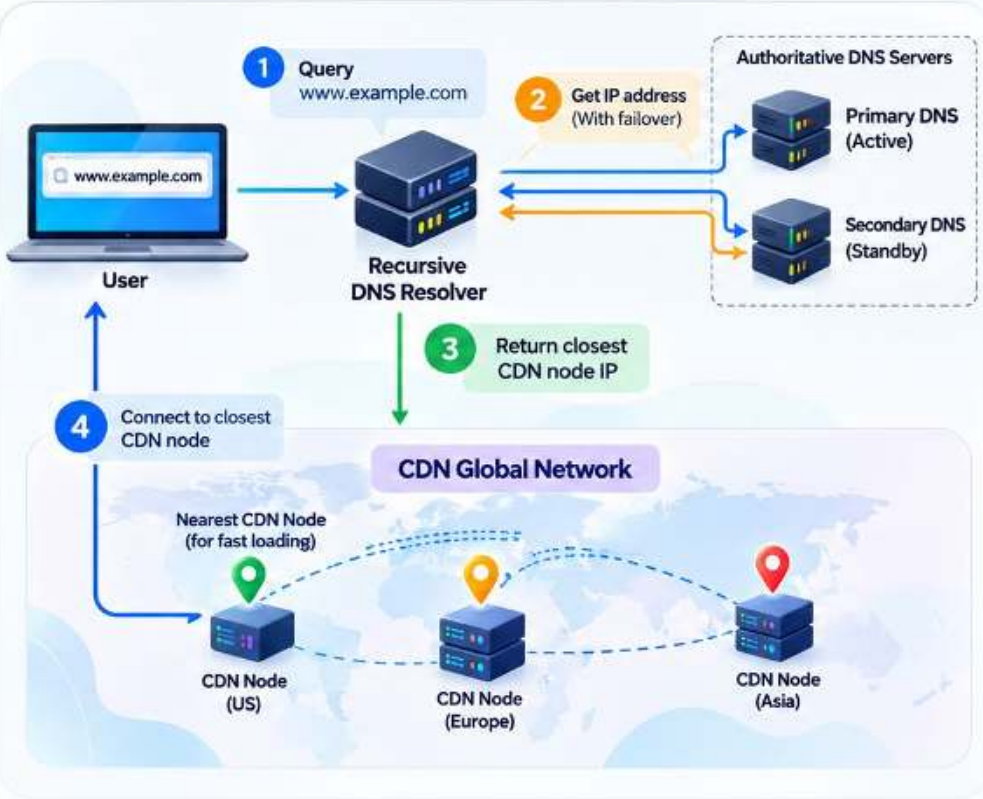
- Primary and secondary DNS servers.
- Automatically route traffic during failures.

3 Content Delivery Networks (CDNs) & DNS

- How DNS directs users to the nearest CDN node for fast loading.
- Improves performance and reduces latency.

4 DNS Security Risks

- DNS poisoning, cache poisoning, and DDoS attacks.
- Use DNSSEC, rate limiting, and monitoring.



Instructor: Rahul Rajat Singh

Interview Questions Related to DNS

1 Explain the DNS resolution process step by step.



2 What is the difference between authoritative and recursive DNS servers?



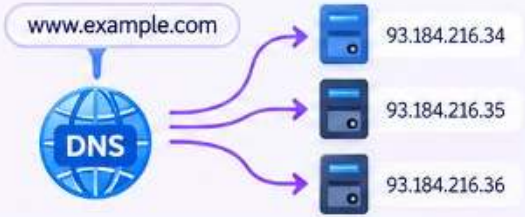
3 How does DNS caching improve performance? Where does it occur?



4 What is TTL in DNS, and why is it important?



5 How does DNS-based load balancing work in large-scale systems?



6 What are common DNS-related security threats, and how can they be mitigated?



Instructor: Rahul Rajat Singh

Summary & Takeaways

1. **DNS is critical** for internet functionality.
2. **Caching improves** efficiency and reduces latency.
3. **DNS load balancing & failover** are essential for large-scale systems
4. **Security threats exist,** but best practices help mitigate them



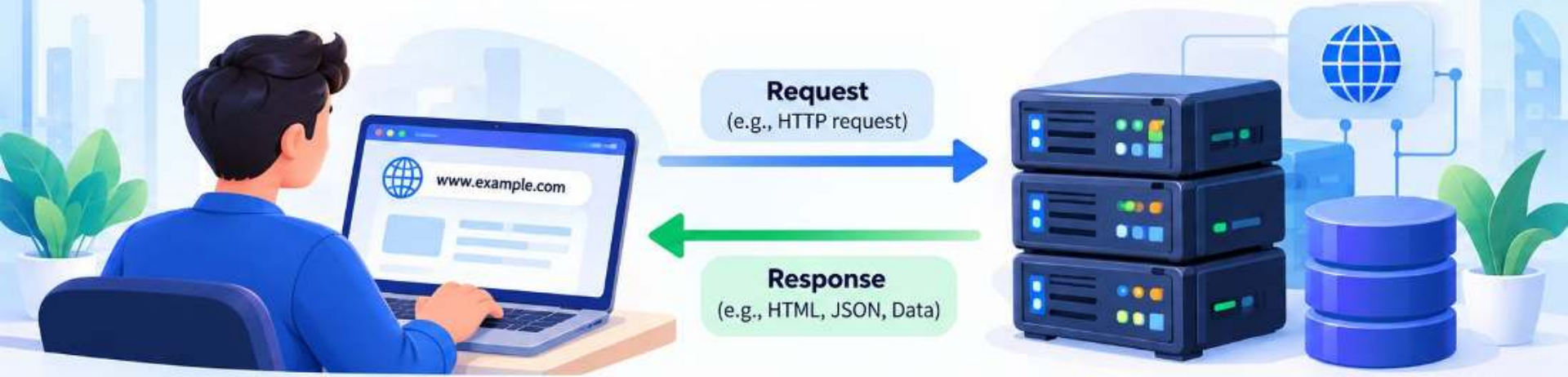
What's Next?

- Client-Server Model Explained

Client-Server Model Explained

Networking & Communication (System Design Fundamentals)

Mastering System Design: From Basics to Cracking Interviews



Instructor: Rahul Rajat Singh

What is the Client-Server Model?

1

Definition

A computing model where clients request services and servers provide them.

2

Foundation

Foundation of modern web, database, and application architectures.

3

Why is it important?

- Enables efficient resource management.
- Used in web browsing, email, APIs, databases, and cloud services.

4

Real-world examples

- Browsing a website (Browser = Client, Web Server = Server).
- Sending an email (Email App = Client, Mail Server = Server).



Real-World Examples





Instructor: Rahul Rajat Singh

Key Components of the Client-Server Model

1 Client



- The user-facing application that sends requests.
- Examples: Web browser, mobile app, API consumer.

2 Server

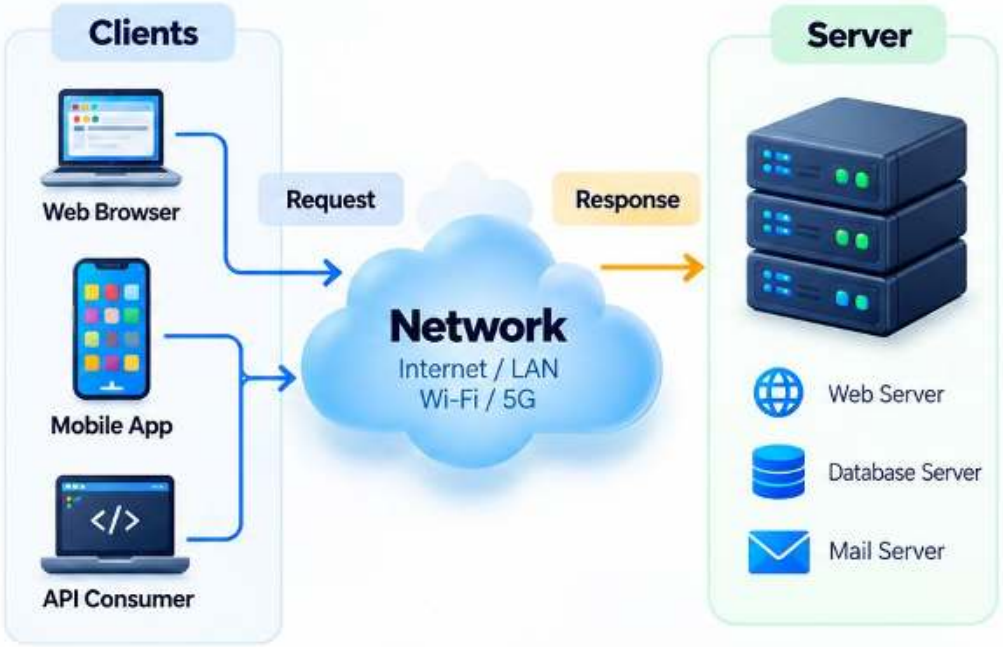


- A system that processes requests and returns responses.
- Examples: Web server, database server, mail server.

3 Network



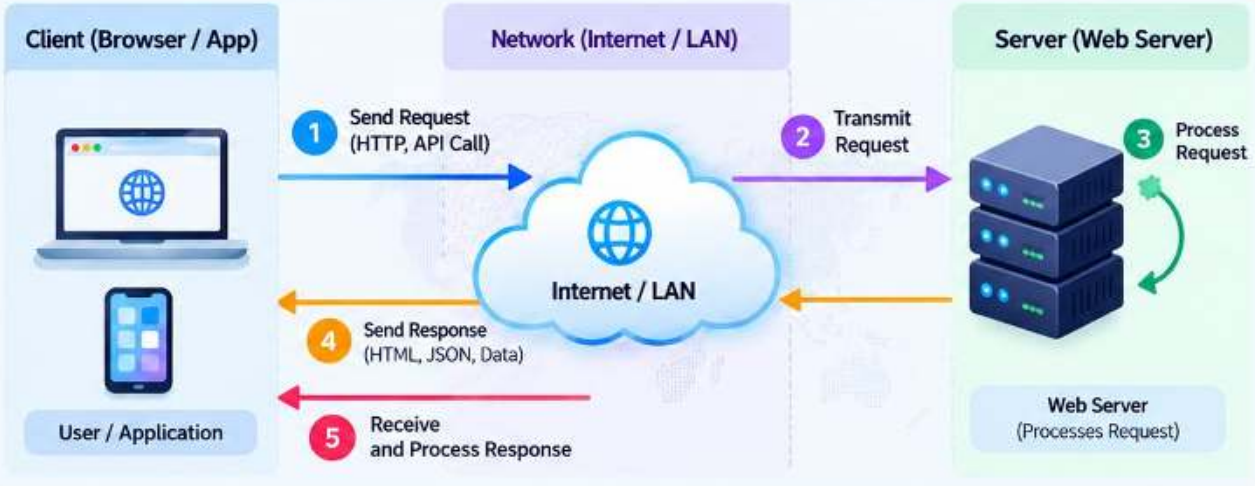
- The medium that facilitates communication between clients and servers.
- Examples: Internet, LAN, Wi-Fi, 5G.



How Clients and Servers Communicate

Basic Steps

1. Client sends a request (HTTP, SQL query, etc.).
2. Request is transmitted over the network.
3. Server receives and processes the request.
4. Server sends a response.
5. Client receives and processes the response.

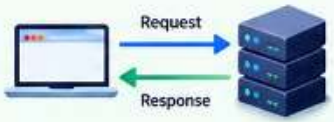


Types of Communication

Request/Response Model

(e.g., HTTP, REST APIs)

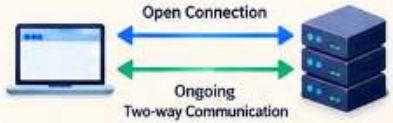
- Client sends a request, server responds.
- Simple and widely used.
- Common for web applications and APIs.



Persistent Connections

(e.g., WebSockets, FTP sessions)

- Keeps the connection open.
- Allows two-way communication.
- Used for real-time applications like chat, live updates, and file transfers.



Instructor: Rahul Rajat Singh

The Request-Response Cycle (HTTP Example)

1. 

Browser request
Browser requests https://example.com.

2. 

DNS resolution
DNS resolves domain to IP address.

3. 

Send HTTP request
Browser sends an HTTP GET request to the web server.

4. 

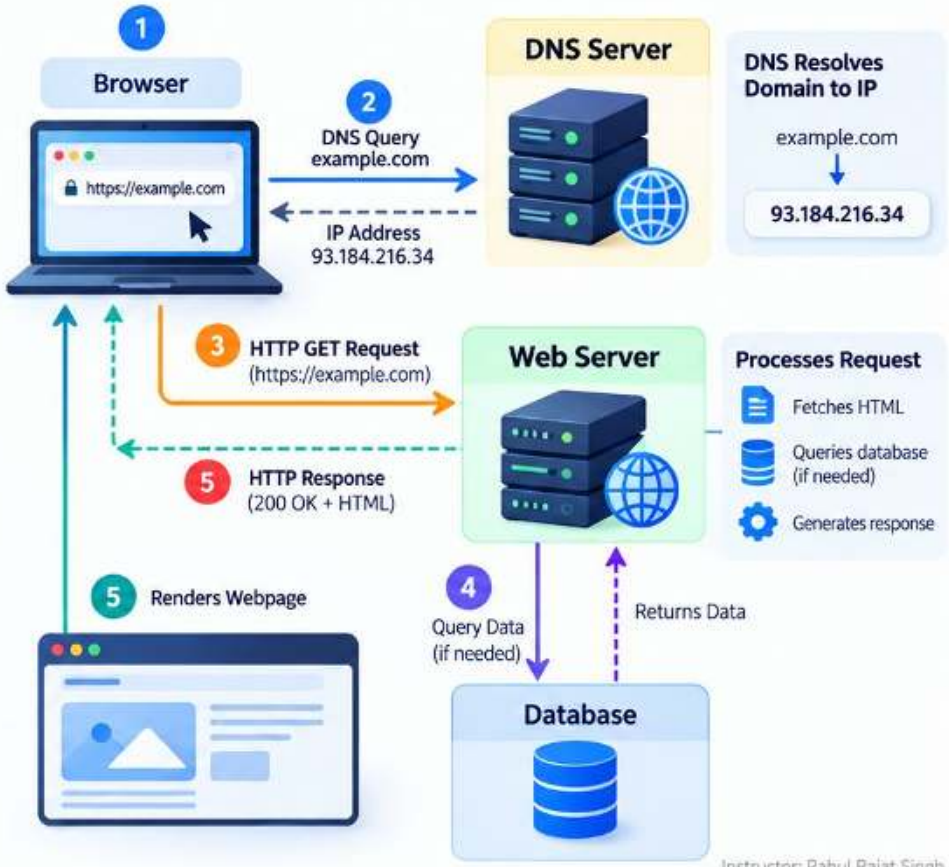
Server processes request
Web server processes request, fetches HTML, queries the database if needed.

5. 

Send HTTP response
Server sends an HTTP response with a status code (200 OK) and the webpage content.

6. 

Browser renders webpage
Browser renders the webpage.

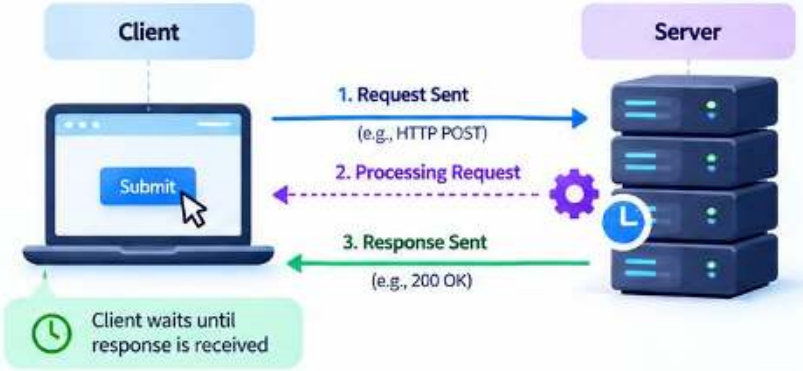


Instructor: Rahul Rajat Singh

Synchronous vs Asynchronous Communication

Synchronous Communication

- Client waits for a response before proceeding.
- Example: A user submits a form and waits for a confirmation message.
- Used in REST APIs, traditional web apps.



Real-World Examples

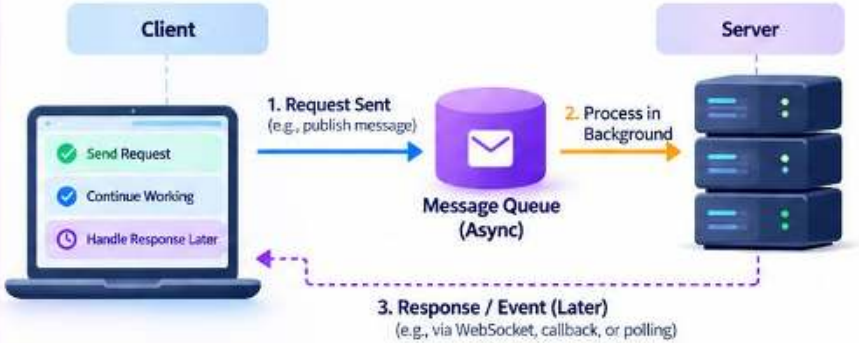
REST APIs

Traditional Web Apps

Form Submissions

Asynchronous Communication

- Client doesn't wait for a response and can perform other tasks.
- Example: WebSockets for real-time chat, AJAX for background requests.
- Used in messaging apps, live updates, IoT applications.



Real-World Examples

Messaging Apps
(e.g., WhatsApp, Slack)

Live Updates
(e.g., notifications, real-time feeds)

IoT Applications
(e.g., sensor data)

Stateless vs Stateful Servers

1 Stateless Servers



- **No memory** of past interactions; each request is independent.
- **Example:** REST APIs, HTTP servers.
- **Benefits:** Scalability, easy caching, load balancing.

2 Stateful Servers



- **Maintain** session information across requests.
- **Example:** WebSockets, multiplayer games, banking applications.
- **Benefits:** Personalization, seamless user experiences.





Common Stateless Examples

REST APIs · HTTP Servers · Microservices · CDN (Static Content)

Common Stateful Examples

WebSockets (Real-time apps) · Multiplayer Games · Banking Applications · Personalized User Experiences

Real-World Examples

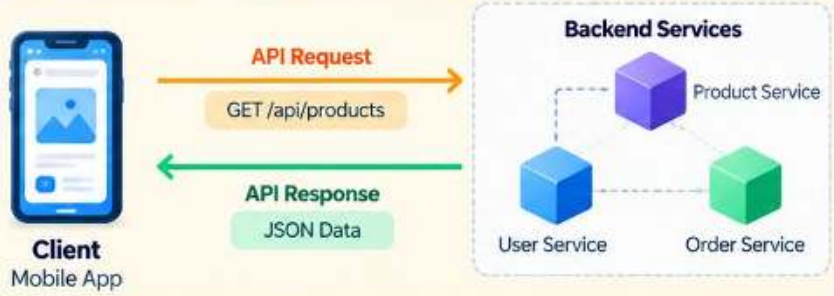
1 Web Applications

Browsing a website (Client = Browser, Server = Web Server).



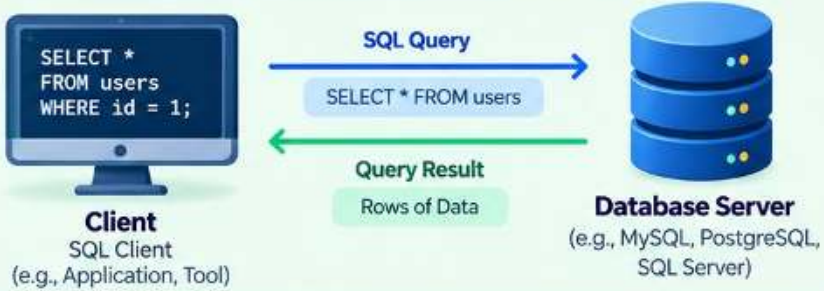
2 APIs & Microservices

Mobile apps calling backend APIs (REST, GraphQL).



3 Databases

A client querying a database (SQL Client → Database Server).



4 Messaging Systems

WhatsApp, Slack using WebSockets for real-time communication.



Interview Questions on the Client-Server Model

1 Fundamental Questions

1. What is the client-server model, and how does it work?
2. How does a client communicate with a server?
3. What are some real-world examples of the client-server model?
4. What is the difference between a client and a server?



2 Request-Response Cycle Questions

1. Explain the HTTP request-response cycle with an example.
2. What are the key differences between synchronous and asynchronous communication?
3. How does a browser load a webpage? Walk me through the steps.
4. What happens in the network, server, and client during a request-response cycle?



3 Architecture and Design Questions

1. What is the difference between stateless and stateful servers?
2. How does caching improve performance in a client-server model?
3. How do load balancers work in a client-server architecture?



4 Advanced Questions

1. What are some security challenges in the client-server model?
2. How does WebSockets differ from traditional request-response communication?
3. What are some limitations of the client-server model? How can they be addressed?
4. How would you design a scalable client-server system for a high-traffic application?



Instructor: Rahul Rajat Singh

Wrap up and What's Next...

Key Takeaways

1. **The client-server model**
   underpins most modern computing.
2. **Different communication patterns**
   (sync vs async, stateless vs stateful)
   optimize performance for different use cases.
3. **Understanding this model**
   helps in system design and scalability planning.



Next Steps

Forward Proxy vs. Reverse Proxy

In the Next Lecture...

We'll dive into Forward Proxy vs. Reverse Proxy in detail and see how they are used in real-world systems.

Forward Proxy vs. Reverse Proxy

Networking & Communication (System Design Fundamentals)

Forward Proxy

Acts on behalf of the client to access external resources.



Use Cases

- Access control
- Content filtering
- Anonymity

Examples

- Corp. proxy for internet access
- VPN, Web proxy

Benefits

- Hides client identity
- Enforces policies
- Controls outbound access

Reverse Proxy

Acts on behalf of the server to handle client requests.



Use Cases

- Load balancing
- SSL/TLS termination
- Security (WAF)
- Routing to backend services

Examples

- Nginx, Apache
- AWS ALB / Azure Application Gateway
- Cloudflare

Benefits

- Improves security
- Distributes traffic
- Hides backend servers
- Enables scalability and high availability

Instructor: Rahul Rajat Singh

What is a Proxy?

1. A proxy is an **intermediary server** between a client and another server.
2. Proxies help with **security, caching, traffic control, and anonymity.**
3. Two main types: **Forward Proxy & Reverse Proxy.**



1 Forward Proxy

Acts on behalf of **clients**, accessing external servers.



- Hides client's IP
- Controls access
- Caches content
- Improves security
- Useful in corporate networks

2 Reverse Proxy

Acts on behalf of **servers**, handling client requests.



- Hides backend servers
- Load balancing
- SSL termination
- Caching
- Improves security
- Used in modern web architectures

Instructor: Rahul Rajat Singh

What is a Forward Proxy?

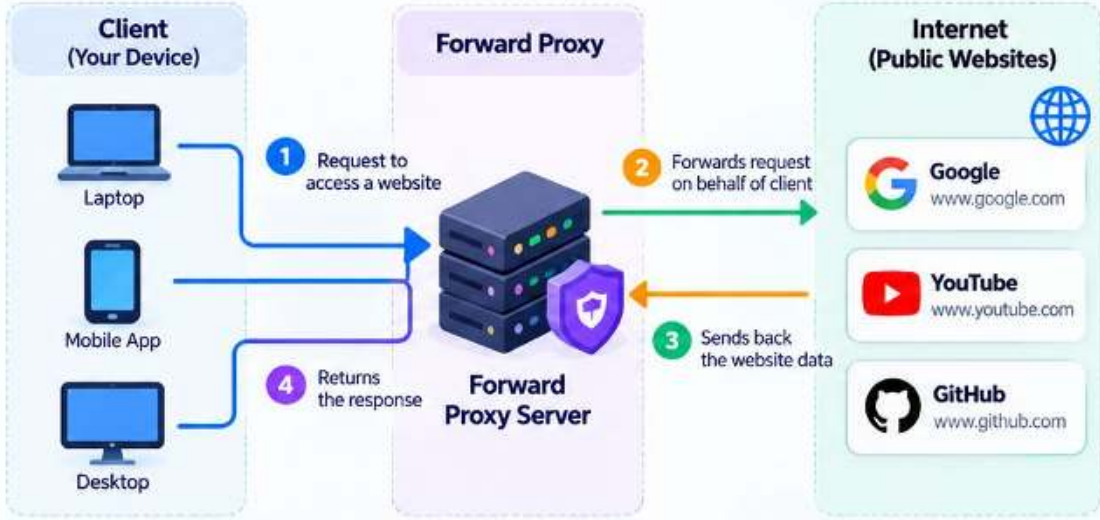
- Sits between a client and the internet (used by clients).
- Clients connect to the forward proxy instead of directly accessing websites.
- Can filter content, provide anonymity and cache requests.

Common Use Cases

Bypassing geo-restrictions
Access websites that are blocked in your region.

Anonymous browsing (VPNs, TOR)
Hides your real IP address.

Caching web pages
Stores frequently accessed content for faster access.



How it Helps

Filter Content
Block unwanted websites or content.

Provide Anonymity
Hide the client's real IP address.

Cache Responses
Store frequently accessed pages.

Enforce Policies
Control and monitor internet usage in organizations.

Instructor: Rahul Rajat Singh

What is a Reverse Proxy?

- Sits between users and backend servers (used by servers).
- Clients don't directly communicate with backend servers—requests go through the reverse proxy.
- Used for **load balancing**, caching, security, and SSL termination.



Common Use Cases

Load Balancing
Distribute traffic across multiple servers.

Caching Content
Cache static content to improve speed.

DDoS Protection & Security
Protect backend servers from attacks and enhance security.

Instructor: Rahul Rajat Singh

Key Differences — Forward vs. Reverse Proxy

Forward Proxy



Position	Between client & internet
Who Uses It?	Clients (users, browsers, apps)
Purpose	Anonymity, filtering, access control
Example	VPN, Web Proxy

VS

Reverse Proxy



Position	Between users & backend servers
Who Uses It?	Web servers, application servers
Purpose	Load balancing, caching, security
Example	Nginx, Cloudflare, AWS ALB

Instructor: Rahul Rajat Singh

Interview Questions on Forward Proxy vs. Reverse Proxy



1 Basic Concepts



1. What is a proxy server, and why is it used?
2. Explain the key differences between a forward proxy and a reverse proxy.

2 Technical Deep Dive

1. How does a forward proxy improve security and privacy?
2. How does a reverse proxy help in load balancing and caching?
3. What are some real-world examples of forward and reverse proxies?



3 Implementation & Use Cases



1. When should you use a forward proxy vs. a reverse proxy in system architecture?
2. What are some common tools and technologies used for each type of proxy?
3. How does a reverse proxy protect backend servers from DDoS attacks?

4 Advanced Topics

1. How does SSL termination work in a reverse proxy?
2. What are the advantages of using Cloudflare, Nginx, or HAProxy as a reverse proxy?
3. How does a transparent proxy differ from a forward or reverse proxy?



Instructor: Rahul Rajat Singh

Wrap-Up & What's Next?

Key Takeaways

1. A **forward proxy** helps clients interact with the internet securely.
2. A **reverse proxy** protects and optimizes backend servers.
3. Understanding proxy usage helps in designing scalable & secure architectures.

Forward Proxy — Helps clients access the internet securely



Reverse Proxy — Protects and optimizes backend servers



Next Topic:

Load Balancers



- How load balancers distribute traffic
- L4 vs L7 load balancing
- Real-world use cases and architectures

Introduction to Load Balancing

Networking & Communication (System Design Fundamentals)

Mastering System Design: From Basics to Cracking Interviews

Rahul Rajat Singh



Introduction to Load Balancing

Modern applications must handle **growing traffic.**

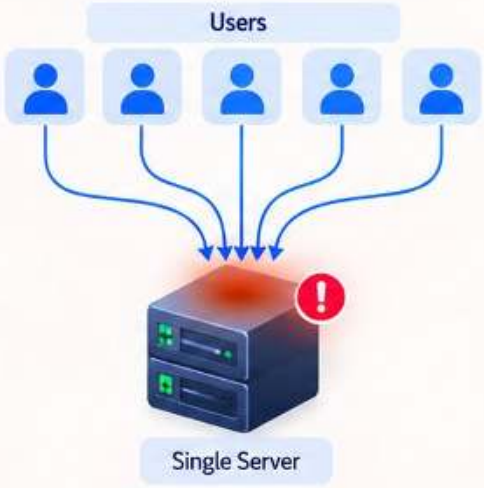
A single server eventually becomes a **bottleneck.**

Load balancing enables systems to **scale beyond one machine.**

It is a foundational building block of **distributed systems.**

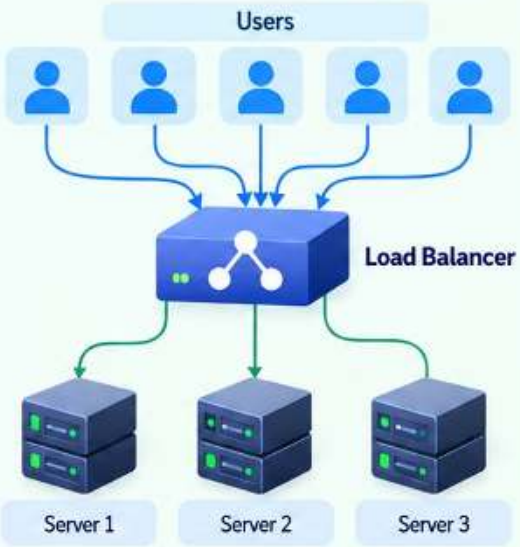
Without Load Balancing

A single server becomes a bottleneck



With Load Balancing

Distributes traffic across multiple servers



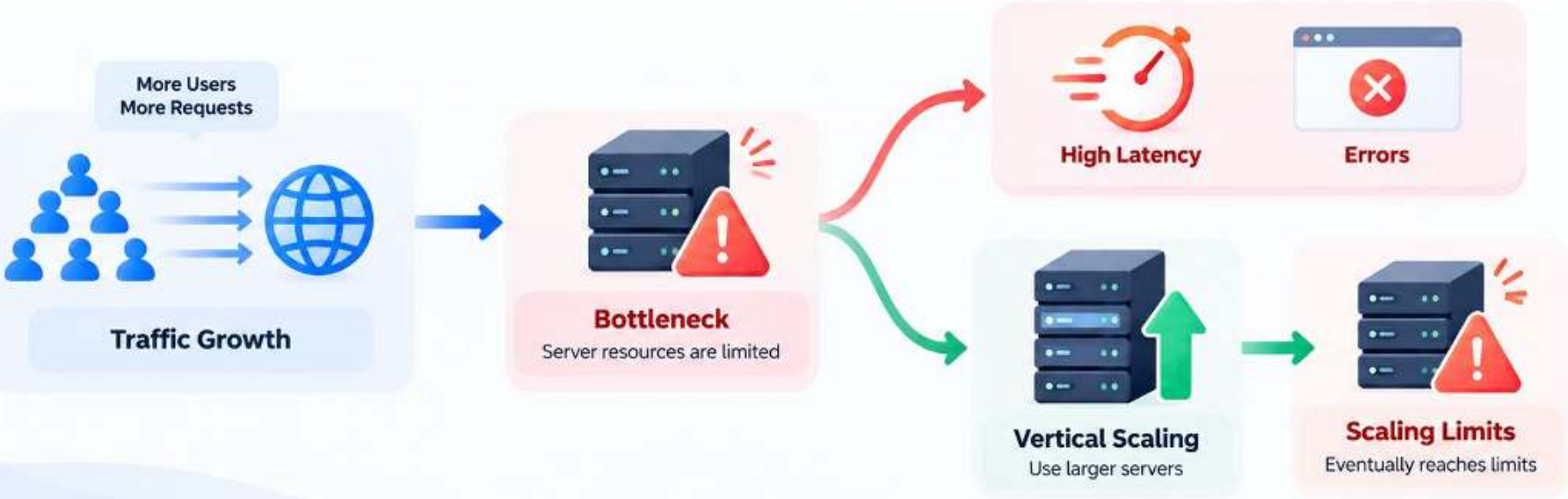
The Scaling Challenge

Server resources are **finite.**

Growing traffic increases **latency and failures.**

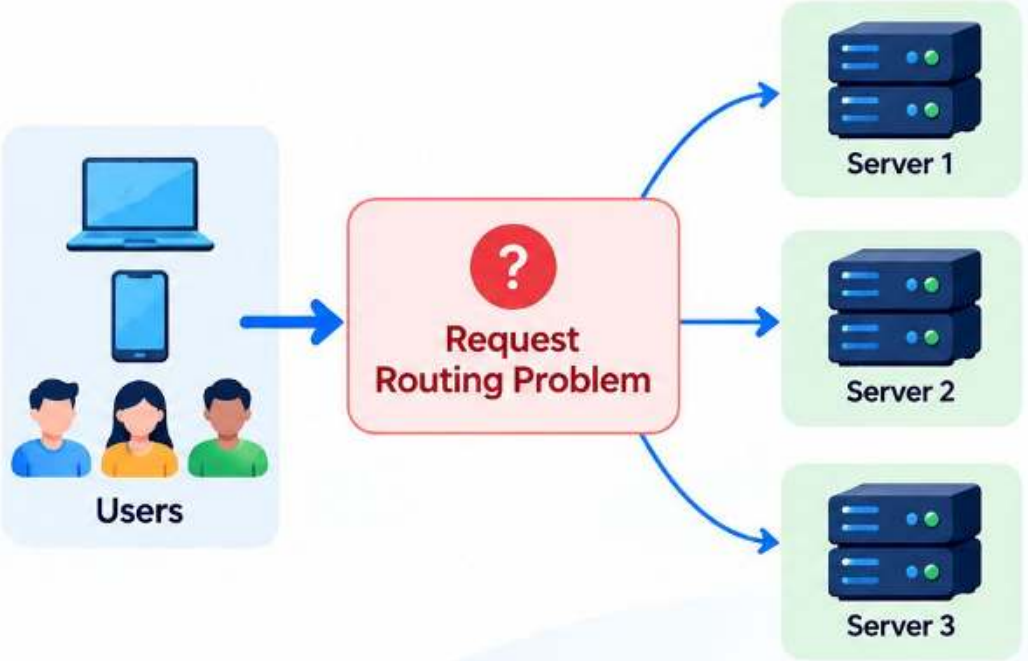
Larger servers help only **temporarily.**

Vertical scaling eventually **reaches limits.**



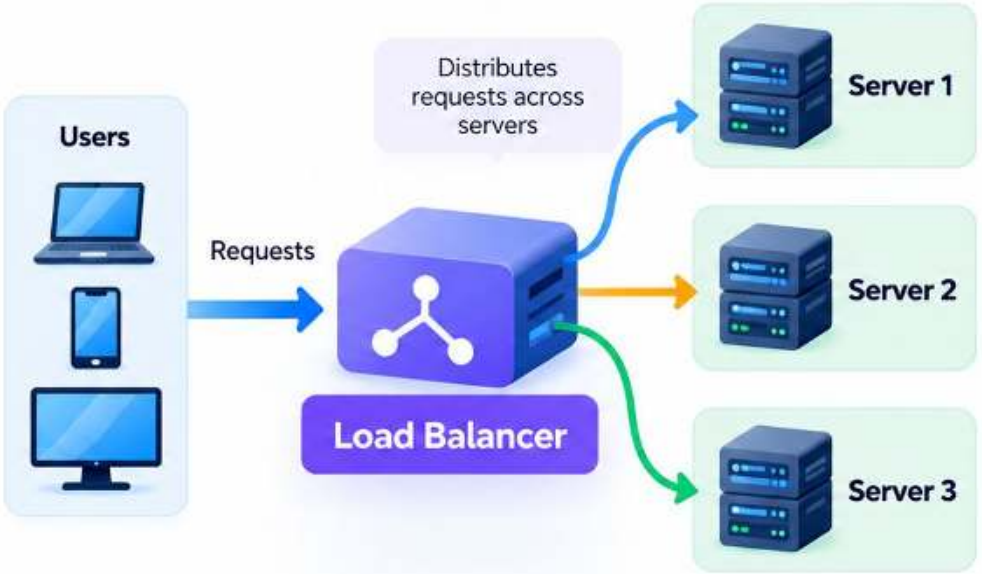
Horizontal Scaling Creates a New Problem

1. Capacity grows by adding servers.
2. Workload can be shared across machines.
3. Users should not choose servers manually.
4. Traffic now needs a distribution mechanism.



Introducing Load Balancing

1. A **load balancer** becomes the system's entry point.
2. Incoming requests are **distributed** across servers.
3. Traffic is no longer dependent on a **single machine**.
4. Applications can **scale more effectively**.



Load Balancing Enables Reliable Systems



Redundancy
Multiple servers provide redundancy.

Automatic Failover
Failed servers can be bypassed.

Safer Maintenance
Maintenance becomes safer.

Higher Availability & Resilience
Availability and resilience improve.

Instructor: Rahul Rajat Singh

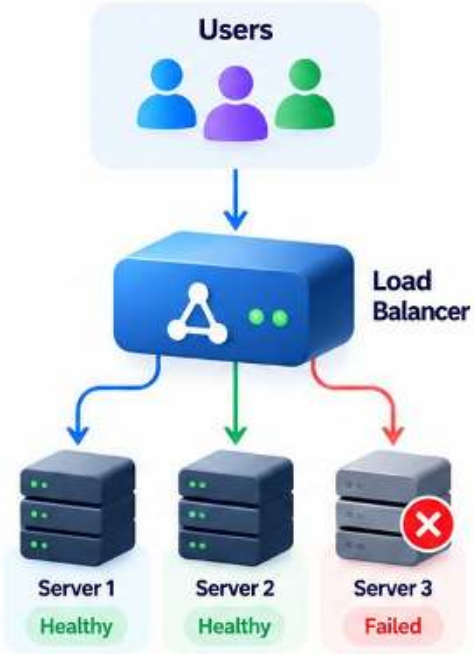
Interview Questions on Load Balancing

1 Fundamentals

1. What problem does load balancing solve?
2. Why does a single server eventually become a bottleneck?
3. What role does a load balancer play in a distributed system?

2 Scaling & Architecture

4. What is the difference between vertical scaling and horizontal scaling?
5. Why is horizontal scaling often preferred over vertical scaling?
6. What challenge arises when an application is deployed across multiple servers?



3 Reliability & Availability

7. How does load balancing improve scalability?
8. How does load balancing improve availability?
9. What happens if one server fails in a load-balanced architecture?

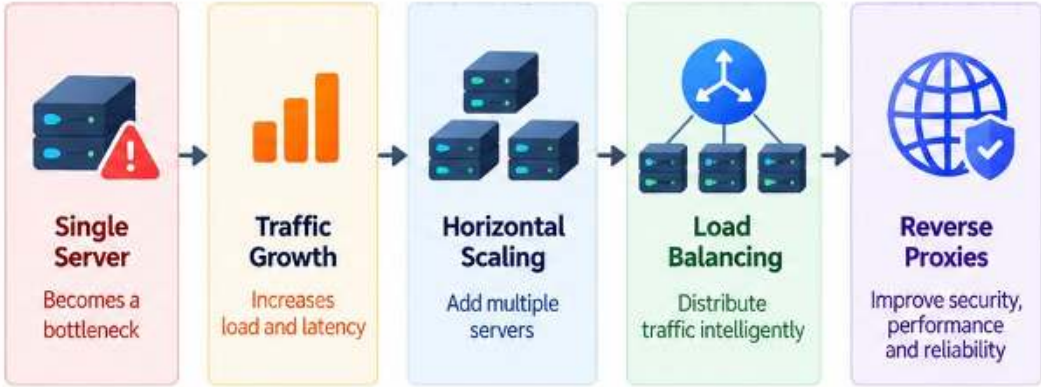
4 Design Thinking

10. Why is load balancing considered a foundational component of modern system design?
11. When should you introduce a load balancer into a system?
12. Walk me through the evolution from a single-server application to a load-balanced architecture.

Instructor: Rahul Rajat Singh

Summary & Key Takeaways

1. Single servers eventually become **scalability bottlenecks**.
2. Vertical scaling provides only **temporary relief**.
3. Horizontal scaling adds capacity through **multiple servers**.
4. Multiple servers require intelligent **traffic distribution**.
5. Load balancing enables **scalable** and **reliable systems**.



What's next:

- API Gateway

What is an API Gateway?

Networking & Communication (System Design Fundamentals)

Mastering System Design: From Basics to Cracking Interviews



Introduction to API Gateways

1. 

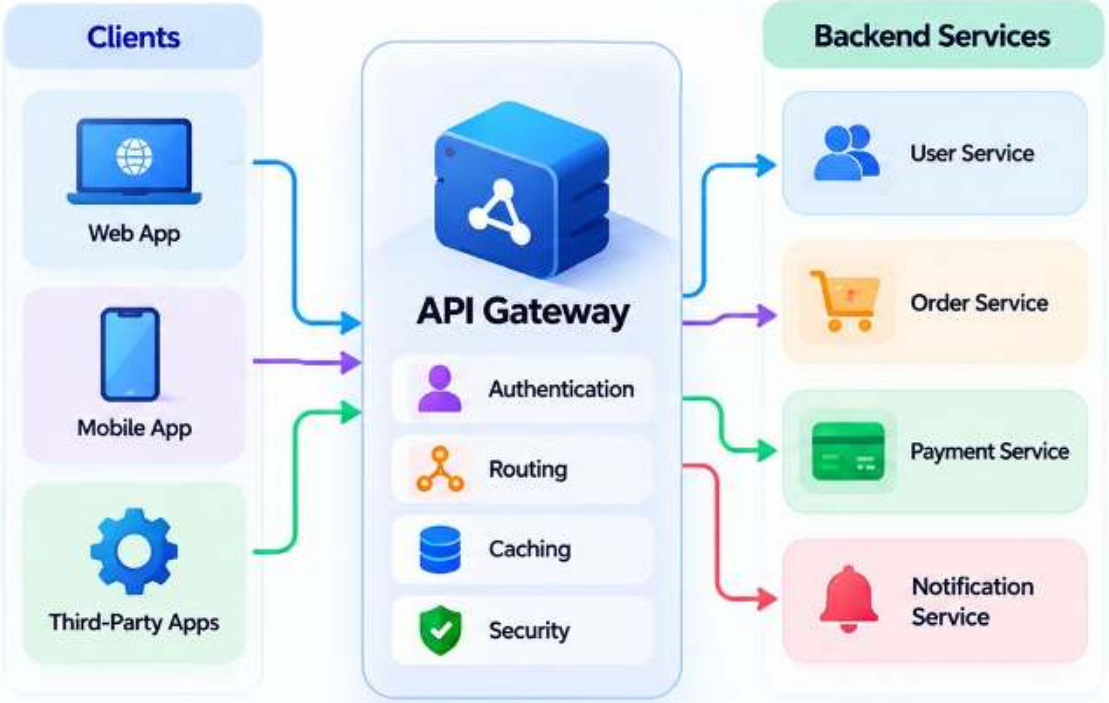
In modern system architecture, APIs connect **web, mobile, and third-party clients** to backend services.

2. 

Directly exposing backend services can lead to **performance, security, and management challenges.**

3. 

An API Gateway is a centralized entry point for all API requests, handling **authentication, routing, caching, and security.**



How API Gateways Work?

- API Gateway acts as a **reverse proxy** between clients and backend services.
- It receives API requests, processes them, and forwards them to the correct service.
- Key functions include request transformation, authentication, rate limiting, and response handling.



Why Use an API Gateway?

- Single entry point for all APIs
- Centralized security and authentication
- Apply rate limiting and throttling
- Simplifies client access to multiple services

Instructor: Rahul Rajat Singh

Benefits of Using an API Gateway

Security
Protects backend services from direct exposure.

Rate Limiting
Prevents abuse & DDoS attacks.

Load Balancing
Distributes traffic efficiently.

Caching
Speeds up response times.

Request Transformation
Converts request formats, protocols, and headers.

Logging & Monitoring
Tracks API performance and security events.



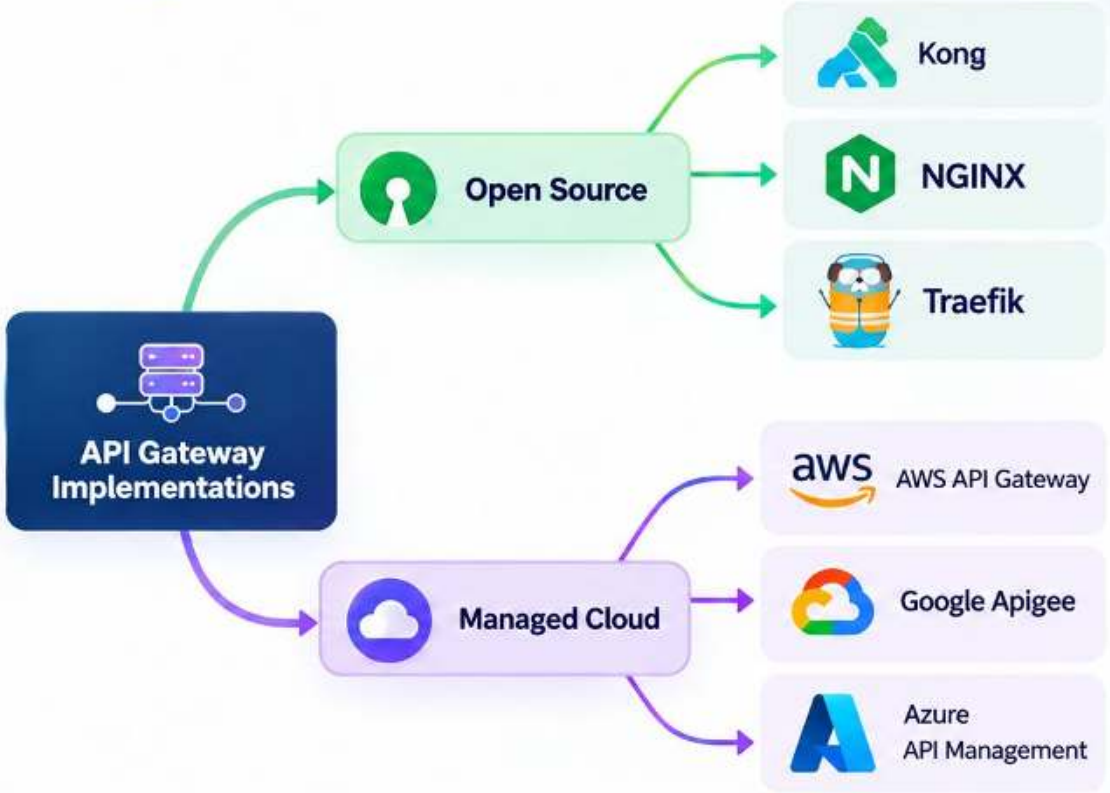
Popular API Gateway Implementations

Open-source solutions

- Kong
- NGINX
- Traefik

Cloud-based solutions

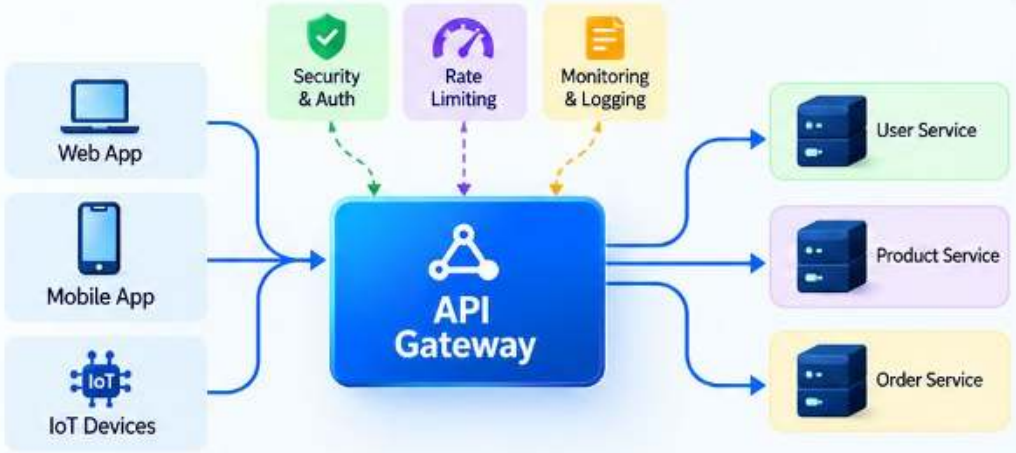
- AWS API Gateway
- Google Apigee
- Azure API Management



When to Use an API Gateway?

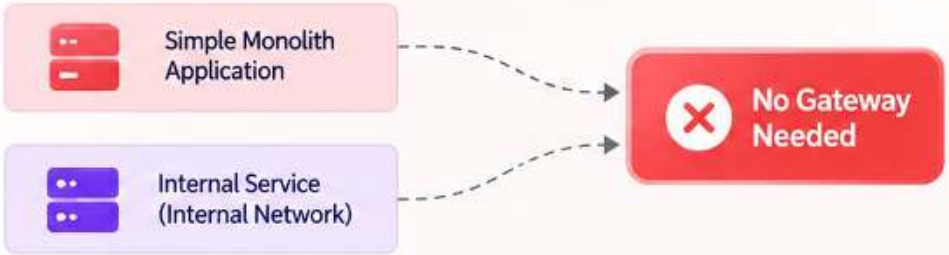
Best suited for:

- Microservices architectures.
- Multi-client APIs (Web, Mobile, IoT).
- APIs requiring security, rate limiting, and monitoring.



Avoid if:

- Simple monolithic apps with minimal API exposure.
- Low-traffic, internal-only services.



API Gateway – Interview Questions

1 Basic Questions

1. What is an API Gateway, and why is it used?
2. How does an API Gateway differ from a Load Balancer?
3. What are the key benefits of using an API Gateway?

2 Technical Questions

1. How does an API Gateway handle authentication and authorization?
2. Explain rate limiting and throttling in API Gateways.
3. What caching strategies can be implemented in an API Gateway?
4. How does an API Gateway improve security against DDoS attacks?

3 Scenario-Based Questions

1. When should you use an API Gateway in a microservices architecture?
2. How would you design an API Gateway for a large-scale system with millions of users?
3. What challenges might arise when implementing an API Gateway, and how would you address them?



Conclusion & Key Takeaways

1 Centralized Entry Point

API Gateway is a centralized entry point for managing API traffic.



2 Security, Rate Limiting, Caching & Logging

Provides security, rate limiting, caching, and logging.



3 Useful for Modern Architectures

Useful for microservices & high-traffic applications.



4 Choose Based on Use Case & Scale

Choosing the right API Gateway depends on use case & scale.



What's Next?

Content Delivery Networks (CDNs)



Learn how CDNs work, and how they help deliver content faster to users around the world.

Content Delivery Networks (CDN) in System Design

Networking & Communication (System Design Fundamentals)

Mastering System Design: From Basics to Cracking Interviews



Instructor: Rahul Rajat Singh

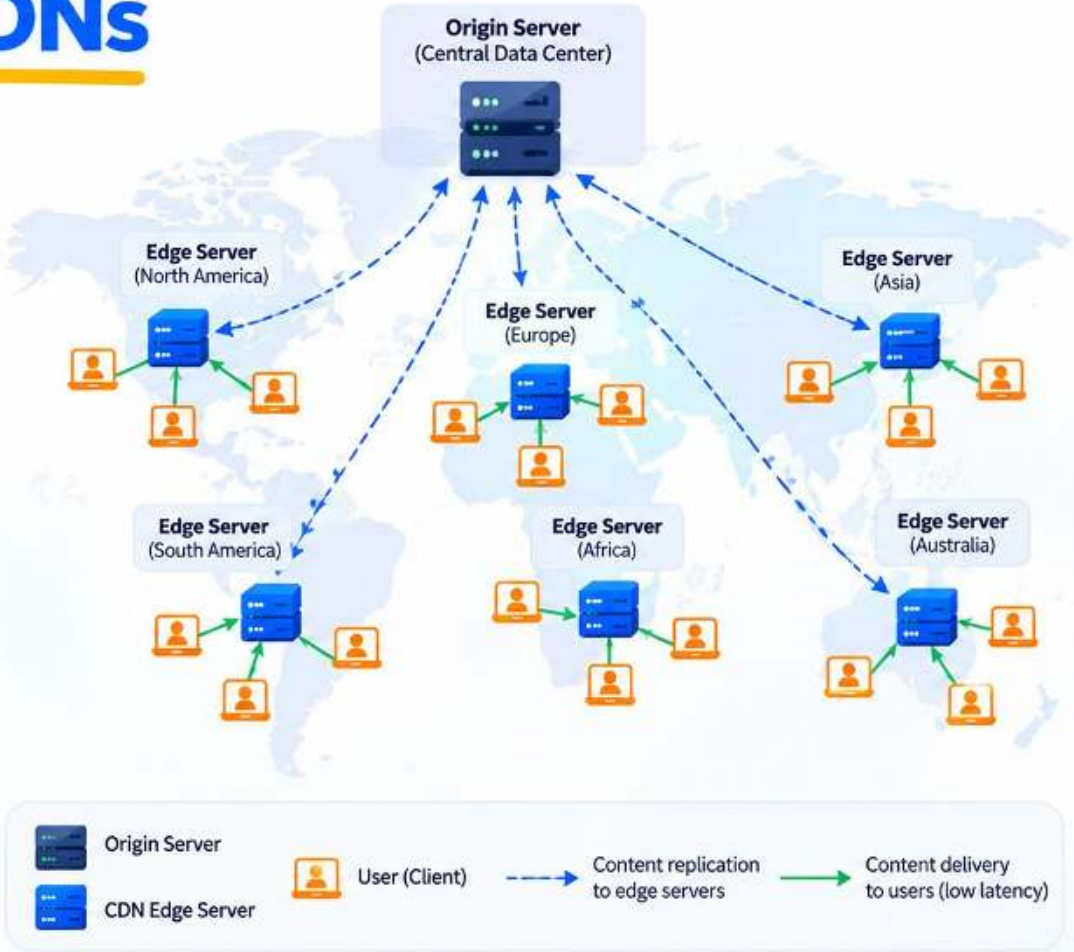
Introduction to CDNs

Definition:

A CDN is a globally distributed network of servers that work together to deliver content efficiently.

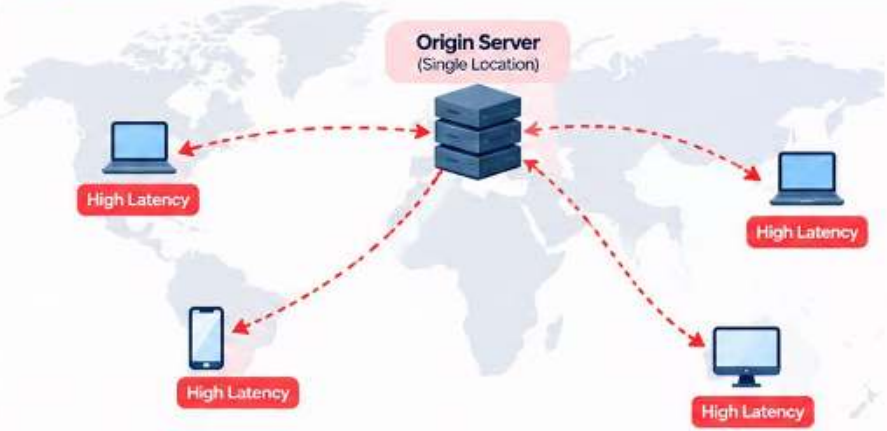
Why CDNs exist:

- Reduces **latency**
- Enhances **content availability**
- Improves **load handling**
- **Security**



Why are CDNs needed?

The problem without CDNs



High latency due to geographic distance

Users far from the origin server experience slower response times.

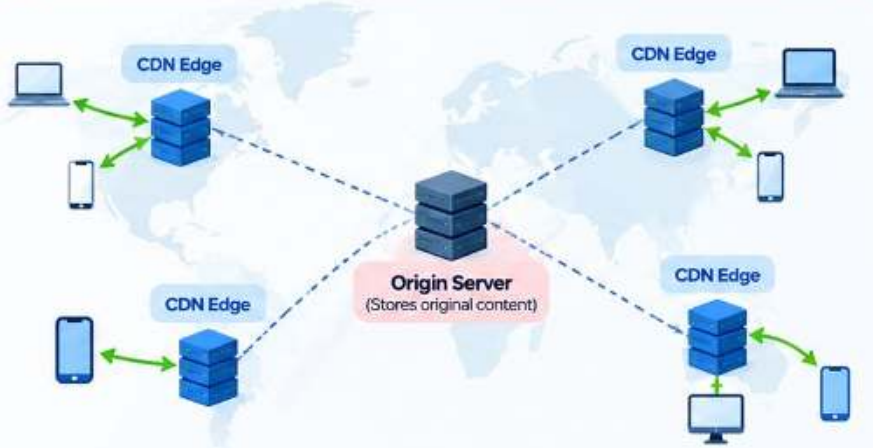
Overloaded origin servers

All traffic hits the same server, which can become a bottleneck during high demand.

Bandwidth constraints & slow load times

Large files and media consume more bandwidth, leading to slower downloads.

With CDNs (Content Delivery Networks)



Faster load times

Content is served from edge locations closer to users, reducing latency.

Less load on origin servers

Static content is cached at edge locations, reducing direct traffic to the origin server.

Better bandwidth utilization

Content is delivered efficiently from edges, reducing bandwidth constraints and improving overall performance.

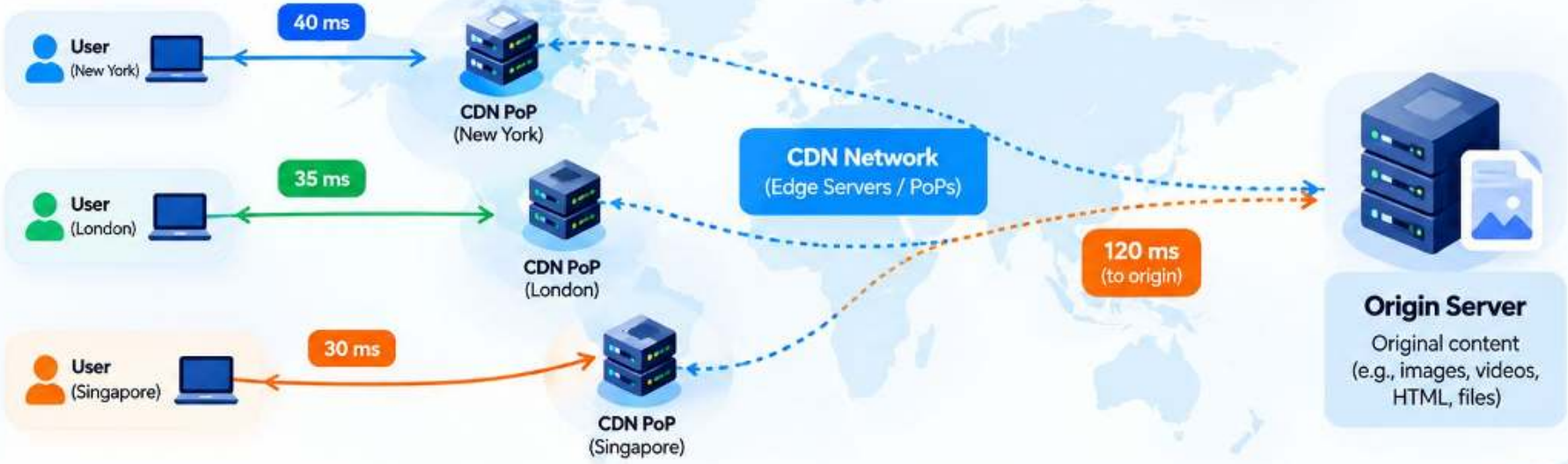
Instructor: Rahul Rajat Singh

CDN Architecture Overview

Origin Server
Stores the original content (e.g., images, videos, HTML, files).

Edge Servers / PoPs
Store cached content closer to users.

Request Routing
Directs users to the nearest PoP based on location and network conditions.



How a CDN Works and Handles a User Request

1 User requests content

User requests content (e.g., website, video, API data).

2 CDN routes to nearest edge

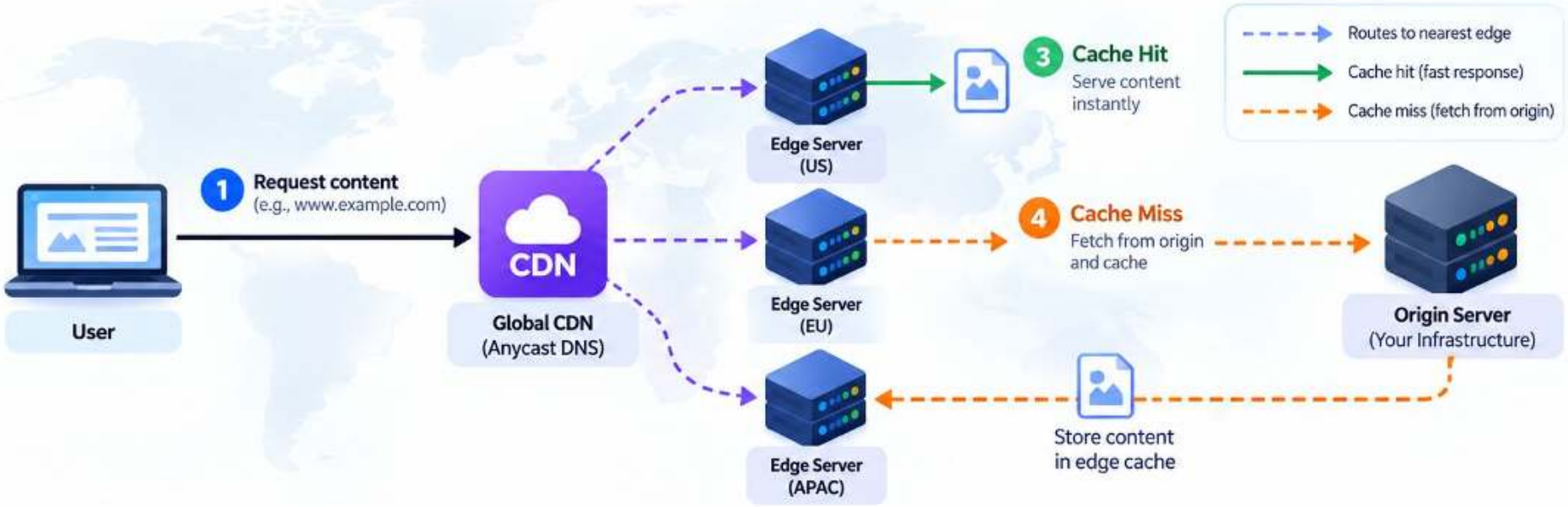
CDN directs the request to the nearest edge server based on geographic location, network latency, and server load.

3 Cache Hit

If the content is available on the edge server, it is delivered instantly.

4 Cache Miss

If the content is not cached, the request is forwarded to the origin server, fetched, and stored at the edge server for future use.



How CDNs Improve Performance, Reliability & Security

1 Caching & Replication
(Reduced Latency)

- Popular content is cached at edge locations worldwide.
- Cache TTL and invalidation strategies maintain content freshness.
- Reduces origin server load and accelerates content delivery.

2 Load Balancing & Intelligent Routing
(Improved Availability)

- Traffic is distributed across multiple PoPs.
- Geo-based, latency-based, and load-aware routing select the best edge location.
- Automatic failover reroutes traffic during outages.

3 Compression & Content Optimization
(Lower Bandwidth Costs)

- Gzip/Brotli compression reduces payload size.
- Image optimization (WebP, AVIF) improves delivery efficiency.
- CSS/JS minification reduces download times.

4 Security & Edge Protection
(Better Security)

- DDoS mitigation through rate limiting and traffic filtering.
- SSL/TLS termination secures communication.
- Protects origin infrastructure from direct exposure.



Use Cases of CDNs

Static vs. Dynamic Content Delivery

- **Static content:** Images, videos, HTML files.
- **Dynamic content:** API responses, personalized pages.

API Acceleration & Edge Computing

- Reduce API response times.
- Edge caching for frequently accessed endpoints.



Interview Questions - CDN



1 Basics of CDN



- What is a CDN, and how does it work?
- Why do we need CDNs in system design?
- What are the key benefits of using a CDN?

2 CDN Architecture & Components



- Explain the difference between an origin server and an edge server in a CDN.
- What is a PoP (Point of Presence) in a CDN?
- How does request routing work in a CDN?

3 Caching Strategies & Content Delivery



- What is a cache hit vs. cache miss, and how does a CDN handle them?
- Explain cache expiration and TTL (Time-To-Live) in a CDN.
- What are cache invalidation strategies, and why are they important?
- How does a CDN handle dynamic content caching?

4 Load Balancing & Failover Handling



- How do CDNs use load balancing to improve reliability?
- Explain different request routing strategies (Geo-based, Latency-based, Load-aware).
- What happens if a CDN PoP fails? How does failover handling work?

5 CDN Optimization Techniques



- What compression and minification techniques do CDNs use?
- How does a CDN optimize image and video delivery?
- How does API acceleration work in a CDN?

6 CDN Security & Challenges



- How does a CDN protect against DDoS attacks?
- What is SSL/TLS offloading, and why is it useful?
- What are some challenges in CDN implementation, and how can they be mitigated?

7 Advanced CDN Topics

- How does a multi-CDN architecture work?
- What is edge computing, and how does it relate to CDNs?
- How would you design a CDN for a large-scale video streaming platform?
- How do CDNs help in real-time applications like online gaming or stock trading?



Wrap-Up & Key Takeaways

1. CDNs improve performance by **caching and distributing content** globally.
2. They **reduce latency**, improve load times, and lower bandwidth costs.
3. CDNs enhance security with **DDoS protection**, **SSL/TLS encryption**, and **WAFs**.
4. Use cases include **static content delivery**, **API acceleration**, and **edge computing**.



Next Topic:

Networking Fundamentals

Summary & Best Practices

Summary of Covered Topics – Networking & Communication

1. **Introduction to Networking in System Design**
   Why networking is crucial in distributed systems.
2. **Understanding IP Addresses**
   Differences between IPv4 & IPv6, public vs. private IPs.
3. **How DNS Works**
   Domain Name Resolution, caching, and its role in large-scale systems.
4. **Client-Server Model Explained**
   How clients and servers interact using request-response cycles.
5. **Forward Proxy vs. Reverse Proxy**
   Their differences, use cases, and impact on system design.
6. **Introduction to Load Balancing**
   Introduction and why it's needed.
7. **What is an API Gateway?**
   Managing API traffic, enforcing security, and enabling efficient routing.
8. **Content Delivery Networks (CDN) in System Design**
   How CDNs improve performance and reduce latency.



Next Topic:
Protocols

Client-Server Model: Interview Questions and Answers

---

1. Fundamental Questions

1. What is the client-server model, and how does it work?

- The client-server model is a computing architecture where clients request services, and servers provide them.
- Clients initiate communication, while servers listen for incoming requests and respond accordingly.
- Common protocols used: HTTP, FTP, DNS, SMTP, etc.
- Example: When a user visits a website, the browser (client) requests the webpage from a web server.

2. How does a client communicate with a server?

- Clients communicate with servers via a network (Internet, LAN, Wi-Fi).
- They use standard protocols such as HTTP, TCP/IP, and WebSockets.
- The communication is typically in a request-response format.
- Example: A mobile app sending an API request to a backend server.

3. What are some real-world examples of the client-server model?

- **Web browsing:** Browsers request web pages from web servers.
- **Email services:** Email clients (Gmail, Outlook) communicate with mail servers (SMTP, IMAP, POP3).
- **Streaming services:** Clients (Netflix app) request video streams from media servers.
- **Online gaming:** Players (clients) interact with a game server that manages gameplay.

4. What is the difference between a client and a server?

- **Client:** Requests data or services (e.g., a web browser fetching a webpage).

- **Server:** Processes requests and returns responses (e.g., a web server hosting a website).

- Clients are typically user-facing applications, while servers handle business logic and data processing.

---

2. Request-Response Cycle Questions

5. Explain the HTTP request-response cycle with an example.

- The **HTTP request-response cycle** involves the following steps:

    1. A client (browser) sends an **HTTP request** (e.g., `GET /index.html`).

    2. The request is transmitted over the network to the web server.

    3. The server processes the request (fetches the requested resource, queries a database if needed).

    4. The server sends back an **HTTP response** (status code + requested data).

    5. The browser renders the response (displays the webpage).

- Example: Visiting `https://example.com` triggers an HTTP request to the web server, which responds with the website's HTML, CSS, and JavaScript files.

6. What are the key differences between synchronous and asynchronous communication?

- **Synchronous Communication:**

    - The client sends a request and waits for the server's response before continuing.

    - Example: REST API requests.

- **Asynchronous Communication:**

    - The client sends a request but does not wait for a response before proceeding.

    - Example: WebSockets for real-time messaging, AJAX requests in web applications.

7. How does a browser load a webpage? Walk me through the steps.

1. User enters a URL (e.g., `https://example.com`).
2. The browser queries the **DNS server** to resolve the domain name to an IP address.
3. The browser sends an **HTTP GET request** to the web server.
4. The web server processes the request and retrieves the required resources (HTML, CSS, JavaScript, images).
5. The server sends an **HTTP response** with a status code and the requested data.
6. The browser parses and renders the content for the user.

---

3. Architecture and Design Questions

8. What is the difference between stateless and stateful servers?

- **Stateless Servers:**
    - Do not retain client session data between requests.
    - Example: REST APIs, HTTP servers.
    - Advantage: Scalability and easy load balancing.
- **Stateful Servers:**
    - Maintain session information for clients.
    - Example: WebSockets for chat applications, online banking sessions.
    - Advantage: Personalized and persistent user experiences.

9. How does caching improve performance in a client-server model?

- **Caching** reduces the need for repeated data fetching, improving speed and reducing server load.
- Types of caching:
    - **Browser Cache:** Stores static assets (CSS, JS, images) locally.

    - **CDN (Content Delivery Network):** Caches content geographically closer to users.
    - **Server-side Cache:** Stores frequently accessed database queries or API responses.
    - **Database Cache:** Uses Redis or Memcached to store query results.

10. How do load balancers work in a client-server architecture?

- **Load balancers** distribute incoming client requests across multiple servers.
- They prevent overload on a single server, improving reliability and scalability.
- Load balancing strategies:
    - **Round Robin:** Requests are distributed in a circular manner.
    - **Least Connections:** Requests go to the server with the fewest active connections.
    - **IP Hashing:** Requests from the same IP address go to the same server.

---

4. Advanced Questions

11. What are some security challenges in the client-server model?

- **Man-in-the-Middle (MITM) Attacks:** Data interception; mitigated by HTTPS encryption.
- **DDoS Attacks:** Overloading a server with excessive requests; mitigated by rate limiting and CDNs.
- **SQL Injection:** Malicious database queries; prevented using parameterized queries.
- **Cross-Site Scripting (XSS):** Injecting malicious scripts into web pages.

12. How does WebSockets differ from traditional request-response communication?

- **WebSockets:**
    - Enable **real-time, bidirectional communication** between the client and server.

    - Unlike HTTP, WebSockets keep the connection **open** for continuous data exchange.
    - Used in **chat applications, stock price updates, real-time gaming**.
- **Traditional HTTP:**
    - Follows a request-response cycle.
    - Each request requires a new connection.
    - Better suited for static content retrieval.

13. What are some limitations of the client-server model? How can they be addressed?

- **Single Point of Failure:** A central server can become a bottleneck.
    - Solution: Use load balancing, replication, and failover mechanisms.
- **Scalability Issues:** High traffic can overwhelm the server.
    - Solution: Implement horizontal scaling (adding more servers) and caching strategies.
- **Security Risks:** Centralized data can be a target for attacks.
    - Solution: Implement encryption, authentication, and firewalls.

14. How would you design a scalable client-server system for a high-traffic application?

- **Use Load Balancers** to distribute traffic.
- **Implement Caching (CDN, Redis, Memcached)** to reduce load.
- **Use Microservices** instead of a monolithic architecture.
- **Scale horizontally** by adding more servers.
- **Optimize Database Queries** using indexing and read replicas.

This document provides **detailed answers** formatted for easy conversion into a PDF. Let me know if you'd like any refinements! 🚀

System Design Interview Questions on IP Addresses

1. How do IPv4 and IPv6 addresses differ?

Interviewers ask this to assess understanding of IP addressing. When answering, focus on:

- **Address Length**
    - IPv4: 32-bit (e.g., `192.168.1.1`)
    - IPv6: 128-bit (e.g., `2001:0db8:85a3::8a2e:0370:7334`)
- **Address Format**
    - IPv4: Dotted decimal notation (`x.x.x.x`, where x ranges from 0-255)
    - IPv6: Hexadecimal colon-separated notation (`xxxx:xxxx:xxxx:xxxx:xxxx:xxxx:xxxx:xxxx`)
- **Scalability**
    - IPv6 offers significantly more addresses and supports auto-configuration.
- **Security**
    - IPv6 includes built-in IPSec encryption, unlike IPv4, which requires additional configurations.

Example Response

IPv4 and IPv6 are the two primary internet protocols. IPv4, with a 32-bit address space, is limited, whereas IPv6, with a 128-bit space, offers a larger pool of addresses and better scalability. IPv6 also improves security with built-in encryption.

---

2. Why do we need private IPs in system design?

This question evaluates understanding of network structuring. Key points:

- **Address Conservation**
    - IPv4 addresses are limited; private IPs allow multiple organizations to reuse the same address ranges internally.
- **Security**
    - Private IPs keep internal systems isolated from the public internet, reducing the risk of attacks.
- **Cost Efficiency**
    - Organizations can assign many private IPs while using a single public IP for external communication.

Example Response

Private IPs allow organizations to efficiently manage internal networking without requiring a large number of public IPs. They provide security by isolating internal systems and reducing attack exposure.

---

3. How does NAT help in addressing the IPv4 shortage?

Network Address Translation (NAT) is widely used to extend IPv4 usability. Key points:

- **Address Reuse**
    - NAT allows multiple devices to share a single public IP address.
- **Security**
    - NAT hides internal IP addresses, reducing exposure to external threats.
- **Scalability**
    - NAT enables ISPs and enterprises to manage large networks with fewer public IPs.

Example Response

NAT mitigates IPv4 shortages by enabling multiple devices to share a single public IP. It improves security by masking internal IPs and is widely used in enterprise and ISP networks.

---

4. Explain how a load balancer distributes traffic using IPs.

Load balancers improve system performance by distributing traffic. Key points:

1. **DNS Load Balancing**
    - Multiple IPs are assigned to a domain, and DNS directs traffic based on availability and location.
2. **Layer 4 Load Balancing** (Transport Layer)
    - Traffic is distributed based on IP addresses and ports.
3. **Layer 7 Load Balancing** (Application Layer)
    - Requests are routed based on HTTP headers, URLs, or session data.

Example Response

A load balancer ensures even distribution of network traffic across multiple servers, improving scalability and reliability. It operates at different layers, including DNS-based distribution and TCP/HTTP routing.

---

5. How does DNS resolve IP addresses in a large-scale system?

DNS resolution follows a multi-step process:

1. **Client Queries DNS Resolver**
    - The client requests the IP for a domain (e.g., `example.com`).
2. **Resolver Checks Cache or Queries Root Server**
    - If cached, the IP is returned. Otherwise, the query is forwarded.
3. **TLD Server Responds**
    - The `.com` TLD server points to the authoritative server.
4. **Authoritative DNS Server Provides the IP**
    - The final IP is returned to the client.
5. **Client Connects and Caches the Result**
    - The client stores the IP to avoid repeated lookups.

Example Response

DNS resolution involves multiple steps, including querying root and authoritative servers. Large-scale systems optimize this process using caching and CDNs to reduce latency.

Conclusion

- IPv4 and IPv6 are the primary internet protocols, with IPv6 offering better scalability.
- Private IPs improve security and address conservation in system design.
- NAT enables efficient use of limited IPv4 addresses.
- Load balancers optimize traffic distribution across multiple servers.
- DNS plays a crucial role in resolving domain names to IP addresses, supporting high-scale architectures.

Final Thoughts & Key Takeaways

Best way to answer these questions in an interview:

- **Define the concept** (e.g., what IPv6 is).
- **Explain why it matters** (e.g., solves address exhaustion).
- **Give real-world examples** (e.g., AWS, Google, Netflix).

By mastering these topics, you'll be well-prepared for **technical system design interviews at top tech companies**.

DNS Interview Questions & Answers

1. Explain the DNS resolution process step by step.

DNS resolution is the process of converting a human-readable domain name (e.g., www.example.com) into an IP address. The step-by-step process is as follows:

1. **User Query:** The user types a domain name into a web browser.
2. **Browser Cache Check:** The browser first checks its local cache to see if the domain's IP address is already stored.
3. **Operating System Cache Check:** If the browser cache does not contain the IP address, the query is sent to the operating system's DNS resolver.
4. **Local DNS Resolver Query:** If the OS cache does not have the record, the query is sent to the configured DNS resolver, typically provided by the ISP.
5. **Root DNS Servers:** If the resolver does not have the IP cached, it queries one of the 13 root DNS servers.
6. **TLD (Top-Level Domain) Server Query:** The root server directs the query to the TLD server (e.g., .com, .org, .net servers).
7. **Authoritative DNS Server Query:** The TLD server provides the IP address of the authoritative DNS server for the domain.
8. **IP Address Retrieval:** The authoritative DNS server responds with the correct IP address.
9. **Response Caching:** The resolver caches the result to speed up future queries.
10. **Website Access:** The browser uses the IP address to establish a connection with the web server and loads the website.

---

2. What is the difference between authoritative and recursive DNS servers?

- **Recursive DNS Server:**
    - Acts as an intermediary between the user and other DNS servers.
    - Performs the full DNS lookup process on behalf of the client.
    - Typically provided by ISPs or third-party DNS providers (e.g., Google Public DNS, Cloudflare DNS).
- **Authoritative DNS Server:**
    - Stores actual DNS records (A, CNAME, MX, etc.) for a domain.
    - Provides the final answer in the DNS resolution process.
    - Managed by domain owners or hosting providers.

Recursive servers fetch data, while authoritative servers store and provide the definitive DNS records.

---

3. How does DNS caching improve performance? Where does it occur?

DNS caching reduces the time required to resolve domain names by storing previously retrieved DNS query results. It improves performance by:

- **Reducing latency:** Eliminates the need to repeat DNS queries for frequently accessed websites.
- **Lowering DNS server load:** Fewer queries reduce traffic to upstream DNS servers.
- **Enhancing user experience:** Websites load faster as queries are resolved locally.

Where does DNS caching occur?

- **Browser Cache:** Stores DNS responses for recently visited websites.
- **Operating System Cache:** Maintains a local DNS cache for frequently accessed domains.
- **Recursive Resolver Cache:** DNS servers (e.g., ISP resolvers) cache responses to serve multiple users efficiently.

---

4. What is TTL in DNS, and why is it important?

- **TTL (Time-To-Live)** is a setting in DNS records that defines how long a record is valid before it must be refreshed.
- **Importance of TTL:**
    - A **short TTL (e.g., 60 seconds)** ensures frequent updates but increases query load.
    - A **long TTL (e.g., 24 hours)** reduces query frequency but delays propagation of changes.
    - TTL balances performance and accuracy in DNS resolution.

---

5. How does DNS-based load balancing work in large-scale systems?

DNS-based load balancing distributes traffic across multiple servers by returning different IP addresses for the same domain.

- **Round-Robin DNS:** Each DNS query gets a different server IP in a rotating order.
- **Geolocation-based Routing:** Directs users to the closest data center to minimize latency.
- **Failover DNS:** If a server fails, the DNS resolver removes it from the list and directs traffic to healthy servers.
- **Anycast DNS:** Uses the same IP address across multiple locations, routing requests to the nearest server.

This approach improves reliability, availability, and performance in large-scale applications.

---

6. What are common DNS-related security threats, and how can they be mitigated?

- **DNS Spoofing / Cache Poisoning:** Attackers inject false DNS records, redirecting users to malicious websites.
    - **Mitigation:** Use DNSSEC (Domain Name System Security Extensions) to verify DNS records.
- **DDoS Attacks on DNS Servers:** Attackers overload DNS servers with traffic, making domains unreachable.
    - **Mitigation:** Implement rate limiting, Anycast DNS, and load balancing.
- **Man-in-the-Middle Attacks:** Intercept DNS queries to manipulate responses.
    - **Mitigation:** Use encrypted DNS protocols like DNS-over-HTTPS (DoH) or DNS-over-TLS (DoT).
- **NXDOMAIN Attack:** Attackers flood DNS resolvers with queries for non-existent domains, exhausting server resources.
    - **Mitigation:** Deploy response rate limiting (RRL) and DNS firewalls.

Securing DNS is critical to ensuring safe and reliable internet communication.

Interview questions - TCP and UDP

1. What is the key difference between TCP and UDP?

TCP (Transmission Control Protocol) and UDP (User Datagram Protocol) are both **transport layer protocols**, but they have fundamental differences:

- **TCP** is a **connection-oriented** protocol. It establishes a reliable connection before sending data and ensures that data packets arrive **in the correct order and without loss**.
- **UDP** is a **connectionless** protocol. It **sends data without establishing a connection** and does not guarantee delivery, order, or error correction.

Feature	TCP (Transmission Control Protocol)	UDP (User Datagram Protocol)
Connection	Connection-oriented (handshake required)	Connectionless (no handshake)
Reliability	Reliable (ensures all packets arrive)	Unreliable (some packets may be lost)
Speed	Slower due to error checking & acknowledgments	Faster as it skips reliability checks
Use Cases	Web browsing, file transfer, email	Streaming, gaming, VoIP, DNS lookups

Summary: TCP prioritizes **reliability**, whereas UDP prioritizes **speed**.

---

2. When should you use TCP over UDP?

TCP should be used when **data integrity is critical**, and transmission **reliability is required**.

✅ **Common use cases for TCP:**

- **Web Browsing (HTTP, HTTPS):** Ensures that all webpage data is received without corruption.
- **File Transfers (FTP, SFTP):** Guarantees complete file downloads/uploads without missing chunks.

- **Email Communication (SMTP, IMAP, POP3):** Ensures emails are received in full, without data loss.
- **Database Queries:** Prevents corruption of data when sending/receiving requests and responses.

TCP is essential whenever **missing or corrupted data cannot be tolerated**.

---

3. When should you use UDP over TCP?

UDP is preferred when **speed is more important than reliability**, and **occasional data loss is acceptable**.

✅ **Common use cases for UDP:**

- **Video Streaming (YouTube, Netflix):** A few lost frames don't significantly affect the user experience.
- **Online Gaming (Multiplayer Games):** Low latency is crucial for real-time interactions.
- **VoIP Calls (Skype, Zoom):** Small packet losses are tolerable, but delays should be minimal.
- **DNS Lookups (Domain Name System):** Quick responses are needed, and lost packets can be retried without noticeable delay.

UDP is ideal for **real-time communication** where **low latency matters more than guaranteed delivery**.

---

4. How does TCP ensure reliability?

TCP guarantees reliable communication using the following mechanisms:

- **Three-Way Handshake:** Establishes a connection before data transmission.
- **Acknowledgments (ACKs):** Confirms receipt of packets.
- **Retransmission of Lost Packets:** If a packet is lost, TCP resends it.
- **Error Checking:** Uses checksums to detect corrupted packets.

- **Ordered Data Delivery:** Reassembles packets in the correct order before passing them to the application.

These features ensure that **TCP delivers all data accurately and in sequence**.

---

5. Why is UDP faster than TCP?

UDP is faster because:

🚀 **No connection establishment** – It does not use a handshake like TCP.
🚀 **No acknowledgments** – Data is sent without waiting for confirmation.
🚀 **No retransmission** – Lost packets are **not** resent, reducing delays.
🚀 **Lower overhead** – It has a simpler header (8 bytes vs. TCP's 20 bytes).

Since **UDP doesn't waste time on ensuring reliability**, it delivers data **as quickly as possible**.

---

6. What are the main disadvantages of TCP and UDP?

🚨 **TCP Disadvantages:**

- **Slower performance** due to reliability mechanisms.
- **Higher resource consumption** (CPU and memory usage).
- **Not suitable for real-time applications** (e.g., VoIP, live streaming).

🚨 **UDP Disadvantages:**

- **No guarantee of delivery** – Some packets may be lost.
- **No built-in error correction** – Applications must handle errors manually.
- **Unordered delivery** – Packets may arrive out of sequence.

Each protocol **has trade-offs**, and choosing between TCP and UDP **depends on the application's needs**.

---

7. How do applications handle reliability when using UDP?

Since UDP **doesn't ensure reliable delivery**, applications must implement their own mechanisms:

- **Sequence Numbers:** Ensures packets arrive in the correct order.
- **Custom Acknowledgments:** Applications can request ACKs for important data.
- **Forward Error Correction (FEC):** Adds extra data to recover lost packets.
- **Application-Level Retransmission:** If a packet is lost, the application requests it again.

For example, **online games and video streaming** use **error correction techniques** to reduce the impact of packet loss.

---

8. Can TCP and UDP be used together?

Yes! Some applications **combine TCP and UDP** to optimize performance.

✅ **Examples:**

- **Online Video Streaming (Netflix, YouTube):** Uses UDP for fast video delivery but TCP for control messages (pause, seek, etc.).
- **Online Games:** UDP for real-time actions (player movement) + TCP for non-time-sensitive data (chat, game stats).
- **VoIP (Skype, Zoom):** UDP for low-latency voice data + TCP for connection setup.

By using **both protocols strategically**, applications balance **speed and reliability**.

---

9. Which protocol does DNS use, TCP or UDP?

DNS (Domain Name System) **primarily uses UDP** because:

- **Speed is crucial** – A DNS query must be resolved quickly.
- **Small packet size** – DNS requests/responses fit within UDP's limits.

However, **DNS can use TCP** for **larger responses**, such as when retrieving a full DNS zone transfer.

---

10. How do firewalls handle TCP vs. UDP traffic?

Firewalls manage TCP and UDP traffic differently:

- **TCP traffic** is easier to monitor since it has a clear connection setup (handshake) and termination.

- **UDP traffic** is harder to track because it's connectionless, making it more vulnerable to abuse (e.g., DDoS attacks).

To secure UDP traffic, firewalls use **rate limiting, deep packet inspection, and UDP filtering**.

Interview Questions & Answers: Forward Proxy vs. Reverse Proxy

1. What is a proxy server, and why is it used?

A **proxy server** is an intermediary system that sits between a client (such as a user's browser or device) and a destination server. When a client makes a request, the proxy server forwards it to the destination server, receives the response, and then relays it back to the client.

Why is a Proxy Server Used?

A proxy server is primarily used for:
✅ **Security & Privacy** – It hides the client's or server's identity by masking IP addresses.
✅ **Caching & Performance Optimization** – Frequently requested content is stored and served faster.
✅ **Traffic Control & Load Balancing** – Helps distribute traffic evenly across multiple servers.
✅ **Content Filtering** – Blocks access to restricted or harmful content.
✅ **Compression & Optimization** – Reduces bandwidth consumption.

---

2. Explain the key differences between a forward proxy and a reverse proxy.

🔹 Forward Proxy

- **Sits between clients and the internet** – The client connects to the forward proxy, which then forwards the request to the target server.
- **Used by clients to access external resources** securely or anonymously.
- **Common Use Cases:** Hiding user identity, bypassing geo-restrictions, caching content.
- **Example Tools:** Squid Proxy, Shadowsocks, VPNs, Tor.

🔹 Reverse Proxy

- **Sits between users and backend servers** – The user connects to the reverse proxy, which then forwards the request to an appropriate backend server.
- **Used by servers to manage, secure, and optimize incoming traffic.**
- **Common Use Cases:** Load balancing, caching, SSL termination, security (DDoS protection).
- **Example Tools:** Nginx, HAProxy, Cloudflare, AWS Elastic Load Balancer.

Comparison Table

Feature	Forward Proxy	Reverse Proxy
Position	Between client and internet	Between user and backend servers
Purpose	Client anonymity, access control	Server protection, load balancing
Common Users	End-users, enterprises	Hosting providers, data centers
Example Use	VPNs, anonymous browsing	CDN services, API gateways

---

3. How does a forward proxy improve security and privacy?

A forward proxy enhances security and privacy in several ways:

🔒 **Hiding Client Identity:**

- A forward proxy **masks the client's IP address** so that the target website only sees the proxy's IP.
- This helps in **anonymous browsing** and bypassing geo-blocking restrictions.

🔒 **Encryption of Traffic:**

- VPN-based forward proxies encrypt internet traffic, **preventing ISPs and hackers** from spying on user activities.

🔒 **Content Filtering & Malware Protection:**

- Organizations use forward proxies to **block malicious websites** and **prevent phishing attacks**.

🔒 **Access Control:**

- Companies restrict employee access to specific websites using forward proxies.

---

4. How does a reverse proxy help in load balancing and caching?

A reverse proxy improves system efficiency by handling requests in an optimized way:

⚖️ **Load Balancing:**

- A reverse proxy distributes incoming requests across multiple backend servers, preventing any single server from being overloaded.
- Algorithms used: **Round Robin, Least Connections, IP Hashing.**

🛑 **Caching Content:**

- Frequently requested content (HTML pages, images, videos) is stored in the reverse proxy's cache.
- Reduces **response time** and **server load** by serving cached content instead of fetching it from backend servers.

🔐 **Security & DDoS Protection:**

- Reverse proxies can block **malicious traffic** before it reaches the backend servers.
- Example: **Cloudflare protects websites from DDoS attacks** by filtering traffic at the proxy level.

---

5. What are some real-world examples of forward and reverse proxies?

🔹 Forward Proxy Examples

- **VPN Services (NordVPN, ExpressVPN):** Mask user identity and allow access to blocked websites.
- **Corporate Web Filters:** Companies use forward proxies to restrict access to non-work-related websites.
- **Tor Network:** A forward proxy used for anonymous browsing.

🔹 Reverse Proxy Examples

- **Cloudflare CDN:** Acts as a reverse proxy to optimize website performance and security.
- **Nginx:** Used by companies to handle high web traffic loads efficiently.
- **AWS Elastic Load Balancer:** Distributes incoming requests across multiple cloud servers.

---

6. When should you use a forward proxy vs. a reverse proxy?

Scenario	Use Forward Proxy?	Use Reverse Proxy?
Hide user identity	✅ Yes	❌ No
Load balancing	❌ No	✅ Yes
Security protection for backend servers	❌ No	✅ Yes
Caching content for performance	✅ Yes	✅ Yes
Blocking access to certain websites	✅ Yes	❌ No

✅ **Use a forward proxy** when **clients** need secure access to external resources (e.g., VPNs).
✅ **Use a reverse proxy** when **backend servers** need security, caching, or load balancing.

---

7. What are some common tools and technologies used for each type of proxy?

Type	Common Tools & Technologies
Forward Proxy	Squid Proxy, Shadowsocks, Tor, VPNs
Reverse Proxy	Nginx, HAProxy, Cloudflare, AWS ELB

---

8. How does a reverse proxy protect backend servers from DDoS attacks?

A reverse proxy defends against **Distributed Denial of Service (DDoS) attacks** by:

🛑 **Traffic Filtering:**

- It **analyzes requests** and blocks **malicious IPs and bots** before they reach backend servers.

📊 **Rate Limiting:**

- Limits the number of requests from a single IP to prevent excessive traffic.

🔍 **Anomaly Detection:**

- Uses **AI-based traffic analysis** to detect attack patterns.

🔄 **Load Distribution:**

- Distributes traffic across multiple servers to **prevent overload** during high-traffic attacks.

---

9. How does SSL termination work in a reverse proxy?

SSL termination is the process where **the reverse proxy decrypts HTTPS traffic before forwarding it** to backend servers.

Benefits of SSL Termination:

✅ **Reduces Server Load** – Backend servers don't need to handle SSL decryption.
✅ **Improves Performance** – Faster response times.
✅ **Centralized Security** – Easier to manage SSL certificates at the proxy level.

Example: **Cloudflare handles SSL termination for websites, reducing the burden on origin servers.**

---

10. What are the advantages of using Cloudflare, Nginx, or HAProxy as a reverse proxy?

Tool	Advantages
Cloudflare	DDoS protection, CDN caching, global load balancing.
Nginx	High-performance web server, easy to configure, great for static content caching.
HAProxy	Best for enterprise-grade load balancing, health checks, and high availability.

Final Thoughts

Understanding **Forward Proxy vs. Reverse Proxy** is **essential for system design**. Companies use them for **security, performance, and scalability**.

✅ If you want **privacy** → **Use a Forward Proxy**
✅ If you want **server protection & load balancing** → **Use a Reverse Proxy**

Interview Questions & Answers on REST & RESTful API Design Principles

---

1️⃣ Basic Questions

1. What is REST, and how does it differ from SOAP?

REST (Representational State Transfer) is an architectural style that defines a set of constraints for designing web services. It primarily uses standard HTTP methods and focuses on resources rather than operations.

Differences between REST and SOAP:

- **Protocol:** REST is an architectural style, while SOAP (Simple Object Access Protocol) is a strict protocol.
- **Data Format:** REST typically uses JSON, while SOAP uses XML.
- **Performance:** REST is lightweight and faster than SOAP, which has a higher overhead due to XML formatting and WS-Security.
- **Flexibility:** REST supports multiple data formats (JSON, XML, HTML, etc.), while SOAP is XML-based.
- **Statefulness:** REST is stateless; SOAP can be stateful.

---

2. What are the six constraints of REST architecture?

The six constraints that define REST are:

1. **Client-Server Architecture** – The client and server are independent of each other, allowing for better scalability.
2. **Statelessness** – Each request from the client contains all the necessary information; the server does not store session data.
3. **Cacheability** – Responses can be cached to improve performance and reduce server load.
4. **Layered System** – An API can be designed in layers (e.g., authentication, load balancing, security) without affecting clients.

5. **Uniform Interface** – Standardized communication methods using HTTP verbs (`GET`, `POST`, `PUT`, `DELETE`).

6. **Code on Demand (Optional)** – The server can send executable code (e.g., JavaScript) to the client.

---

3. What is the difference between a REST API and a RESTful API?

- **REST API**: Any API that follows some REST principles but may not strictly adhere to all six constraints.

- **RESTful API**: An API that fully follows all REST constraints and principles.

---

4. What is a resource in REST, and how is it represented?

- A **resource** is any object or entity that can be accessed via the API (e.g., users, orders, products).

- Resources are represented using **URIs** (Uniform Resource Identifiers).

    - Example: `/users/{id}`, `/products/{id}`

---

5. What are endpoints in a REST API?

- An **endpoint** is a specific URL where a client interacts with a resource.

- Example:

    - `GET /users/{id}` – Retrieve user details.

    - `POST /orders` – Create an order.

2️⃣ HTTP Methods & Status Codes

6. Explain the difference between `GET`, `POST`, `PUT`, `PATCH`, and `DELETE`.

- `GET` – Retrieve resource data (safe, idempotent).
- `POST` – Create a new resource (not idempotent).
- `PUT` – Update a resource completely (idempotent).
- `PATCH` – Partially update a resource (not always idempotent).
- `DELETE` – Remove a resource (idempotent).

---

7. When would you use `PUT` vs. `PATCH`?

- **Use `PUT`** when replacing an entire resource (e.g., updating all fields).
- **Use `PATCH`** when making partial updates (e.g., changing only one field).

---

8. What are the commonly used HTTP status codes in REST APIs?

- `200 OK` – Successful request.
- `201 Created` – Resource successfully created.
- `204 No Content` – Successful request, but no response body.
- `400 Bad Request` – Client-side error (e.g., malformed request).
- `401 Unauthorized` – Authentication is required.
- `403 Forbidden` – Client lacks permission.
- `404 Not Found` – Resource does not exist.
- `500 Internal Server Error` – Unexpected server-side issue.

3️⃣ RESTful API Design & Best Practices

9. What are the best practices for designing RESTful APIs?

✅ Use **plural nouns** for resource names (`/users`, not `/user`).
✅ Implement **proper HTTP status codes**.
✅ Support **versioning** (`/v1/resources`).
✅ Use **pagination** for large datasets (`?page=2&limit=20`).
✅ Implement **rate limiting** to prevent abuse.
✅ Use **OAuth or JWT** for authentication.

---

10. How do you design a RESTful API for a blogging platform?

Endpoints could include:

- `GET /posts` – Get all posts
- `POST /posts` – Create a new post
- `GET /posts/{id}` – Get a specific post
- `POST /posts/{id}/comments` – Add a comment

---

11. What is HATEOAS in REST?

HATEOAS (Hypermedia as the Engine of Application State) is a principle where API responses include links to relevant actions.
Example:

json

```
{
  "id": 1,
  "name": "John",
  "links": {
    "self": "/users/1",
    "orders": "/users/1/orders"
  }
}
```

---

12. How do you handle authentication and authorization in REST APIs?

✅ Use **OAuth 2.0** for user authentication.
✅ Use **JWT (JSON Web Token)** for session management.
✅ Implement **API keys** for third-party access.

4️⃣ Advanced & Real-World Questions

13. How does caching work in REST APIs?

- Use HTTP headers:
    - `Cache-Control: max-age=3600` (cache for 1 hour).
    - `ETag` – Versioning for resources.

---

14. How do you implement pagination in REST APIs?

Use query parameters:

- `GET /users?page=2&limit=10`

---

15. How does versioning work in REST APIs?

- URI-based: `/v1/resource`
- Header-based: `Accept-Version: v1`
- Query-based: `?version=1`

---

16. Explain the differences between REST, GraphQL, and gRPC.

Feature	REST	GraphQL	gRPC
Data Format	JSON, XML	JSON	Protocol Buffers

Query Flexibility	Fixed Endpoints	Custom Queries	Strict Methods
Performance	Medium	High	Very High
Use Case	General APIs	Complex Data Fetching	Low-latency Services

---

17. How would you improve the performance of a REST API?

✅ Enable **caching**.
✅ Use **gzip compression**.
✅ Implement **asynchronous processing**.
✅ Use **database indexing**.

---

18. Can a REST API be stateful?

No, REST APIs should be **stateless**. However, some services may use **session-based authentication**, making them stateful.

---

19. How does REST handle security vulnerabilities?

- Prevent **SQL injection** by using parameterized queries.
- Use **CSRF tokens** to prevent cross-site request forgery.
- Implement **HTTPS** for secure communication.

5️⃣ Summary

- REST follows six constraints: **stateless, cacheable, client-server, uniform interface, layered system, code-on-demand (optional)**.
- Use **proper HTTP methods** and **status codes**.
- **Security** (OAuth, JWT) and **performance** (caching, pagination) are crucial.

Interview questions - Beyond REST

1️⃣How would you compare REST, gRPC, and GraphQL?Key Takeaways:

Feature	REST	gRPC	GraphQL
Communication Protocol	HTTP 1.1, HTTP/2	HTTP/2 (Binary)	HTTP (Text-based JSON)
Data Format	JSON, XML	Protocol Buffers (Binary)	JSON
Performance	Slower due to text-based data	High (efficient binary format, multiplexing)	Moderate (can be optimized, but parsing JSON adds overhead)
Flexibility	Fixed endpoints with predefined responses	Strongly typed contracts using `.proto` files	Highly flexible queries with field-level selection
Best for	Public APIs, simple CRUD applications	Microservices, real-time streaming, low-latency systems	Frontend-driven applications, APIs requiring flexible queries
Downsides	Over-fetching / under-fetching issues	Harder to debug due to binary serialization, steep learning curve	Complex caching, query optimization required
Streaming Support	No built-in support	Built-in bidirectional streaming	Requires WebSockets for real-time updates

- **Use REST** for simple, standardized APIs with broad adoption.
- **Use gRPC** for performance-sensitive systems requiring fast communication, especially microservices.
- **Use GraphQL** when clients need flexibility to query only required data.

2️⃣When would you use gRPC over REST?

✅ **When performance and efficiency matter:**

- gRPC uses **binary serialization (Protocol Buffers)**, which is much **faster** and more **efficient** than JSON.
- REST's text-based JSON is larger and slower to parse.

✅ **When you need real-time communication & streaming:**

- gRPC supports **bidirectional streaming** (e.g., continuous updates from a server).
- REST requires **polling**, which is inefficient for real-time applications.

✅ **For microservices communication:**

- gRPC allows direct **service-to-service calls** with **auto-generated client & server code**, making it easier to maintain.
- REST requires manual API calls with HTTP request parsing, which adds overhead.

✅ **For multi-language systems:**

- gRPC supports **automatic code generation** in multiple languages (Java, Python, Go, C++, etc.), making cross-language communication seamless.

✅ **For low-bandwidth environments:**

- gRPC's **compressed binary format** reduces data size, making it efficient for **mobile devices, IoT, and edge computing**.

🔴 **When NOT to use gRPC:**

- **If the API is public-facing** (gRPC is harder to use in web browsers, whereas REST is widely supported).
- **If debugging needs to be simple** (JSON is easier to inspect than Protocol Buffers).

3️⃣ What are the trade-offs of using GraphQL in a large-scale system?

✅ **Advantages:**
✔ **Flexible Queries:** Clients request exactly what they need, reducing over-fetching/under-fetching.
✔ **Efficient Data Fetching:** A single request can aggregate data from multiple sources.
✔ **Strongly Typed Schema:** Well-defined schema helps API evolution.

🔴 **Trade-offs & Challenges:**

❌ **Complex Caching:**

- REST APIs work well with HTTP caching (`Cache-Control`, CDNs).
- **GraphQL responses are dynamic**, making caching harder without additional tools like **DataLoader**.

❌ **Performance Overhead:**

- **Deeply nested queries** can result in expensive database joins.
- **Unoptimized queries** may overload databases.
- Requires **query cost analysis and rate limiting** to prevent abuse.

❌ **Security Concerns:**

- Allows **arbitrary queries**, which can lead to **DoS attacks** if not properly rate-limited.
- Needs **query complexity control** to avoid expensive operations.

❌ **Increased Backend Complexity:**

- Unlike REST (which maps 1:1 to resources), GraphQL **requires resolvers** for each query type, increasing backend development effort.

🚀 **Mitigation Strategies:**

- **Caching Solutions:** Use **Redis**, **CDNs**, and **Apollo Cache**.
- **Query Batching & Rate Limiting:** Limit nested queries, implement query complexity scoring.
- **GraphQL Federation:** Split schema into multiple services for **scalability**.

4️⃣ How does gRPC handle authentication & security?

✅ **Authentication in gRPC:**
1️⃣ **TLS Encryption (Transport Security)**

- gRPC uses **TLS 1.2+** for secure communication (similar to HTTPS).
- Ensures encryption of all requests and responses.

2️⃣ **Authentication Mechanisms:**

- **OAuth 2.0**: Use JWT tokens for authentication.
- **mTLS (Mutual TLS)**: Ensures **both client & server verify each other's identity**.
- **API Keys**: Lightweight authentication for internal services.

✅ **Security Features in gRPC:**
✔ **Token-based Authentication**: Pass JWT or API keys in metadata.
✔ **mTLS (Mutual Authentication)**: Used in microservices for extra security.
✔ **Authorization with RBAC**: Enforce **role-based access control** at the service level.
✔ **Rate Limiting**: Prevent **DoS attacks** by limiting the number of requests.
✔ **Logging & Monitoring**: Use tools like **gRPC interceptors** for monitoring requests.

🔴 **Potential Risks & Solutions:**

- **Man-in-the-middle attacks?** → Use **TLS encryption**.
- **Leaked API keys?** → Rotate and **store in a secure vault (e.g., AWS Secrets Manager)**.
- **DoS Attacks?** → Implement **rate limiting & request validation**.

5️⃣ How do you scale GraphQL APIs efficiently?

✅ **Scaling Strategies for GraphQL:**

1️⃣ **Use GraphQL Federation (Distributed GraphQL Services)**

- Split GraphQL schema into multiple **federated services** instead of a monolithic server.
- Tools: **Apollo Federation**, **Hasura Remote Schemas**.

2️⃣ **Implement Query Caching**

- Use **Apollo Cache**, **Redis**, or **CDN** to store frequently accessed responses.
- Example: **Persisted Queries** cache popular GraphQL queries.

3️⃣ **Optimize Database Queries**

- Use **DataLoader** to batch multiple queries into a **single request**, reducing database calls.
- Convert **deeply nested GraphQL queries** into efficient **SQL joins**.

4️⃣ **Enforce Query Complexity Limits**

- **Prevent expensive queries** by limiting **query depth & execution time**.
- Tools: **GraphQL Shield**, **Persisted Queries**.

5️⃣ **Use Server-Side Pagination & Rate Limiting**

- Implement **cursor-based pagination** instead of fetching large datasets at once.

Example:

```
query {
  users(first: 10, after: "cursor123") {
    edges {
      node { id, name, email }
    }
  }
}
```

-

6️⃣ **Horizontal Scaling (Load Balancers & Clusters)**

- Deploy GraphQL servers behind **load balancers (Nginx, AWS ALB)** to handle more requests.
- Use **containerized deployment** (Kubernetes, Docker) to autoscale GraphQL instances.

🚀 **Best Practices for GraphQL at Scale:**
✔ Use **Apollo Gateway** for federated architecture.
✔ **Cache query results** to reduce database load.

✔ **Rate-limit deep/nested queries** to prevent abuse.
✔ **Monitor query performance** with tools like **GraphQL Metrics** & **Datadog**.

Final Thoughts

✅ **REST is simple, gRPC is fast, GraphQL is flexible**.
✅ **gRPC is best for microservices, GraphQL for frontend-heavy applications**.
✅ **Security & performance tuning is critical** in both gRPC & GraphQL.
✅ **Scaling GraphQL requires caching, pagination, and federation**.

Interview Questions - Real time communication

1️⃣ What is real-time communication, and why is it important?

🔹 **Answer:**
Real-time communication (RTC) refers to **instantaneous data exchange** between systems with minimal latency. Unlike traditional request-response models where data retrieval happens on demand, RTC ensures continuous, live updates without requiring users to manually refresh or trigger requests.

🔹 **Why is it important?**

- **Low Latency:** Enables immediate transmission of information.
- **Seamless User Experience:** No need for manual refreshes or delays.
- **Essential for Critical Applications:** Used in **chat applications, live streaming, stock market updates, online gaming, and IoT systems** where timely data delivery is crucial.

🔹 **Examples:**

- WhatsApp messaging
- Live stock market tickers (NASDAQ, NYSE)
- Multiplayer gaming (Fortnite, Call of Duty)
- Live sports score updates

2️⃣ How does WebSockets work, and how does it differ from traditional HTTP?

🔹 **Answer:**
WebSockets provide a **persistent, full-duplex connection** over a single TCP connection, allowing both the client and server to send data at any time.

🔹 **How WebSockets Work:**
1️⃣ The client sends an **HTTP upgrade request** to the server.
2️⃣ If the server supports WebSockets, it responds with a **101 Switching Protocols** status.
3️⃣ A **persistent connection** is established, eliminating the need for repeated HTTP

requests.
4️⃣ Both client and server can exchange messages **asynchronously** without re-establishing the connection.

🔹 **Differences from Traditional HTTP:**

Feature	WebSockets	Traditional HTTP
Connection	Persistent, full-duplex	Connection closes after each request-response cycle
Latency	Low (data sent instantly)	Higher (requires a new request each time)
Communication	Bi-directional (both client and server can send data)	Client-initiated only
Overhead	Minimal (uses a single connection)	High (repeated requests create overhead)

🔹 **Example:**

- **WebSockets:** Used in live chat apps where messages are pushed in real-time.
- **Traditional HTTP:** Used in blogs or static web pages where data doesn't change frequently.

3️⃣ Explain the WebSocket handshake process.

🔹 **Answer:**
The WebSocket handshake is the **initial request-response process** that upgrades an HTTP connection to a WebSocket connection.

🔹 **Steps in the WebSocket Handshake:**
1️⃣ The **client sends an HTTP request** with an `Upgrade` header requesting a WebSocket connection:

```
GET /chat HTTP/1.1
Host: example.com
Upgrade: websocket
Connection: Upgrade
Sec-WebSocket-Key: x3JJHMbDL1EzLkh9YZrd6w==
Sec-WebSocket-Version: 13
```

2️⃣ The **server responds** with an HTTP 101 Switching Protocols status if it supports WebSockets:

```
HTTP/1.1 101 Switching Protocols
Upgrade: websocket
Connection: Upgrade
Sec-WebSocket-Accept: HSmrc0sMlYUkAGmm5OPpG2HaGWk=
```

3️⃣ After the handshake, the **WebSocket connection remains open**, allowing both the client and server to exchange messages in real-time.

🔹 **Why is the WebSocket Key Used?**

- The `Sec-WebSocket-Key` ensures that the request is not mistakenly interpreted as a standard HTTP request.
- The server hashes this key and sends back the response in `Sec-WebSocket-Accept`.

4️⃣ What is long polling, and how does it work?

🔹 **Answer:**
Long polling is a technique where the **client sends an HTTP request to the server and waits for a response until new data is available**. If there is no new data, the server holds the request open instead of responding immediately.

🔹 **How It Works:**
1️⃣ The **client sends an HTTP request** to the server.
2️⃣ The **server does not immediately respond**. Instead, it holds the request open until there is new data.
3️⃣ Once new data is available, the **server responds**.
4️⃣ The **client immediately sends a new request** to listen for further updates.

🔹 **Example:**

- Gmail uses **long polling** to check for new emails without refreshing the page.

🔹 **Key Differences from WebSockets:**

Feature	WebSockets	Long Polling
Connection	Persistent	Multiple HTTP requests

Efficiency	Low overhead	High overhead due to repeated requests
Latency	Lower	Higher
Best For	High-frequency updates	Less frequent updates

🔹 INTERMEDIATE QUESTIONS

5️⃣ What are the advantages of WebSockets over long polling?

🔹 **Answer:**
✅ **Persistent Connection:** WebSockets keep a **single connection** open, reducing overhead.
✅ **Low Latency:** Data is **pushed in real-time**, unlike long polling where the client waits.
✅ **Efficient:** Avoids unnecessary HTTP headers, reducing **network traffic**.
✅ **Bidirectional Communication:** Both client and server can **send messages** anytime.

🔹 **When WebSockets are better:**

- **Live chat applications** (Slack, WhatsApp)
- **Stock market updates** (NASDAQ)
- **Multiplayer gaming** (Fortnite)

6️⃣ In what scenarios would you prefer long polling over WebSockets?

🔹 **Answer:**
Use **long polling** when:
✅ **WebSockets are not supported** (e.g., older browsers or firewalls blocking WebSockets).
✅ **Limited real-time needs** (e.g., notifications don't require instant updates).
✅ **RESTful API integration** (long polling works well with standard HTTP).
✅ **Simpler backend infrastructure** (no need for a WebSocket server).

🔹 **Example Use Cases:**

- Social media feed updates (Facebook, Twitter)
- Email notifications (Gmail new mail alerts)

7️⃣ How does WebSockets handle connection failures or network interruptions?

🔹 **Answer:**
1️⃣ **Automatic reconnection:** Most WebSocket clients implement **reconnection logic** in case of failure.
2️⃣ **Heartbeat mechanism:** WebSockets use **ping/pong messages** to detect connectivity issues.
3️⃣ **Backoff strategies:** Clients retry connections with **increasing time intervals** to avoid excessive requests.

🔹 **Example Implementation:**

- **Slack WebSockets:** If a user switches between Wi-Fi and mobile data, Slack **automatically reconnects** without losing messages.

8️⃣ Can you use WebSockets with load balancers? If yes, how?

🔹 **Answer:**
Yes, WebSockets can be used with load balancers, but they require **sticky sessions** or **connection-aware load balancing** to maintain state.

🔹 **Techniques:**
✅ **Sticky Sessions:** Ensures that a WebSocket connection stays on the same server.
✅ **Reverse Proxy (NGINX/HAProxy):** Supports WebSocket proxying by **passing the `Upgrade` header**.
✅ **WebSocket Gateways:** Tools like **AWS API Gateway** manage WebSocket connections at scale.

🔹 **Example:**

- Slack uses **AWS Elastic Load Balancer (ELB)** with sticky sessions for WebSockets.

9️⃣ What are some challenges of scaling WebSockets in a distributed system?

🔹 **Answer:**
⚠️ **Maintaining state across multiple servers** (requires sticky sessions or session persistence).
⚠️ **Handling large concurrent connections** (millions of users).

⚠ **Load balancing issues** (WebSockets require intelligent routing).
⚠ **Handling connection failures** (requires reconnection logic).

🔹 **Solution:**

- Use **Redis Pub/Sub** or **Kafka** for **message broadcasting across multiple WebSocket servers**.
- Implement **sticky sessions** in load balancers.
- Use **WebSocket gateways** like AWS API Gateway for scalability.

CORS & Web Security – Interview Questions & Answers

1️⃣ What is the Same-Origin Policy, and why does it exist?

✅ **Answer:**

The **Same-Origin Policy (SOP)** is a security mechanism enforced by web browsers that prevents scripts running on one origin (protocol + domain + port) from interacting with resources on a different origin.

🔹 **Why it exists?**

- Prevents malicious websites from making unauthorized requests on behalf of a user.
- Protects sensitive user data, such as authentication tokens and session cookies.
- Helps mitigate cross-site attacks like **Cross-Site Request Forgery (CSRF)**.

2️⃣ How does CORS enable cross-origin requests?

✅ **Answer:**

CORS (Cross-Origin Resource Sharing) is a browser security feature that allows controlled access to resources from a different origin by using HTTP headers.

- When a browser makes a **cross-origin request**, the server must send **CORS headers** in its response to indicate whether the request is allowed.

The most important header is:

```
Access-Control-Allow-Origin: https://example.com
```

- This header tells the browser that `example.com` is allowed to access the resource.

3️⃣ What is a preflight request, and when is it required?

✅ **Answer:**

A **preflight request** is an **OPTIONS** request that browsers send **before** making a cross-origin request to check if the actual request is permitted.

🔹 **When is it required?**

- When the request **uses HTTP methods other than GET, HEAD, or POST** (e.g., PUT, DELETE).
- When the request **includes custom headers** (e.g., `Authorization`, `X-Custom-Header`).
- When the request **sends non-simple content types**, such as `application/json`.

🔹 **Example Flow:**

1️⃣ Browser sends an **OPTIONS** request.
2️⃣ Server responds with allowed methods and headers.
3️⃣ If approved, browser makes the actual request.

Example response from server:

```
Access-Control-Allow-Origin: https://example.com
Access-Control-Allow-Methods: GET, POST, PUT
Access-Control-Allow-Headers: Authorization, Content-Type
```

4️⃣ How do you configure CORS headers on a server?

✅ **Answer:**

To configure CORS, the server must return proper **CORS headers** in its response.

🔹 **Example in Express.js (Node.js)**

```
app.use((req, res, next) => {
  res.setHeader('Access-Control-Allow-Origin', '*'); // Allow all
origins
  res.setHeader('Access-Control-Allow-Methods', 'GET, POST, PUT,
DELETE');
  res.setHeader('Access-Control-Allow-Headers', 'Content-Type,
Authorization');
  next();
});
```

- `Access-Control-Allow-Origin`: Specifies which domains can access the resource.

- `Access-Control-Allow-Methods`: Defines allowed HTTP methods.
- `Access-Control-Allow-Headers`: Lists allowed custom headers.

🔹 **In Nginx Configuration:**

```
add_header 'Access-Control-Allow-Origin' '*';
add_header 'Access-Control-Allow-Methods' 'GET, POST, OPTIONS';
add_header 'Access-Control-Allow-Headers' 'Authorization,
Content-Type';
```

This allows **all origins** (`*`), but it can be restricted to specific domains for better security.

5️⃣ What are common security risks associated with CORS?

✅ **Answer:**

While CORS is essential for enabling cross-origin requests, improper configuration can lead to security risks:

- **Overly permissive CORS settings (`Access-Control-Allow-Origin: *`)**
    - Allows any website to access the resource, increasing the risk of data leaks.
- **Allowing credentials without proper restrictions (`Access-Control-Allow-Credentials: true`)**
    - Can expose sensitive user sessions to unauthorized third-party sites.
- **Exposing sensitive HTTP headers (`Access-Control-Allow-Headers`)**
    - Misconfiguring this can expose private API keys, authentication tokens, or user data.

🔹 **How to mitigate risks?**

✅ Restrict origins to trusted domains.
✅ Avoid `Access-Control-Allow-Origin: *` when credentials are involved.
✅ Implement **proper authentication & authorization** mechanisms.

6️⃣ What are alternatives to CORS for handling cross-origin requests?

✅ **Answer:**

Instead of CORS, some architectures use:

1️⃣ **Reverse Proxy:**

- A backend server (e.g., Nginx) acts as an intermediary, making requests on behalf of the client.
- The client interacts only with the proxy, eliminating the need for CORS.

Example in Nginx:

```
location /api/ {
  proxy_pass http://backend-server.com/;
}
```

-

2️⃣ **JSONP (JSON with Padding) [Deprecated]**

- Used before CORS was widely supported.
- Works by dynamically loading a `<script>` tag instead of using AJAX.
- **Not secure**, as it exposes APIs to cross-site attacks.

3️⃣ **API Gateway Handling CORS**

- API Gateways (e.g., AWS API Gateway, Kong) can **manage CORS policies centrally**, ensuring secure and consistent configurations.

7️⃣ How do API Gateways and Reverse Proxies help with CORS?

✅ **Answer:**

🔹 **Reverse Proxies:**

- The **client requests the proxy**, which then forwards the request to the backend.

- Since the client and proxy are on the **same origin**, **CORS restrictions do not apply**.

Example setup in Nginx:

```
location /api/ {
  proxy_pass http://backend-service.com/;
}
```

-

🔹 **API Gateways:**

- API Gateways **control CORS policies** across multiple backend services.
- They allow **fine-grained control** over which origins, methods, and headers are permitted.
- Example in AWS API Gateway:
    - Configure **CORS settings** in API Gateway to allow specific origins.

✅ **Benefits:**
✔ Reduces **security risks** from misconfigured CORS headers.
✔ **Centralized control** over access policies.
✔ Improves **performance & request handling**.

API Gateway - Interview Questions & Detailed Answers

---

1. What is an API Gateway, and why is it used?
An API Gateway is a server that acts as an intermediary between clients and backend services. It handles request routing, authentication, rate limiting, caching, and response transformation. It is used to manage API traffic efficiently, improve security, reduce backend load, and ensure scalability.

---

2. How does an API Gateway differ from a Load Balancer?
An API Gateway provides additional features beyond load balancing, such as authentication, rate limiting, caching, and API composition. A Load Balancer simply distributes network traffic among multiple backend servers to improve availability and scalability. While Load Balancers operate at the network or transport layer (Layer 4/7), API Gateways operate at the application layer (Layer 7), enabling more advanced request handling.

---

3. What are the key benefits of using an API Gateway?

- **Security:** Protects backend services by enforcing authentication and authorization.
- **Rate Limiting & Throttling:** Controls traffic to prevent abuse and maintain system stability.
- **Load Balancing:** Distributes requests across multiple services to improve scalability.
- **Caching:** Reduces backend load and speeds up response times.
- **Request Transformation:** Converts API requests between different formats (e.g., JSON to XML).
- **Monitoring & Logging:** Tracks API performance, usage, and security threats.

---

4. How does an API Gateway handle authentication and authorization?
API Gateways enforce security through authentication (verifying user identity) and authorization (checking user permissions). Common methods include:

- **API Keys:** Unique identifiers required for API access.
- **OAuth 2.0 & JWT (JSON Web Tokens):** Used for secure access management.

- **mTLS (Mutual TLS):** Ensures encrypted communication between clients and APIs.
- **LDAP & SAML:** Used for enterprise authentication.

When a request reaches the API Gateway, it verifies the credentials/token before forwarding the request to backend services. If authentication fails, the request is denied.

---

5. Explain rate limiting and throttling in API Gateways.
Rate limiting and throttling control how many requests a client can make within a specified time to prevent API abuse.

- **Rate Limiting:** Restricts the number of API calls per user/IP per second, minute, or hour. Example: A user can make only 100 requests per minute.
- **Throttling:** Allows excess requests but slows down response times instead of rejecting them immediately. Useful for handling high-traffic scenarios.
- **Burst Limits:** Temporary high limits that adjust dynamically based on usage patterns.

Common rate-limiting algorithms include:

- **Token Bucket Algorithm:** Requests consume tokens from a fixed bucket; new tokens are added periodically.
- **Leaky Bucket Algorithm:** Requests are processed at a fixed rate, preventing traffic spikes.
- **Fixed Window & Sliding Window Counters:** Limit requests per time window.

---

6. What caching strategies can be implemented in an API Gateway?
Caching improves performance by storing frequently accessed responses to reduce redundant backend calls. Common caching strategies include:

- **In-memory Caching:** Stores responses in memory (e.g., Redis, Memcached).
- **Response Caching:** Stores API responses and serves cached results for repeated requests.
- **Edge Caching (CDN):** Uses Content Delivery Networks to cache API responses globally.

- **Per-Route Caching:** Different endpoints have different caching rules (e.g., caching GET but not POST requests).

- **Time-to-Live (TTL):** Sets expiration times for cached responses to ensure freshness.

Example: A product details API can cache responses for 5 minutes to reduce database queries.

---

7. How does an API Gateway improve security against DDoS attacks?
An API Gateway protects against Distributed Denial of Service (DDoS) attacks by:

- **Rate Limiting & Throttling:** Prevents excessive requests from overwhelming backend services.

- **IP Whitelisting & Blacklisting:** Blocks requests from suspicious or unauthorized IP addresses.

- **Web Application Firewall (WAF) Integration:** Filters malicious traffic based on predefined security rules.

- **Bot Detection & CAPTCHA:** Identifies automated bots and requires human verification.

- **TLS Termination:** Encrypts and decrypts traffic to prevent man-in-the-middle attacks.

Example: If an attacker floods an API with millions of requests per second, the API Gateway enforces rate limiting to block excess traffic.

---

8. When should you use an API Gateway in a microservices architecture?
An API Gateway is useful in microservices architectures when:

- **Multiple services need a unified entry point.** Instead of clients calling each service directly, an API Gateway routes requests efficiently.

- **Security and authentication are required.** API Gateways enforce access controls to protect microservices.

- **Rate limiting and caching are necessary.** This ensures performance and system stability.

- **Request transformation is needed.** API Gateways can convert between formats like REST, GraphQL, and gRPC.

- **API monitoring and analytics are important.** It enables logging, tracing, and monitoring API usage.

Example: In an e-commerce system, an API Gateway can route requests to separate microservices for authentication, product catalog, and payments.

---

9. How would you design an API Gateway for a large-scale system with millions of users?
For a high-traffic system, an API Gateway must be designed for **scalability, reliability, and security**:

- **Deploy in a Distributed Architecture:** Use multiple API Gateway instances behind a Load Balancer to handle traffic spikes.

- **Enable Auto-Scaling:** Dynamically adjust resources based on demand (e.g., Kubernetes-based deployment).

- **Use Rate Limiting & Throttling:** Prevent excessive API calls from overloading the system.

- **Implement Caching:** Reduce backend workload by caching frequently requested responses.

- **Ensure High Availability:** Use multi-region deployment with failover mechanisms.

- **Monitor & Log API Traffic:** Track errors, response times, and security threats with observability tools.

Example: A **global video streaming service** (like Netflix) would use a CDN-enabled API Gateway to route requests to the nearest data center, improving latency.

---

10. What challenges might arise when implementing an API Gateway, and how would you address them?

- **Single Point of Failure:** API Gateway failure can disrupt all API traffic. Solution: Deploy multiple API Gateway instances with failover support.

- **Increased Latency:** Extra processing (authentication, transformation, logging) can slow down requests. Solution: Optimize configurations, enable caching, and minimize

request overhead.

- **Complexity in Microservices:** Managing API routes, security policies, and monitoring can be challenging. Solution: Use an API Gateway management platform (e.g., Kong, Apigee).

- **Scalability Issues:** A poorly designed API Gateway can become a performance bottleneck. Solution: Use horizontal scaling with distributed API Gateway nodes.

- **Versioning & Backward Compatibility:** Managing API versions can be complex. Solution: Implement versioning strategies (e.g., `/v1/resource`, `/v2/resource`).

Example: If an **API Gateway becomes a bottleneck in an IoT system**, scaling it with a **Kubernetes-based deployment** ensures high availability.

CDN System Design Interview Questions & Answers

Comprehensive Guide

Table of Contents

1. **CDN Basics**
2. **CDN Architecture & Components**
3. **Caching Strategies & Content Delivery**
4. **Load Balancing & Failover Handling**
5. **CDN Optimization Techniques**
6. **CDN Security & Challenges**
7. **Advanced CDN Topics**

1️⃣ CDN Basics

1. What is a CDN, and how does it work?

A **Content Delivery Network (CDN)** is a **distributed network of servers** that work together to deliver content to users efficiently. The primary goal of a CDN is to **reduce latency and load times** by caching content in multiple geographically distributed servers called **edge servers** or **PoPs (Points of Presence)**.

How It Works:

1. A user requests content (e.g., an image, video, or webpage).
2. The request is directed to the nearest CDN **edge server** based on factors like **geographic location, network latency, and server load**.
3. If the requested content is already **cached** on the edge server (**Cache Hit**), it is delivered instantly.

4. If the content is not cached (**Cache Miss**), the request is forwarded to the **origin server**, fetched, and stored at the edge server for future requests.

---

2. Why do we need CDNs in system design?

Without a CDN, content is served directly from the **origin server**, which leads to:

- **High latency** due to geographic distance between the user and the server.
- **Overloaded origin servers**, causing slower responses and downtime.
- **Bandwidth constraints**, leading to slow page load times and higher operational costs.
- **Security risks**, including DDoS attacks and malicious traffic.

CDNs **solve these problems** by **distributing** traffic across multiple edge locations, **caching frequently accessed content**, and **protecting against cyber threats**.

---

3. What are the key benefits of using a CDN?

✅ **Reduced Latency** – Faster content delivery by serving from the closest PoP.
✅ **Lower Bandwidth Costs** – Caching reduces requests to the origin, minimizing data transfer costs.
✅ **Increased Availability & Load Balancing** – Distributes traffic across multiple servers, preventing overload.
✅ **Enhanced Security** – Protects against **DDoS attacks**, **traffic filtering**, and **SSL/TLS encryption**.

2️⃣ CDN Architecture & Components

4. Explain the difference between an origin server and an edge server in a CDN.

- **Origin Server:** The central server that hosts the original content. It is responsible for serving content **when a cache miss occurs**.
- **Edge Server (PoP):** A geographically distributed server that caches content closer to users to reduce latency.

💡 **Analogy:** The **origin server** is like a **main warehouse**, while **edge servers** are **local distribution centers** that store frequently accessed goods.

---

5. What is a PoP (Point of Presence) in a CDN?

A **PoP (Point of Presence)** is a **CDN data center** that contains **edge servers** to store and serve cached content to users. The more PoPs a CDN has, the **faster and more reliable** the content delivery.

---

6. How does request routing work in a CDN?

CDNs use various **routing strategies** to direct user requests to the optimal edge server:

- **Geo-Based Routing:** Users are directed to the closest PoP.
- **Latency-Based Routing:** Requests go to the PoP with the lowest network latency.
- **Load-Aware Routing:** Traffic is balanced across multiple PoPs to prevent overload.

3️⃣ Caching Strategies & Content Delivery

7. What is a cache hit vs. cache miss, and how does a CDN handle them?

- **Cache Hit:** The requested content is found in the CDN cache and served immediately.
- **Cache Miss:** The content is not in the cache, so it is fetched from the origin, stored at the edge, and then served to the user.

---

8. Explain cache expiration and TTL (Time-To-Live) in a CDN.

- **TTL (Time-To-Live):** Defines how long content stays cached before it expires.
- **CDN Cache Expiration:** Once TTL expires, the CDN fetches updated content from the origin.

---

9. What are cache invalidation strategies, and why are they important?

- **Manual Purge:** Manually removes outdated content.
- **Stale-While-Revalidate:** Serves stale content while fetching fresh content in the background.
- **Versioning:** Appends version numbers (e.g., `image_v2.png`) to force cache updates.

4️⃣ Load Balancing & Failover Handling

10. How do CDNs use load balancing to improve reliability?

CDNs distribute traffic across multiple PoPs using:
✅ **Round-robin balancing**
✅ **Latency-based routing**
✅ **Geo-based routing**

---

11. What happens if a CDN PoP fails?

The CDN automatically **reroutes traffic** to the next best PoP, ensuring uninterrupted service.

5️⃣ CDN Optimization Techniques

12. What compression and minification techniques do CDNs use?

CDNs optimize content delivery using:

- **Gzip & Brotli Compression** (reduces file size).
- **Minification of CSS/JS** (removes unnecessary characters).
- **Image Optimization** (WebP, AVIF formats).

6️⃣ CDN Security & Challenges

13. How does a CDN protect against DDoS attacks?

- **Rate Limiting:** Blocks excessive requests from a single source.
- **Traffic Filtering:** Identifies and drops malicious requests.
- **Anycast Routing:** Distributes attack traffic across multiple PoPs to absorb the impact.

---

14. What is SSL/TLS offloading, and why is it useful?

SSL/TLS Offloading means the CDN **handles encryption** at the edge, reducing the burden on the origin server.

7️⃣ Advanced CDN Topics

15. How does a multi-CDN architecture work?

A **multi-CDN strategy** uses multiple CDN providers for **better redundancy, performance, and failover handling**.

---

16. How would you design a CDN for a large-scale video streaming platform?

Key design considerations:

- **Segmented Caching** (store different video chunks at different PoPs).
- **Adaptive Bitrate Streaming** (adjusts video quality based on user bandwidth).
- **Load Balancing** (distributes viewers across multiple servers).

Final Thoughts

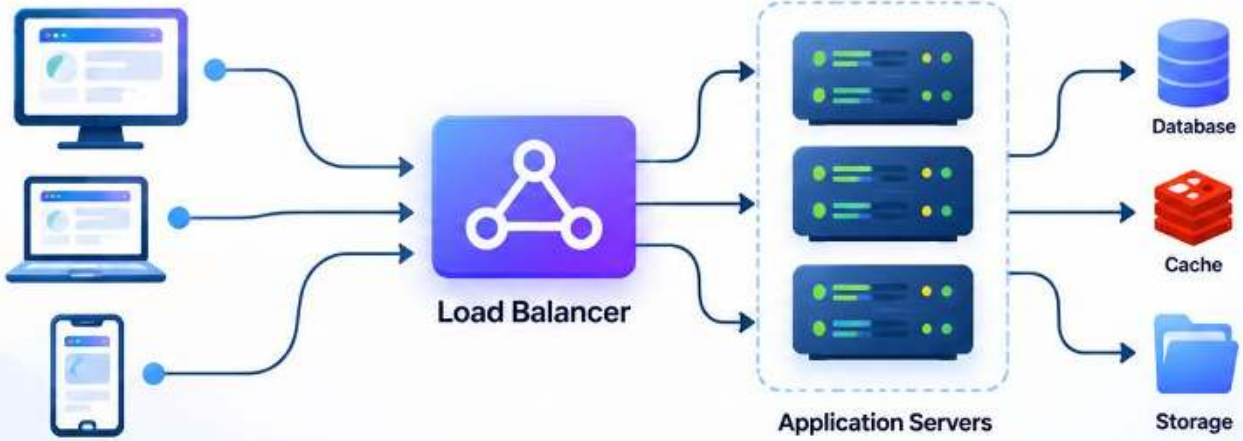
Understanding **CDN architecture, caching, security, and optimization techniques** will help you design **high-performance, scalable systems**.

💡 **Pro Tip:** Always think about **latency, redundancy, and scalability** when discussing CDNs in system design interviews!

Introduction to Scalability

Scalability in System Design

Mastering System Design: From Basics to Cracking Interviews





Instructor: Rahul Rajat Singh

What is Scalability?

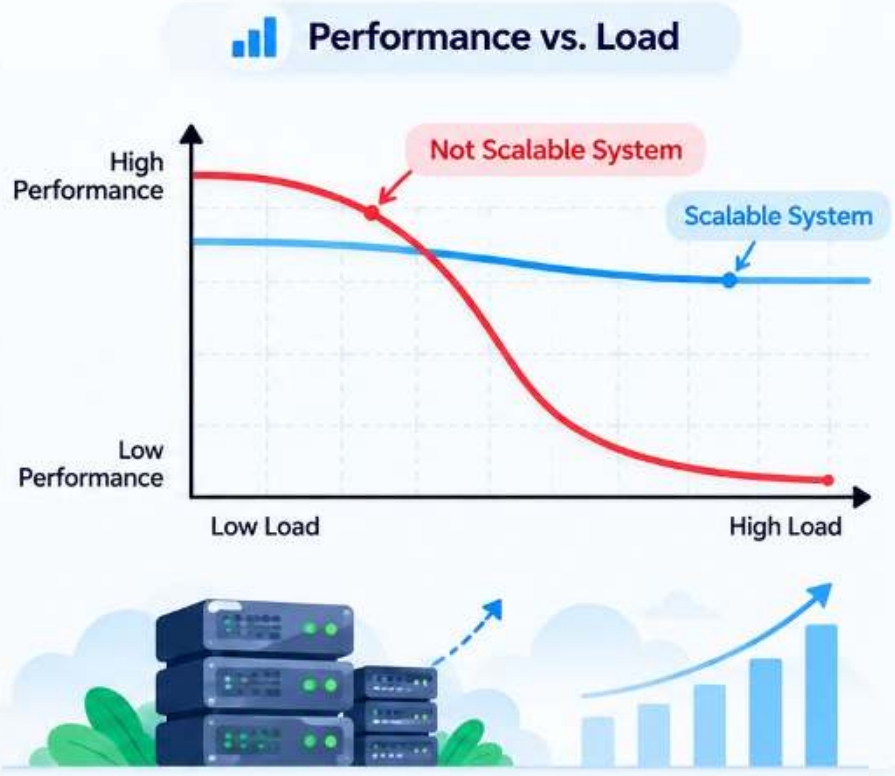
Scalability is the ability of a system to handle an increasing amount of work, or its potential to accommodate growth.

It ensures **performance, reliability**, and **availability** under growing load.



Why Do Systems Need to Scale?

1. **User base growth**
   e.g., launching in new regions
2. **Increasing data volume**
   e.g., IoT, analytics
3. **Peak events**
   e.g., Black Friday, ticket sales
4. **Avoid service degradation**
   or downtime
5. **Meet performance SLAs**



Types of Scalability (Intro)

1 Vertical Scaling

Add more CPU/RAM to one server.



2 Horizontal Scaling

Add more servers to distribute the load.



We'll go deep into these types in the next lecture.

Common Challenges in Scaling

Latency

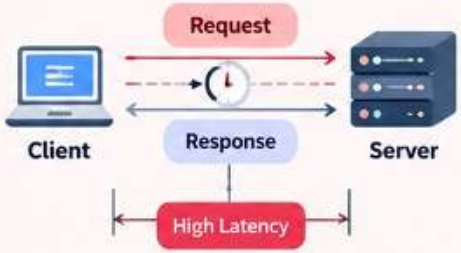
Delay between request and response.

Common Causes

- Network hops
- Slow database queries
- Synchronous calls

Common In

- Amplified in microservices and distributed systems



Bottlenecks

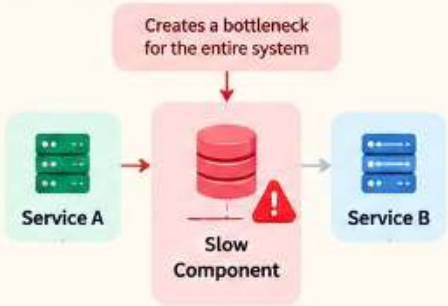
One slow component can cause system-wide slowdown.

Examples

- Database locks
- Memory limits
- Single-threaded processing

Challenge

- Hard to predict as load grows



Downtime

More nodes and components mean more failure points.

Common Causes

- More nodes = more failure points
- Updates, redeployment
- scaling events can cause outages

Impact

- High availability becomes harder



Cost

Infrastructure isn't free — scaling has real costs.

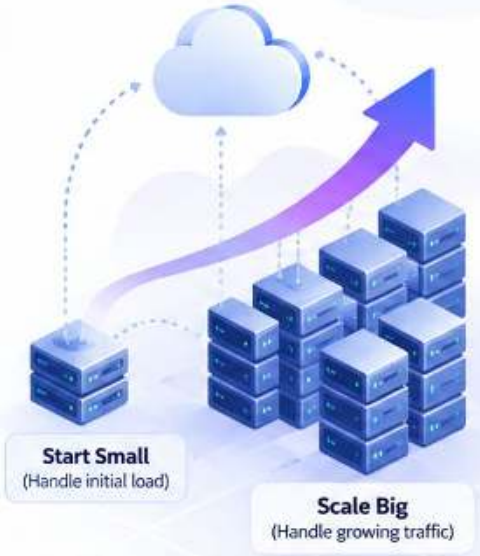
Cost Factors

- CPU, RAM, bandwidth, etc.
- Autoscaling without limits = budget nightmare
- Over-provisioning = wasted spend



Interview Questions - Scaling

1. What does **scalability** mean in the context of system design?
2. Can you explain a **real-world example** where scalability was critical to success or failure?
3. What are the main **challenges** systems face as they scale?
4. How would you identify a **bottleneck** in a scalable architecture?
5. Why does **latency** increase with scale, and how can you mitigate it?
6. How do you balance scalability with **cost** in cloud-based systems?



Think beyond adding servers: identify bottlenecks, latency, reliability, and cost.

Instructor: Rahul Rajat Singh

Summary & What's Next

1. **Scalability** helps systems **grow** without breaking.
2. **Two main types**: **vertical** and **horizontal**.
3. But it introduces **latency**, **cost**, and **other risks**.

What's Next?

Scaling Strategies — how to actually implement horizontal, vertical, and diagonal scaling with trade-offs and real-world choices.



Scaling Strategies: Horizontal, Vertical & Diagonal

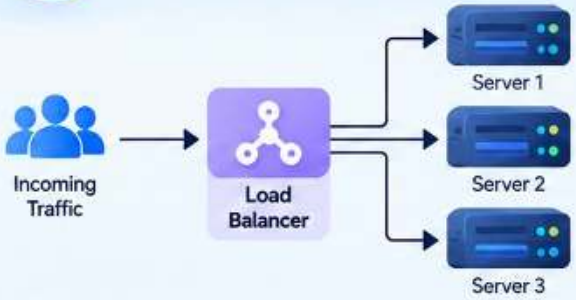
Scalability in System Design

Mastering System Design: From Basics to Cracking Interviews

Rahul Rajat Singh

Horizontal Scaling

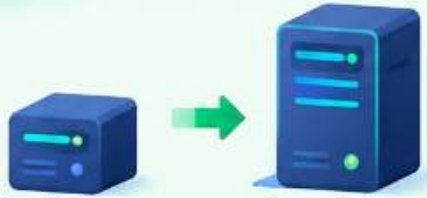
Add more machines (instances)



- Handles increased traffic by adding more servers.
- Common in web applications and microservices (behind a load balancer).

Vertical Scaling

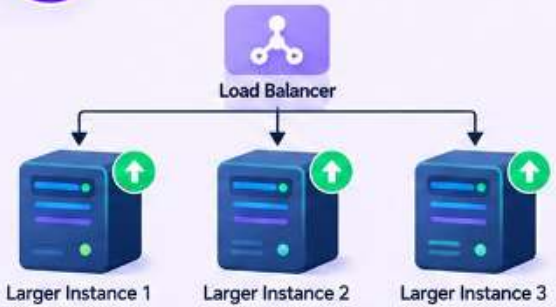
Add more power to a single machine



- Increase CPU, RAM, storage on the same machine.
- Simple to implement, but has hardware limits.

Diagonal Scaling

Combine vertical + horizontal scaling



- Add more powerful machines and more machines.
- Best for large-scale, high-traffic systems.

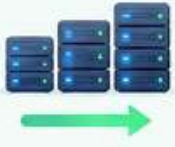
Types of Scalability – Deep Dive

1 Vertical Scaling (Scale Up)



- Upgrade one machine (CPU, RAM, Disk)
- Easy to implement (less complexity)
- Limits: Physical cap, risk of single point of failure

2 Horizontal Scaling (Scale Out)



- Add more machines/nodes
- Distribute traffic/load across servers
- Requires load balancer, stateless design
- Complex setup (coordination, replication)

3 Diagonal Scaling (Hybrid)



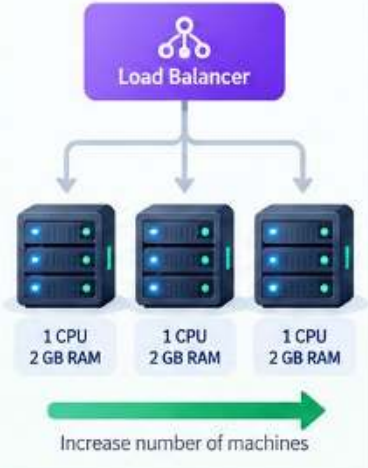
- Start vertical, then go horizontal
- Hybrid approach: start vertical, add horizontal as needed
- Common in cloud-native apps
- Cost-effective + long-term ready

VERTICAL SCALING
Scale Up



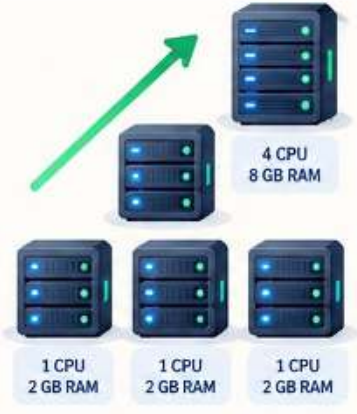
- More power in one server
- Simple to implement
- Limited by hardware
- Single point of failure risk

HORIZONTAL SCALING
Scale Out



- Add more machines
- Distribute traffic/load
- Requires load balancer
- Stateless design
- Complex setup (coordination)

DIAGONAL SCALING
Scale Up + Scale Out



- Start vertical, then go horizontal
- Hybrid approach: add both as needed
- Common in cloud-native apps
- Cost-effective + long-term ready

Trade-Offs: Cost vs Complexity vs Performance

1 Vertical Scaling

Increase the capacity of a single server (more CPU, RAM, storage).



Cost	Low-Mid
Complexity	Low
Performance	Medium

2 Horizontal Scaling

Add more servers to handle the load (scale out).



Cost	High
Complexity	High
Performance	High

3 Diagonal Scaling

Combine vertical and horizontal scaling (bigger servers + more servers).



Cost	Medium
Complexity	Medium
Performance	High

Key Takeaway More performance usually means more **cost** or **complexity**.

Real-World Examples and When to Choose What?

Examples

Twitter
Moved from monolith → horizontal scaling (microservices)

Small startups
Vertical scaling for MVP

AWS Lambda apps
Start diagonal with autoscaling

When to Choose

Startups
Vertical (cheaper, simpler)

Scaling apps
Horizontal (resilience + capacity)

Cloud-native
Diagonal (flexibility + cost balance)

Vertical Scaling

Increase the power of a single server (better CPU, RAM, storage)



Horizontal Scaling

Add more servers to handle higher load (resilience + capacity)



Diagonal Scaling (Cloud-native)

Combine vertical and horizontal scaling with autoscaling



Decision Guide

- Consider your **stage** (MVP or scale)
- Think about **resilience** needs
- Evaluate **flexibility**
- Balance with **cost**

Interview Questions - Scaling Strategies

1. What is the difference between **horizontal** and **vertical** scaling?
2. What is **diagonal scaling** and when is it a good idea?
3. What are the **trade-offs** between horizontal and vertical scaling in terms of performance and complexity?
4. Can you describe a **scenario** where horizontal scaling wouldn't help?
5. When would you choose **vertical scaling** over horizontal?
6. What **challenges** arise in horizontal scaling and how would you solve them?



Summary & What's Next

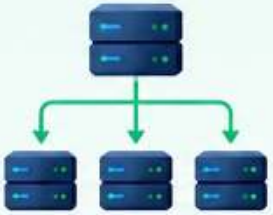
Scaling Strategies

Vertical Scaling



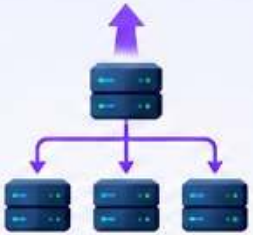
Simpler, but hits limits

Horizontal Scaling



Scalable, but needs planning

Diagonal Scaling



Hybrid approach for cloud success

What's Next?

Understanding Load Balancers:
Types, Algorithms & Cloud Solutions



Understanding Load Balancers

Scalability in System Design

Mastering System Design: From Basics to Cracking Interviews





Rahul Rajat Singh

Why Load Balancing is Needed

High Availability
Ensures system uptime even under high traffic.

Traffic Distribution
Spreads requests evenly across servers.

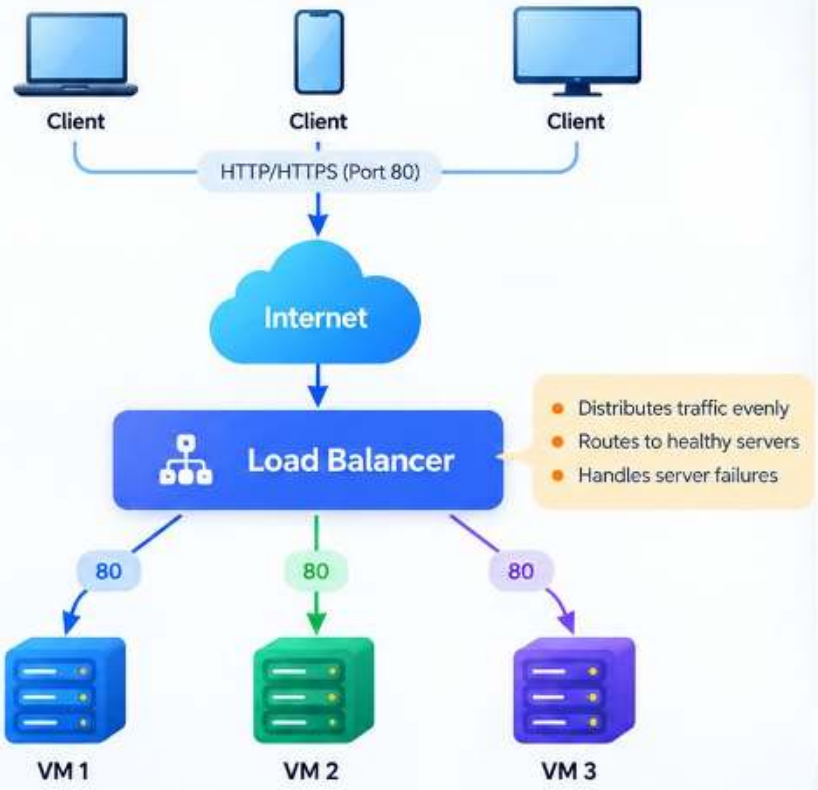
Prevents Overload
Avoids overburdening a single server.

Improves Performance
Reduces latency and enhances response times.

Handles Failures Gracefully
Redirects traffic in case of server failure.

Supports Scalability
Helps scale systems efficiently.

Real-World Example
A high-traffic e-commerce site uses load balancing to handle peak-hour requests.



Types of Load Balancers

Based on Layer

L4 TCP/UDP

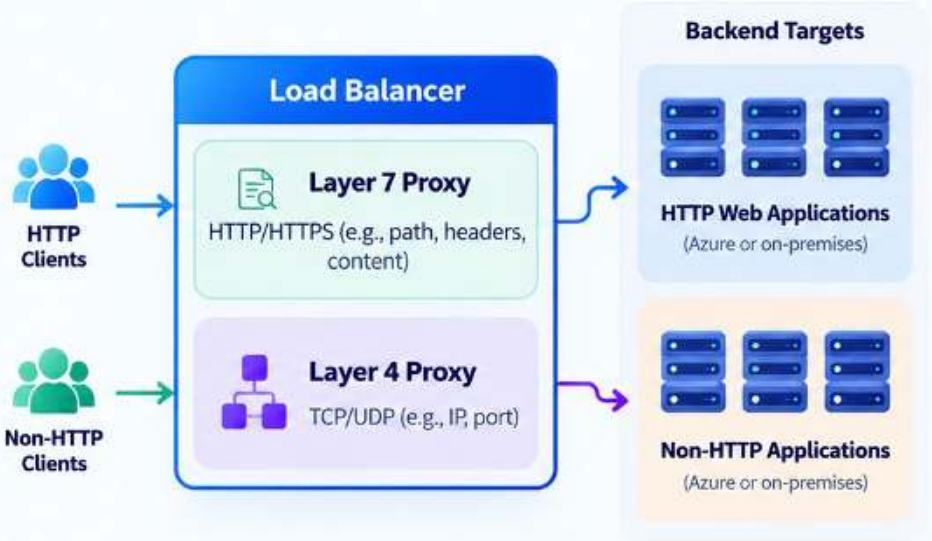
1. **Layer 4 (Transport Layer)**
   Operates at TCP/UDP level, distributing requests based on network-level data.

L7 HTTP/HTTPS

2. **Layer 7 (Application Layer)**
   Operates at HTTP/HTTPS level, making routing decisions based on request content.

Based on Deployment

1. **Hardware Load Balancers**
   Specialized devices (e.g., F5, Citrix NetScaler).
2. **Software Load Balancers**
   Nginx, HAProxy, Envoy.
3. **Cloud-based Load Balancers**
   AWS Elastic Load Balancer (ELB), Google Cloud Load Balancing.



Key Takeaway

Choose the right load balancer type and deployment model based on your application's protocol, routing needs, and scale.

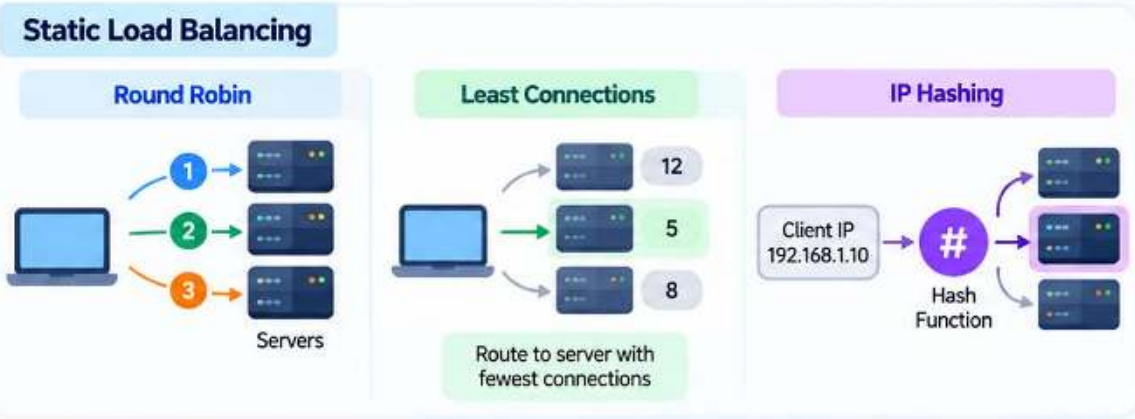
Load Balancing Strategies

1 Static Load Balancing

Round Robin
Distributes requests sequentially to each server.

Least Connections
Directs traffic to the server with the fewest connections.

IP Hashing
Routes requests based on client IP.

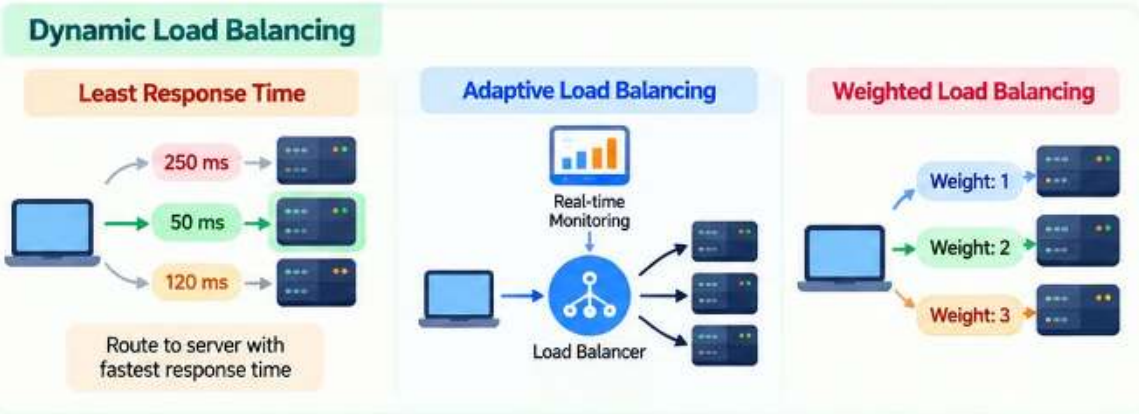


2 Dynamic Load Balancing

Least Response Time
Sends requests to the server with the fastest response.

Adaptive Load Balancing
Uses real-time monitoring to make decisions.

Weighted Load Balancing
Assigns different weights to servers based on capacity.

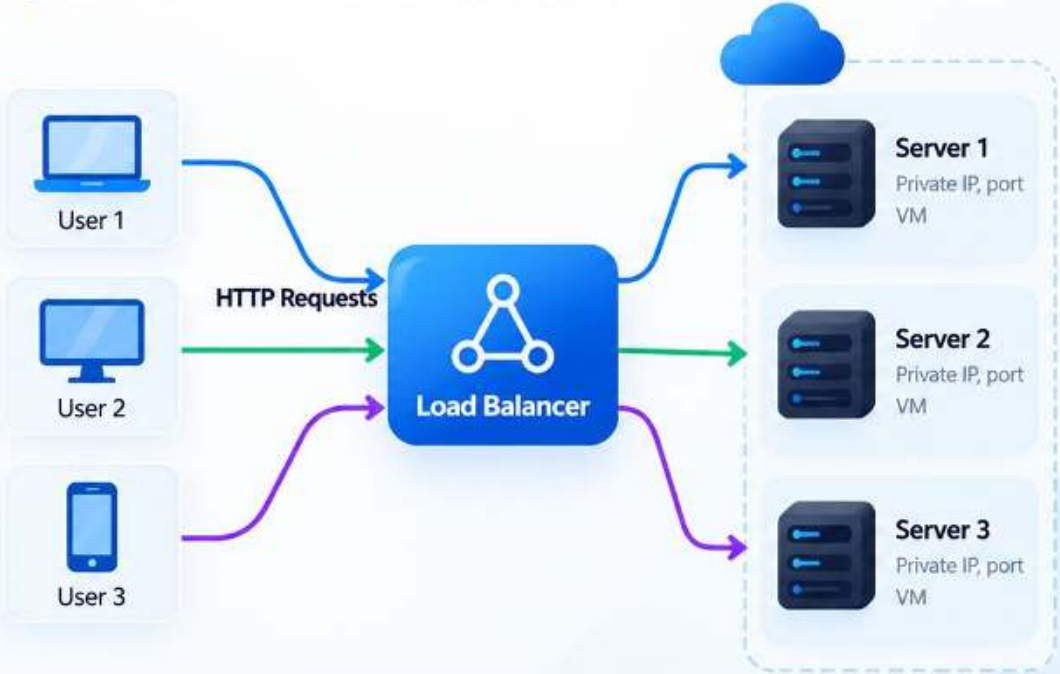


Instructor: Rahul Rajat Singh

Load Balancer in Action

Example Scenario

1. A web application with multiple servers.
2. Users send requests, and the load balancer distributes them efficiently.



Choosing the Right Load Balancer

Layer 4 vs. Layer 7

When to use each type based on your application needs.

Scalability Needs

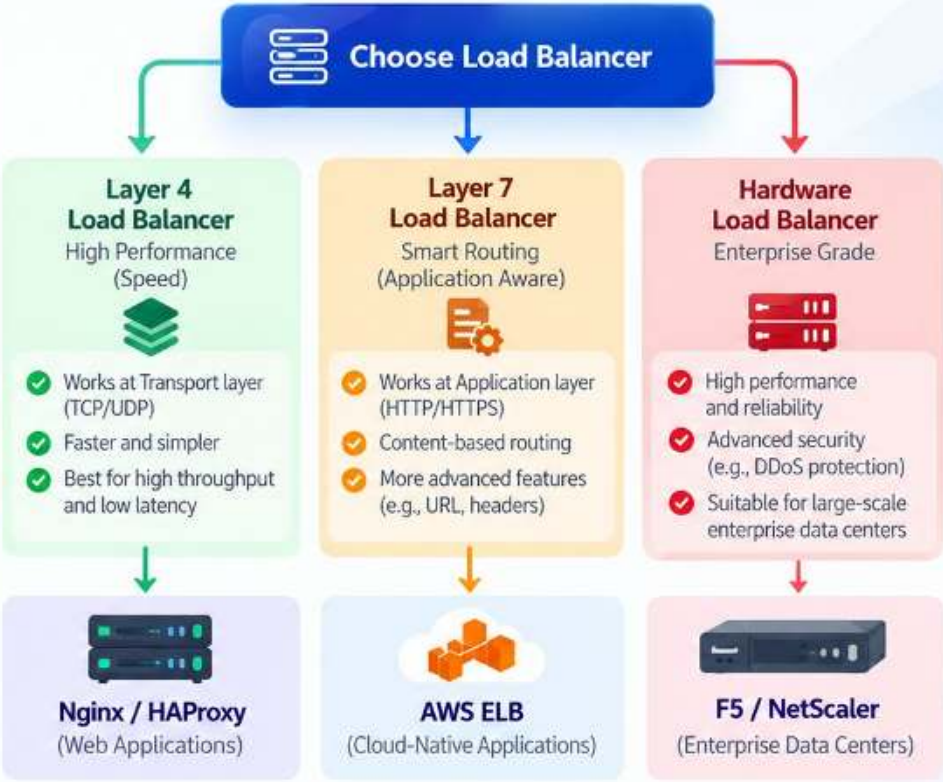
Match the right load balancer to your traffic volume and scaling requirements.

Security Concerns

Look for SSL termination and DDoS protection capabilities.

Common Use Cases

- Nginx/HAProxy for web applications.
- AWS ELB for cloud-native applications.
- Hardware Load Balancers for enterprise data centers.



Interview Questions on Load Balancing

1 Fundamentals



Q1 What is load balancing, and why is it important?

Q2 Explain the difference between Layer 4 and Layer 7 load balancing.

Q3 How does a load balancer handle high availability and failover?

2 Strategies & Use Cases



Q1 Compare Round Robin and Least Connections strategies.

Q2 What are the advantages of Weighted Load Balancing?

Q3 When would you use a software load balancer over a hardware one?

3 Real-World Scenarios



Q1 How would you design a scalable load balancing solution for a large e-commerce site?

Q2 What factors should be considered when choosing a load balancing strategy?

Q3 How does a load balancer improve security?

Key Takeaway Think beyond the definition: explain trade-offs, failure handling, and real-world design choices.

Instructor: Rahul Rajat Singh

Summary & Key Takeaways

1. Load balancing enhances **scalability, reliability, and performance.**
2. Different types exist based on **layers and deployment models.**
3. Choosing the right strategy depends on **traffic patterns and system needs.**
4. Essential for **highly available and resilient** architectures.



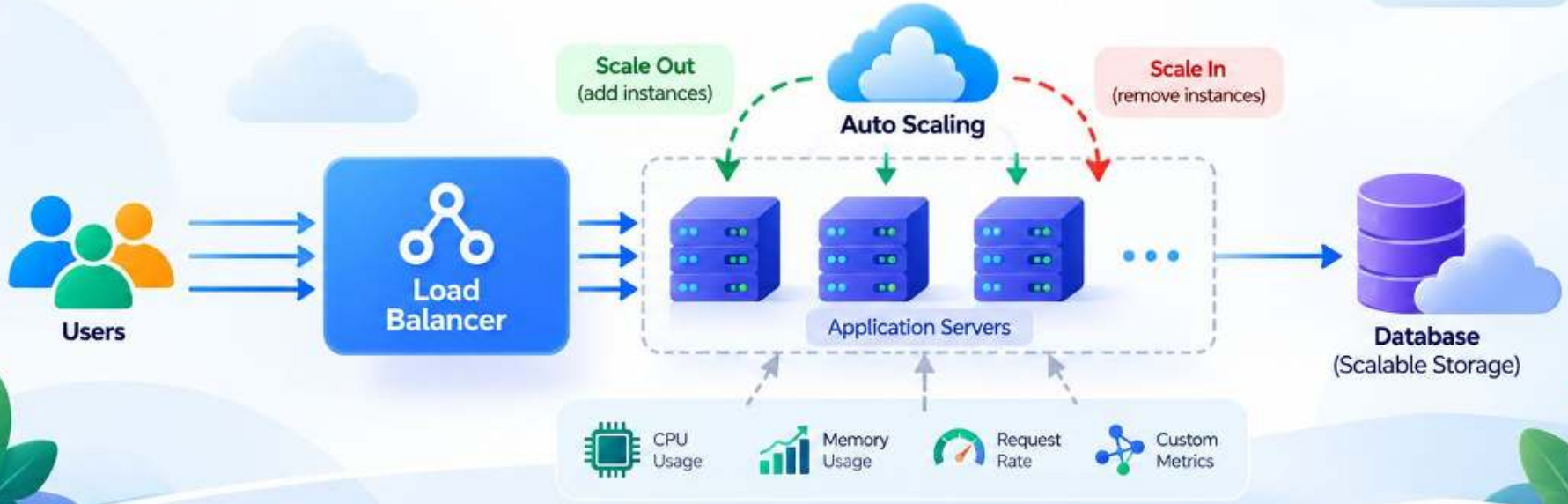
What's Next?

Autoscaling & Best Practices in Cloud Environments

Autoscaling & Best Practices in Cloud Environments

Scalability in System Design

Mastering System Design: From Basics to Cracking Interviews



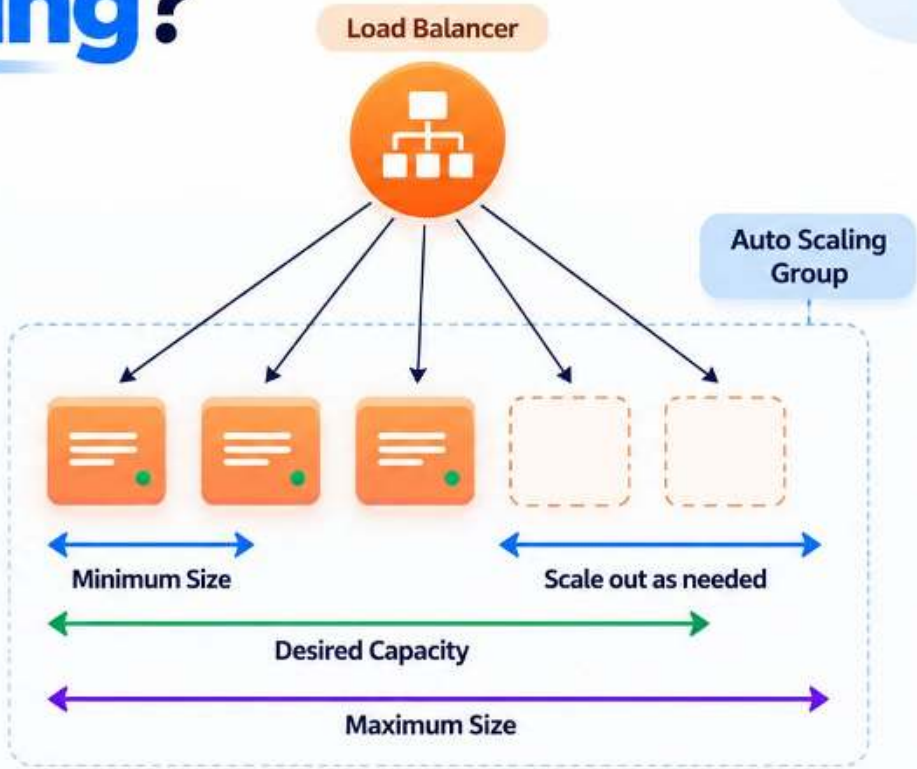
Instructor: Rahul Rajat Singh

What is Autoscaling?

Automatic adjustment of compute resources based on load.

Ensures **performance, availability,** and **cost-efficiency**.

Common in **microservices, web apps,** and event-driven systems.



How Autoscaling Works

1 Triggers

Autoscaling is triggered based on metrics that indicate higher or lower load.

- CPU usage
- Memory usage
- Request rate
- Queue length

2 Types

Autoscaling can scale out (add/remove instances) or scale up (resize a single instance).

Horizontal Scaling

Add or remove instances to handle load.

Vertical Scaling

Resize a single instance (more CPU/RAM).

3 Scaling Policies

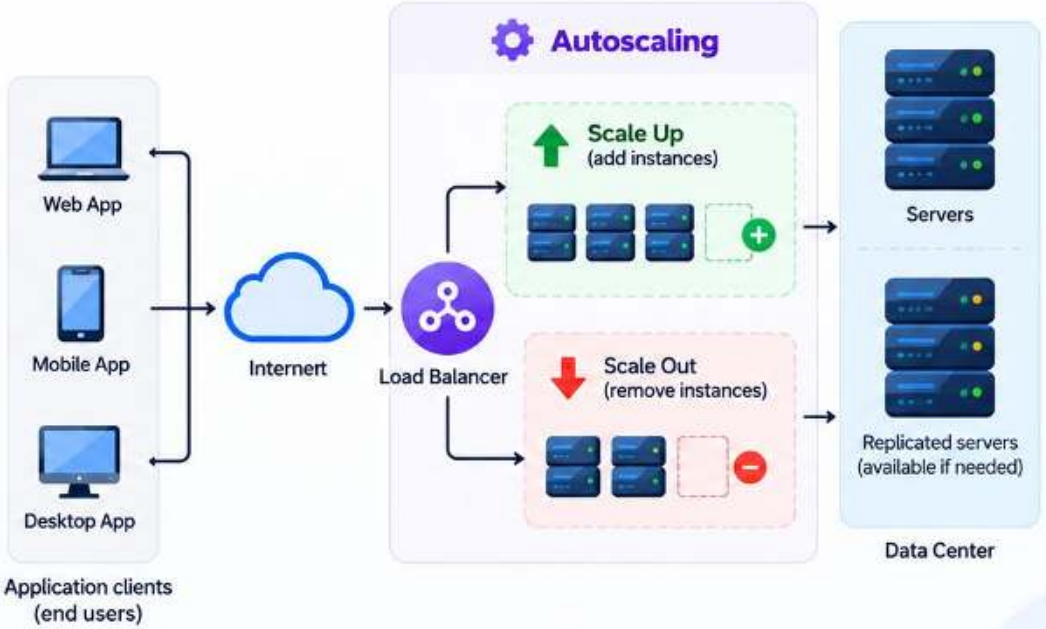
Autoscaling policies define when and how to scale.

Reactive

Scale based on thresholds (e.g., CPU > 70%).

Predictive

Scale based on trends and ML predictions.



Autoscaling Across Cloud Providers

Major cloud providers offer **built-in autoscaling** across compute and container services.

AWS

Auto Scaling for EC2, Lambda, ECS, EKS



- EC2 Auto Scaling
- AWS Lambda (automatic scaling)
- ECS Service Auto Scaling
- EKS Cluster Auto Scaling

Azure

Autoscaling via VM Scale Sets, App Services, AKS



- VM Scale Sets (Automatic Scaling)
- App Services (Auto Scale)
- AKS Cluster Auto Scaling

GCP

Autoscaling with MIGs, Cloud Run, GKE, Functions



- Managed Instance Groups (MIGs)
- Cloud Run (Automatic Scaling)
- GKE Cluster Autoscaler
- Cloud Functions (Automatic Scaling)

Key Takeaway | Scale capacity automatically as demand changes.

Monitoring & Proactive Scaling

1 Use Metrics Like

CPU
Track CPU utilization

Memory
Monitor memory usage

Network
Track network in/out traffic

Queue Depth
Monitor message queue length

Custom KPIs
Business-specific metrics

2 Proactive Scaling

Predictive Algorithms
Scale based on ML models or usage trends

Scheduled Scaling
Scale based on known traffic patterns

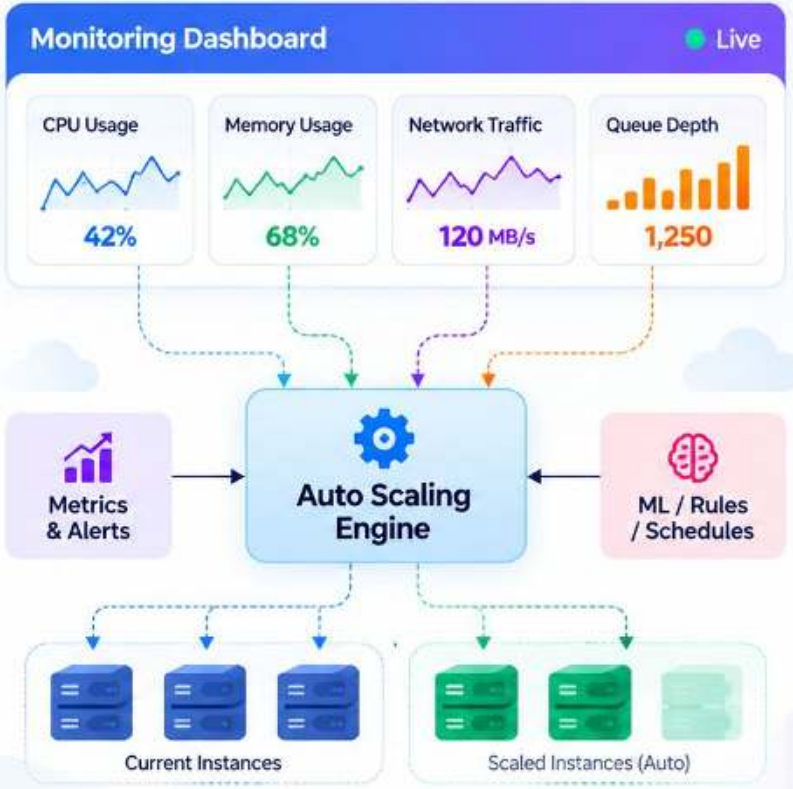
3 Tools for Monitoring & Scaling

CloudWatch
Monitoring & alarms (AWS)

Prometheus + Grafana
Metrics & dashboards

Azure Monitor
Monitoring & insights (Azure)

GCP Operations
Monitoring, logging & alerts (GCP)



Cost Optimization Strategies



1 Avoid Over-Provisioning

Scale just enough based on actual demand.



Right Sized (Optimal Cost)

Over-Provisioned (Higher Cost)

2 Use Spot/ Preemptible Instances

Ideal for batch workloads and non-critical tasks.



3 Apply Resource Limits & Quotas

Prevent unexpected costs with limits and budgets.



- CPU Limits
- Memory Quotas
- Budget Alerts

4 Rightsize Regularly

Review actual usage and remove/resize unused resources.



5 Use Auto-Pausing or Scale-to-Zero

Automatically stop resources when idle.



Key Takeaway | Optimize for the workload you have today, and continuously rightsize as usage patterns evolve.

Instructor: Rahul Rajat Singh

Interview Questions



1 Conceptual Understanding

- 1.1 What is autoscaling, and why is it important in distributed systems?
- 1.2 What's the difference between horizontal and vertical scaling?
- 1.3 How does predictive autoscaling work?

2 Cloud-Specific Scenarios

- 2.1 How does autoscaling work in AWS/Azure/GCP?
- 2.2 How would you set up autoscaling for a containerized application?
- 2.3 What metrics would you monitor for effective autoscaling?

3 Best Practices

- 3.1 How can you ensure cost optimization when implementing autoscaling?
- 3.2 What are some challenges with autoscaling in real-time systems?

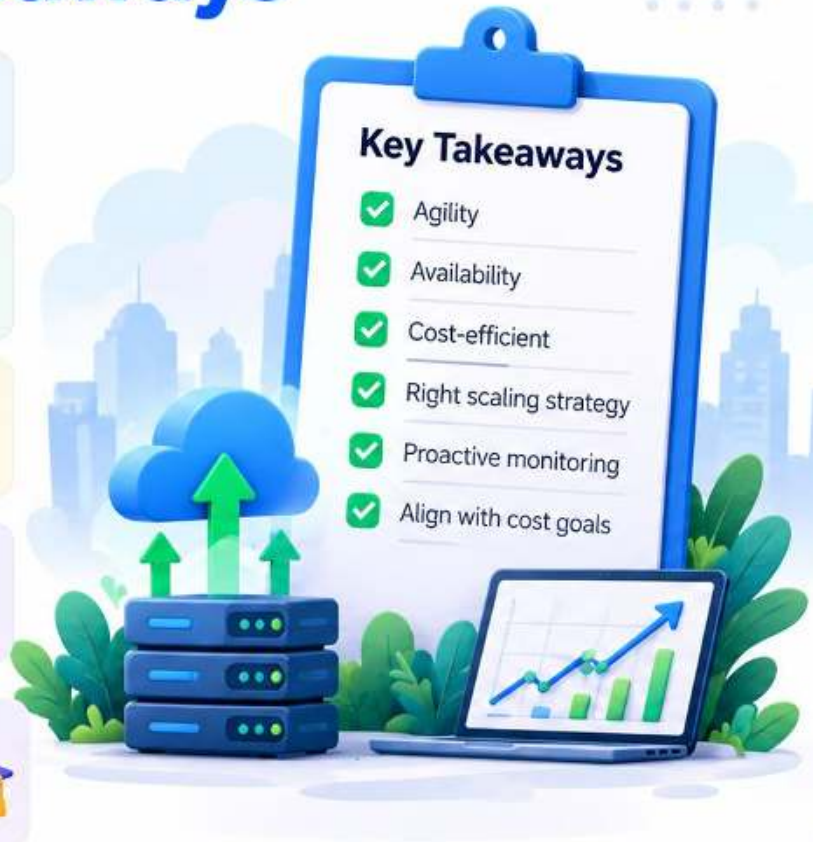
Summary and Key Takeaways

1. **Autoscaling ensures**
   agility, availability, and cost-efficiency.

2. **Choose the right scaling strategy**
   Horizontal, vertical, or predictive based on your needs.

3. **Monitor proactively**
   Stay ahead of load and scale before issues occur.

4. **Align with cost goals**
   Optimize autoscaling based on traffic patterns and budget.



What's Next?

Summary and recap - Scalability in system design.

Section Summary - Scalability in System Design

In this section, we explored the **core building blocks of scalability** in modern systems:

01 **Introduction to Scalability**
Why systems need to scale and the challenges that come with it.

02 **Scaling Strategies**
Deep dive into Horizontal, Vertical, and Diagonal scaling approaches.

03 **Load Balancing**
How different types of load balancers and algorithms distribute traffic efficiently.

04 **Autoscaling**
Best practices and how cloud platforms like AWS, Azure, and GCP support autoscaling.



What's Next?

Database and Storage

Interview Questions - Scaling strategies

📘 Interview Question 1: What is the difference between horizontal and vertical scaling?

Answer:

- **Horizontal Scaling** (Scaling Out): Adding more machines or instances to distribute the load.
  Example: Adding more application servers behind a load balancer.
  ✅ Scales well for web apps and microservices.
  ⚠️ Adds complexity in state management, coordination, and deployment.

- **Vertical Scaling** (Scaling Up): Increasing the resources (CPU, RAM, disk) of a single server.
  Example: Upgrading a database server from 16GB RAM to 64GB RAM.
  ✅ Simpler and faster to implement.
  ⚠️ Has physical/OS limits and can be a single point of failure.

📘 Interview Question 2: What is diagonal scaling and when is it a good idea?

Answer:

Diagonal Scaling is a hybrid approach:

- Start with **vertical scaling** (easier, cheaper for small scale).

- Then move to **horizontal scaling** as load increases beyond a single machine's limits.

✅ **Use cases:**

- Startups: Begin with minimal infra and gradually scale horizontally as demand grows.

- Systems with burst traffic: Use vertical scaling for fast reactions, and horizontal for long-term elasticity.

This approach balances **simplicity + future-proofing**, avoiding premature complexity.

📘 Interview Question 3: What are the trade-offs between horizontal and vertical scaling in terms of performance and complexity?

Answer:

Factor	Horizontal Scaling	Vertical Scaling
Performance	Can scale almost linearly (with effort)	Limited by single machine capacity
Complexity	Requires distributed system design	Easier to manage initially
Fault Tolerance	More resilient (failover possible)	Single point of failure
Cost	Higher operational cost & complexity	High-capacity machines are expensive
Deployment	Requires orchestration & load balancing	Simple deploys, fewer moving parts

Summary:

- Horizontal = More scalable, resilient, but complex.
- Vertical = Quick wins, limited long-term.

📘 Interview Question 4: Can you describe a scenario where horizontal scaling wouldn’t help?

Answer:

Horizontal scaling won’t help when:

- The workload **isn’t parallelizable**. Example: A legacy monolithic app that relies on shared state or global locks.
- There’s a **non-distributable bottleneck**, like a single-threaded operation or a relational DB that doesn't scale well horizontally.

- There's a **stateful service** without session persistence or sticky sessions in the load balancer.

Real-world example:

- Scaling a PostgreSQL database for analytics queries — simply adding more nodes won't help unless sharding or read replicas are implemented.

📘 Interview Question 5: When would you choose vertical scaling over horizontal?

Answer:

Choose **vertical scaling** when:

- You need a **quick fix** without refactoring code.
- The system is **monolithic** or tightly coupled.
- Your **team lacks experience** with distributed systems.
- You're in the **early stages** of a project/startup.
- The system has **low concurrency needs** and doesn't warrant added complexity.

✅ Vertical scaling is ideal when you **don't need high elasticity** or can't justify the cost/complexity of a distributed architecture.

📘 Interview Question 6: What challenges arise in horizontal scaling and how would you solve them?

Answer:

Common Challenges:

1. **State Management**
    - Solution: Use external stores like Redis or Memcached for sessions.

2. **Data Consistency**
    - Solution: Implement distributed transactions, eventual consistency patterns, or use CQRS.
3. **Load Distribution**
    - Solution: Use smart load balancers (e.g., with least-connections or IP hashing).
4. **Service Discovery**
    - Solution: Use tools like Consul, Eureka, or cloud-native DNS-based discovery.
5. **Deployment & Orchestration**
    - Solution: Use containers (Docker) and orchestrators like Kubernetes for managing instances.
6. **Increased Latency**
    - Solution: Minimize inter-service calls, implement caching and optimize communication paths.

Interview Questions & Answers — Introduction to Load Balancing

1. What problem does load balancing solve?

Answer:
Load balancing solves the problem of handling increasing traffic in a scalable and reliable manner. As the number of users grows, a single server can become overwhelmed by incoming requests. A load balancer distributes traffic across multiple servers, ensuring that no single server carries all the workload. This improves system performance, enables scalability, and helps maintain availability even when individual servers experience issues.

2. Why does a single server eventually become a bottleneck?

Answer:
Every server has finite resources, including CPU, memory, storage, and network bandwidth. As user traffic increases, these resources become increasingly utilized until the server reaches its capacity limits. Once this happens, response times increase, requests begin to queue, and users may experience timeouts or failures. Regardless of how powerful a server is, there is always a maximum capacity it can handle, making it a bottleneck as the application grows.

3. What is the difference between vertical scaling and horizontal scaling?

Answer:
Vertical scaling involves increasing the resources of an existing server, such as adding more CPU cores, memory, or storage. Horizontal scaling involves adding more servers and distributing the workload across them.

Vertical scaling is generally simpler because the application remains on a single machine. However, it has physical and financial limits. Horizontal scaling requires additional infrastructure and traffic distribution mechanisms but provides significantly greater scalability and fault tolerance, making it the preferred approach for large-scale systems.

4. Why is horizontal scaling often preferred over vertical scaling?

Answer:
Horizontal scaling is often preferred because it provides a more sustainable path for long-term growth. While vertical scaling can temporarily increase capacity, there is always a limit to how much a single server can be upgraded. Additionally, larger servers become increasingly expensive and create a greater dependency on a single machine.

By adding more servers, organizations can continue increasing capacity as demand grows. Horizontal scaling also improves fault tolerance because the system can continue operating even if one server fails.

5. What challenge arises when an application is deployed across multiple servers?

Answer:
Once multiple application servers are introduced, incoming requests must be distributed among them efficiently. Users should not need to know which server to connect to, and traffic should be directed in a way that makes effective use of available resources.

Without a mechanism to manage this distribution, some servers may become overloaded while others remain underutilized. This challenge creates the need for a load balancer to act as a central traffic distribution layer.

6. What role does a load balancer play in a distributed system?

Answer:
A load balancer acts as a single entry point for incoming requests. Instead of clients connecting directly to individual application servers, they send requests to the load balancer, which then forwards those requests to one of the available backend servers.

This abstraction simplifies client interactions, enables horizontal scaling, and allows the system to distribute traffic efficiently. It also provides a foundation for improving availability and reliability within distributed architectures.

7. How does load balancing improve scalability?

Answer:
Load balancing improves scalability by allowing traffic to be distributed across multiple servers rather than relying on a single machine. As traffic increases, additional servers can be added to the system, and the load balancer can begin routing requests to them.

This approach enables capacity to grow incrementally and helps organizations handle increasing workloads without requiring major architectural changes or constant hardware upgrades.

8. How does load balancing improve availability?

Answer:
Load balancing improves availability by reducing dependency on any single server. When multiple servers are available to handle requests, the system can continue operating even if one server becomes unavailable.

This redundancy helps minimize downtime and ensures that users can continue accessing the application despite infrastructure failures, maintenance activities, or unexpected outages.

9. What happens if one server fails in a load-balanced architecture?

Answer:
In a load-balanced architecture, a failed server can be removed from active traffic routing, allowing requests to be directed to the remaining healthy servers. As a result, the application continues serving users with minimal disruption.

This capability significantly improves reliability because a single server failure no longer causes the entire application to become unavailable. The impact of failures is isolated, making the system more resilient.

10. Why is load balancing considered a foundational component of modern system design?

Answer:
Load balancing is considered foundational because it enables many of the characteristics expected from modern applications, including scalability, high availability, reliability, and fault tolerance.

As systems grow beyond a single server, some mechanism is needed to distribute traffic and manage multiple instances. Load balancing provides this capability and serves as a critical building block for distributed systems, cloud-native architectures, and large-scale applications.

11. When should you introduce a load balancer into a system?

Answer:
A load balancer should be introduced when a single server can no longer meet performance, scalability, or availability requirements. This typically occurs when traffic increases significantly, when multiple application servers are required, or when the business demands higher uptime and reliability.

Introducing a load balancer becomes especially valuable when organizations begin adopting horizontal scaling strategies and need a reliable way to distribute traffic across multiple servers.

12. Walk me through the evolution from a single-server application to a load-balanced architecture.

Answer:
Most applications begin with a single server because it is simple and cost-effective. As traffic grows, that server eventually reaches its capacity limits, leading to performance issues and reduced reliability.

The first response is often vertical scaling by upgrading the server's resources. However, this approach eventually reaches practical limits. To continue growing, additional application servers are introduced through horizontal scaling.

Once multiple servers exist, traffic must be distributed among them efficiently. A load balancer is added in front of the servers to act as a single entry point and route requests appropriately. This evolution enables the system to handle larger workloads, improve availability, and become more resilient to failures.

Interview Questions & Answers on Load Balancing

1. What is load balancing, and why is it important?

Answer: Load balancing is the process of distributing incoming network traffic across multiple backend servers to ensure efficient utilization, prevent overload, and improve system availability.

- **Ensures High Availability**: Prevents system downtime by redirecting traffic in case of server failure.
- **Optimizes Resource Utilization**: Spreads requests evenly to avoid overloading a single server.
- **Improves Performance**: Reduces latency by routing traffic to the best-performing server.
- **Enhances Scalability**: Supports horizontal scaling by adding more servers as demand grows.
- **Increases Fault Tolerance**: Redirects requests if a server fails, ensuring system reliability.

---

2. Explain the difference between Layer 4 and Layer 7 load balancing.

Answer:

Layer 4 Load Balancing (Transport Layer)

- Operates at the **network transport level** (TCP/UDP).
- Distributes traffic based on **IP addresses and port numbers** without inspecting request content.
- Faster and more efficient for **simple traffic distribution**.
- Examples: **AWS Network Load Balancer (NLB), HAProxy (L4 Mode)**.

Layer 7 Load Balancing (Application Layer)

- Works at the **application level** (HTTP/HTTPS).
- Routes requests based on **content, headers, cookies, or URL paths**.
- Supports **advanced features** like SSL termination, caching, and authentication.
- Examples: **AWS Application Load Balancer (ALB), Nginx, Traefik**.

Key Difference:
Layer 4 is **faster** but **less flexible**, while Layer 7 is **intelligent** but **adds overhead**.

---

3. How does a load balancer handle high availability and failover?

Answer:

- **Health Checks**: Continuously monitors server health using **ping, HTTP checks, or TCP checks**.
- **Automatic Failover**: If a server becomes unresponsive, the load balancer redirects traffic to healthy servers.
- **Redundancy**: Can be deployed in **active-active** or **active-passive** configurations.
- **Session Persistence**: Maintains user sessions across multiple requests to prevent disruptions.
- **Global Load Balancing**: Uses **GeoDNS** or **Anycast Routing** to distribute traffic across data centers.

---

4. Compare Round Robin and Least Connections strategies.

Answer:

Round Robin

- Sends requests to servers in a **circular order** (Server 1 → Server 2 → Server 3 → Repeat).
- **Best for:** Uniform workloads and **servers with equal capacity**.
- **Limitations:** Can overload servers if they have different processing power.

Least Connections

- Sends requests to the server with the **fewest active connections**.
- **Best for:** Scenarios where some requests take longer than others (e.g., database queries).
- **Limitations:** Requires tracking active connections, increasing computational overhead.

Key Difference:
Round Robin is **simpler** but assumes equal server capacity, while Least Connections **dynamically** adjusts based on load.

---

5. What are the advantages of Weighted Load Balancing?

Answer:
Weighted Load Balancing assigns **different priorities** to servers based on their capacity.

- **Better Resource Utilization**: High-performance servers receive more traffic.
- **Custom Traffic Distribution**: Allows fine-tuned control over request routing.
- **Supports Heterogeneous Environments**: Works well when servers have **different processing power**.
- **Examples**: Weighted Round Robin, Weighted Least Connections.

---

6. When would you use a software load balancer over a hardware one?

Answer:

Software Load Balancer

- Runs as an application on standard hardware.
- **Pros**:
    - Cost-effective and flexible.
    - Easily scalable (deployed in containers or VMs).

    - Supports **open-source solutions** like Nginx, HAProxy, Envoy.

- **Cons**:
    - Requires server resources.
    - May introduce additional latency under heavy traffic.

Hardware Load Balancer

- A dedicated device optimized for handling large-scale traffic.
- **Pros**:
    - High performance with dedicated hardware acceleration.
    - Built-in security features (e.g., **DDoS protection**).
- **Cons**:
    - Expensive and less flexible.
    - Harder to scale dynamically.

Use Case:

- Use **software load balancers** for **cloud-native applications**.
- Use **hardware load balancers** for **enterprise-level, high-traffic systems**.

---

7. How would you design a scalable load balancing solution for a large e-commerce site?

Answer:

1. **Use Multiple Load Balancers**:
    - Deploy **primary and secondary load balancers** for redundancy.
    - Distribute traffic globally using **DNS-based load balancing**.
2. **Choose the Right Load Balancer**:

- Use **Layer 7** load balancing for dynamic content.
- Use **Layer 4** load balancing for database connections.

3. **Implement Load Balancing Strategies**:
   - **Round Robin** for static content servers.
   - **Least Connections** for dynamic request handling.
4. **Ensure High Availability**:
   - Use **auto-scaling groups** to handle traffic spikes.
   - Implement **health checks** to detect and bypass failed servers.
5. **Optimize Performance**:
   - Enable **caching** (e.g., CDN) to reduce load on backend servers.
   - Use **Gzip compression and minification** to reduce response sizes.

---

8. What factors should be considered when choosing a load balancing strategy?

Answer:

1. **Traffic Pattern**:
   - If traffic is **evenly distributed**, use **Round Robin**.
   - If requests vary in complexity, use **Least Connections**.
2. **Server Capacity**:
   - If servers have different capacities, use **Weighted Load Balancing**.
3. **Session Persistence**:
   - If user sessions must be maintained, use **Sticky Sessions**.
4. **Performance vs. Complexity**:
   - Layer 4 is **faster but less flexible**.

- Layer 7 is **slower but allows intelligent routing**.

5. **Scalability Needs**:
    - For **cloud-native applications**, use **cloud-based load balancers** (e.g., AWS ELB).
    - For **on-premises applications**, use **software or hardware-based solutions**.

---

9. How does a load balancer improve security?

Answer:

1. **DDoS Protection**
    - Detects and blocks **malicious traffic spikes**.
    - Some hardware load balancers provide **built-in DDoS mitigation**.
2. **SSL Termination**
    - Offloads **SSL decryption** from backend servers.
    - Ensures **secure connections** with HTTPS.
3. **Access Control**
    - Restricts access using **firewalls and IP whitelisting**.
4. **Application Firewall Integration**
    - Prevents **SQL injection, cross-site scripting (XSS), and other attacks**.
5. **Rate Limiting**
    - Limits requests per second to prevent abuse.

Interview questions - Autoscaling & Cloud Best Practices

Conceptual Understanding

Q1. What is autoscaling, and why is it important in distributed systems?
A: Autoscaling is the automatic adjustment of compute resources based on current demand. It's critical in distributed systems because it ensures:

- High availability by scaling out during peak traffic.
- Cost-efficiency by scaling in during idle times.
- Better user experience by minimizing latency and avoiding overload.

Q2. What's the difference between horizontal and vertical scaling?
A:

- **Horizontal Scaling (Scale Out/In):** Adding or removing instances (e.g., more servers, pods).
  ✅ More resilient, better for stateless services.
- **Vertical Scaling (Scale Up/Down):** Increasing or decreasing the resources (CPU/RAM) of a single machine.
  ⚠️ Limited by hardware and often requires downtime.

Q3. How does predictive autoscaling work?
A: Predictive autoscaling uses historical usage patterns and machine learning to forecast future demand. Based on this prediction:

- Resources are provisioned in advance.
- Example: AWS Predictive Scaling uses past traffic patterns to scale EC2 instances proactively before load increases.

⚙️ Cloud-Specific Scenarios

Q4. How does autoscaling work in AWS/Azure/GCP?
A:

- **AWS:** Uses Auto Scaling Groups (ASGs) tied to CloudWatch metrics. Works with EC2, ECS, EKS, Lambda.
- **Azure:** Uses Virtual Machine Scale Sets (VMSS), App Service Plans, and AKS with rules from Azure Monitor.
- **GCP:** Uses Managed Instance Groups (MIGs), Cloud Functions, and Cloud Run with GCP Monitoring. GKE uses Horizontal Pod Autoscaler (HPA).

Q5. How would you set up autoscaling for a containerized application?
A:

- Use an orchestrator like **Kubernetes** or **ECS/EKS/AKS**.
- Set up **Horizontal Pod Autoscaler** based on CPU usage or custom metrics.
- Use a **load balancer** to distribute traffic across scaled pods.
- Configure monitoring and alerting to validate scaling behavior.

Q6. What metrics would you monitor for effective autoscaling?
A:

- CPU and memory usage
- Request rate per instance
- Queue depth (e.g., SQS, Kafka)
- Latency and error rates
- Custom business metrics (e.g., active users, transactions/sec)

💸 Best Practices

Q7. How can you ensure cost optimization when implementing autoscaling?
A:

- Set scaling thresholds wisely to avoid unnecessary scaling.
- Use **spot/preemptible instances** for non-critical workloads.
- Employ **scale-to-zero** for idle services (Cloud Run, Lambda).
- Regularly **rightsize** instances based on usage.
- Use **budgets and alerts** to monitor cloud spending.

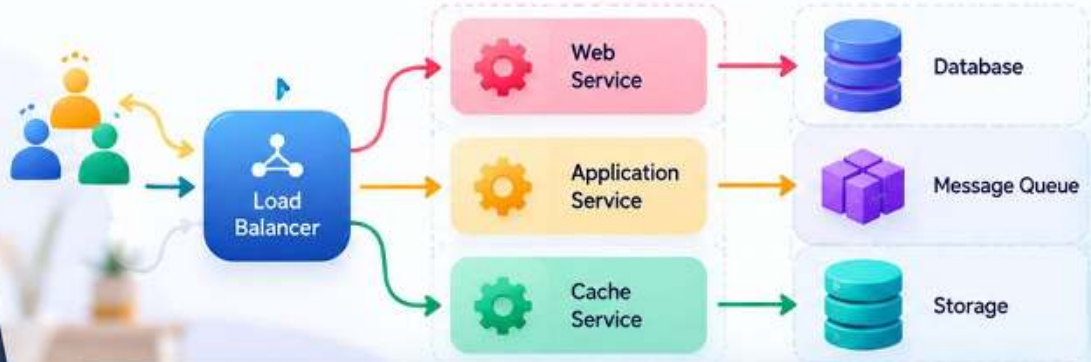
Q8. What are some challenges with autoscaling in real-time systems?
A:

- **Scaling latency**: Time taken to spin up new instances may cause temporary lag.
- **Cold starts**: Especially in serverless platforms like Lambda or Cloud Functions.
- **Over-provisioning**: Can lead to higher costs.
- **Under-provisioning**: If thresholds are too high or scaling is delayed, it can cause outages.

Introduction to System Performance

Performance - Concepts, Tools & Techniques

Mastering System Design: From Basics to Cracking Interviews









Performance | Concepts | Tools & Techniques | System Design for Interviews

Instructor: Rahul Rajat Singh

What is Performance in System Design?

Performance = How efficiently a system meets its functional requirements under load.

Key Dimensions:

Speed
Time to respond

Capacity
Amount of work handled

Efficiency
Resource usage under load

Performance is not a single metric — it's a **multi-dimensional goal.**



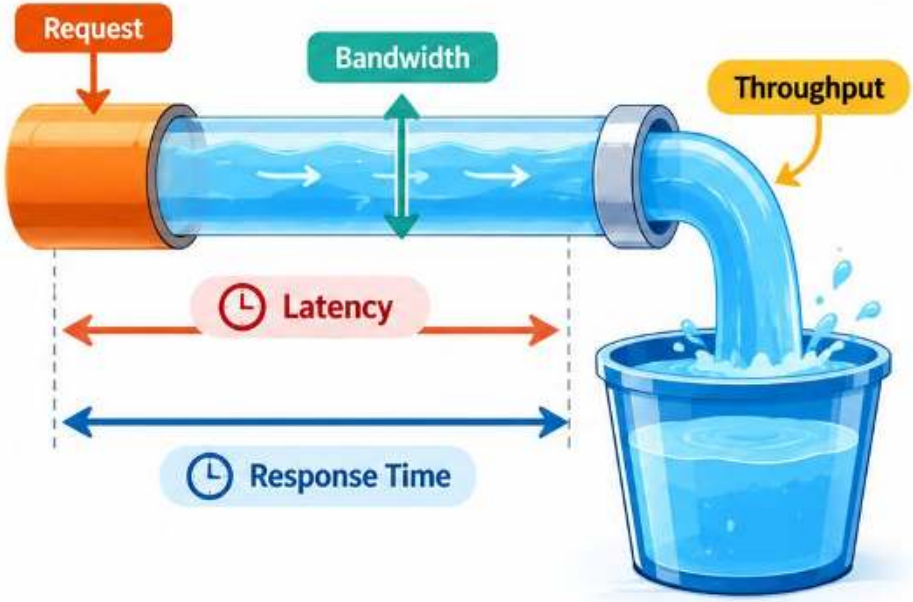
Latency vs. Throughput

Latency

- Time taken to process one request
- Measured in ms or s
- Affects responsiveness

Throughput

- Number of requests processed per second
- Measured in RPS or TPS
- Affects scalability



Low latency ≠ High throughput
and vice versa.

Both must be balanced
based on use case.

Instructor: Rahul Rajat Singh

Scalability vs. Responsiveness

Scalability

Ability to handle increased load without performance degradation.

Horizontal vs. Vertical scaling

Responsiveness

System's ability to respond quickly.

Tightly linked to latency

Key Takeaway

Good design should ensure responsiveness at scale.



Measuring Performance

Performance must be measurable & trackable.

SLA | **Service Level Agreement**
External contractual guarantee.

SLO | **Service Level Objective**
Internal target.

SLI | **Service Level Indicator**
Actual metric value.



Example

1. **SLA:** 99.9% uptime
2. **SLO:** 95% of requests < 300ms
3. **SLI:** Actual measurement (e.g., 93% of requests < 300ms)

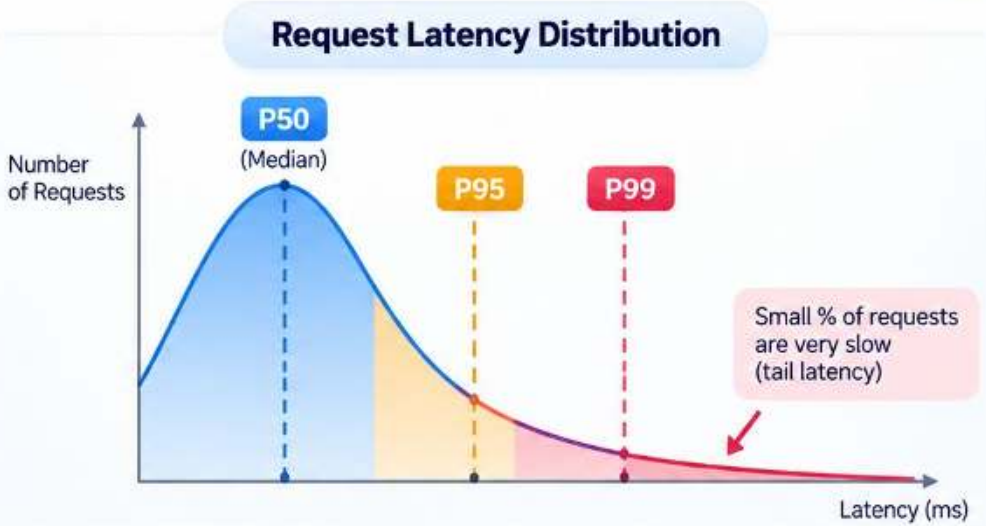
Understanding Percentiles

Mean/average ≠ useful in tail-latency-sensitive systems.

Percentiles provide better insight:

- **P50**: Median
- **P95**: 95% of requests are faster than this
- **P99**: Tail latency — critical for user experience

Track tail latencies for real-world performance insights.



1. **50% of requests** ≤ P50
2. **45% of requests** P50 → P95
3. **4% of requests** P95 → P99
4. **1% slowest** Beyond P99

Why Performance Matters in Modern Applications

Users expect near-instant responses
across mobile and web.

Poor performance leads to:

- Drop-offs and bounce rates
- Lost revenue
- System instability under load

Performance is a feature,
not an afterthought.

It shapes user experience,
operating cost, and reputation.



PERFORMANCE

- User Experience
- Business Growth
- System Stability
- Cost Optimization
- Reputation

Performance Testing Overview



Load Testing



Tests system performance under **normal load conditions.**

Stress Testing



Tests system behavior **beyond normal limits.**

Spike Testing



Tests system response to a **sudden large load.**

Endurance Testing



Tests system performance over an **extended time period.**

Performance Testing

- **Load Test** — Normal Traffic
- **Stress Test** — Beyond Capacity
- **Spike Test** — Sudden Traffic Burst
- **Endurance Test** — Long Duration Load



Goals:

1. Identify bottlenecks
2. Ensure reliability under real-world scenarios



Introduction to Performance Monitoring

1 Monitoring ≠ Testing

Monitoring is continuous—
not a one-time test.

2 Key Tools

APM
New Relic,
Datadog

Logs & Metrics
ELK, Prometheus
+ Grafana

3 What to Track

- Latency & throughput
- Error rates
- Resource usage (CPU, memory, DB queries)



Instructor: Rahul Rajat Singh

Interview Questions — System Performance

1. 

What is the difference between latency and throughput?

2. 

How do SLAs, SLOs, and SLIs differ? Provide real-world examples.

3. 

Why are percentiles (like P95, P99) important in performance monitoring?

4. 

What strategies would you use to identify a system's performance bottleneck?

5. 

How would you ensure responsiveness in a highly scalable system?

6. 

What tools or techniques have you used for performance testing and monitoring?

7. 

How would you design a system to handle sudden traffic spikes?

8. 

Explain the trade-offs between performance and cost in cloud environments.

Summary and Key Takeaways

01



Performance = speed, efficiency & scalability under load.

02



Understand latency vs. throughput.

03



Set and measure SLAs, SLOs, and percentiles.

04



Performance must be tested, monitored, and prioritized.

05



Essential for modern, scalable, and user-friendly systems.

WHAT'S NEXT →



Caching for Speed Optimization

Caching for Speed Optimization

Performance – Concepts, Tools & Techniques

Mastering System Design: From Basics to Cracking Interviews



Instructor: Rahul Rajat Singh

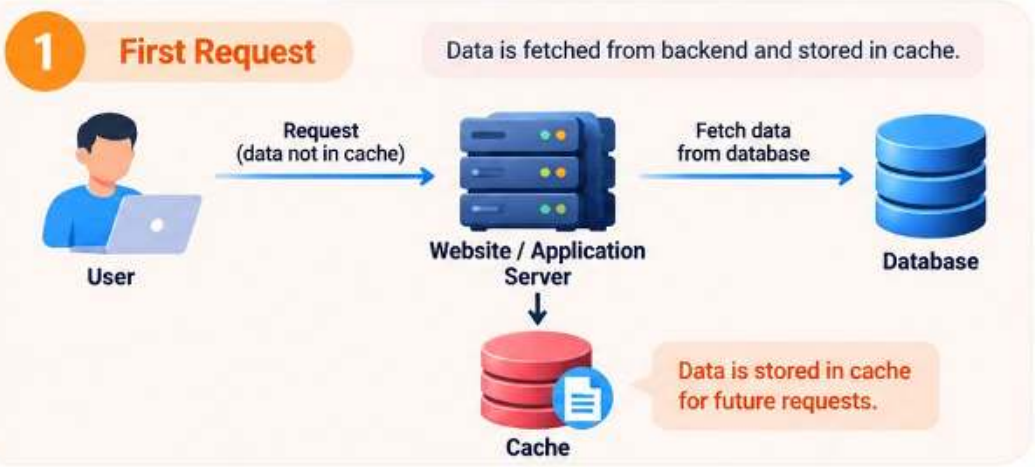
Why Caching Matters

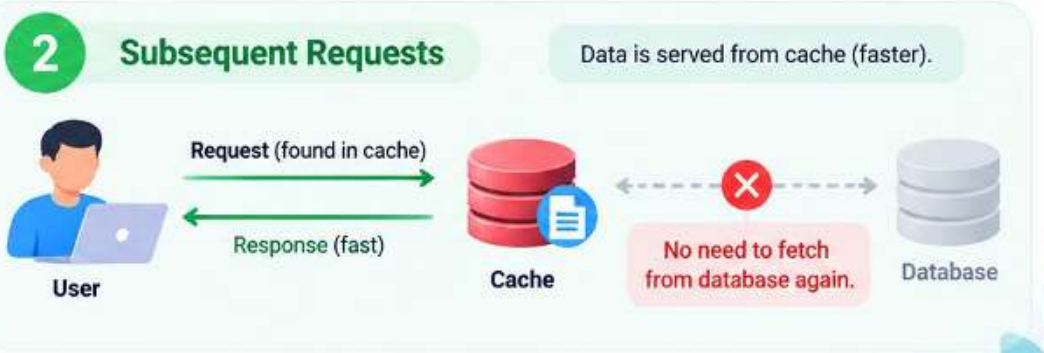
Reduces latency
by avoiding expensive recomputation or data retrieval.

Eases load
on backend services and databases.

Improves user experience
and system scalability.

Critical
in low-latency, high-throughput architectures.





Types of Caching

1. 

Client-side caching

Browser memory: localStorage, service workers

2. 

Server-side caching

Application-level memory and in-memory caches such as Redis

3. 

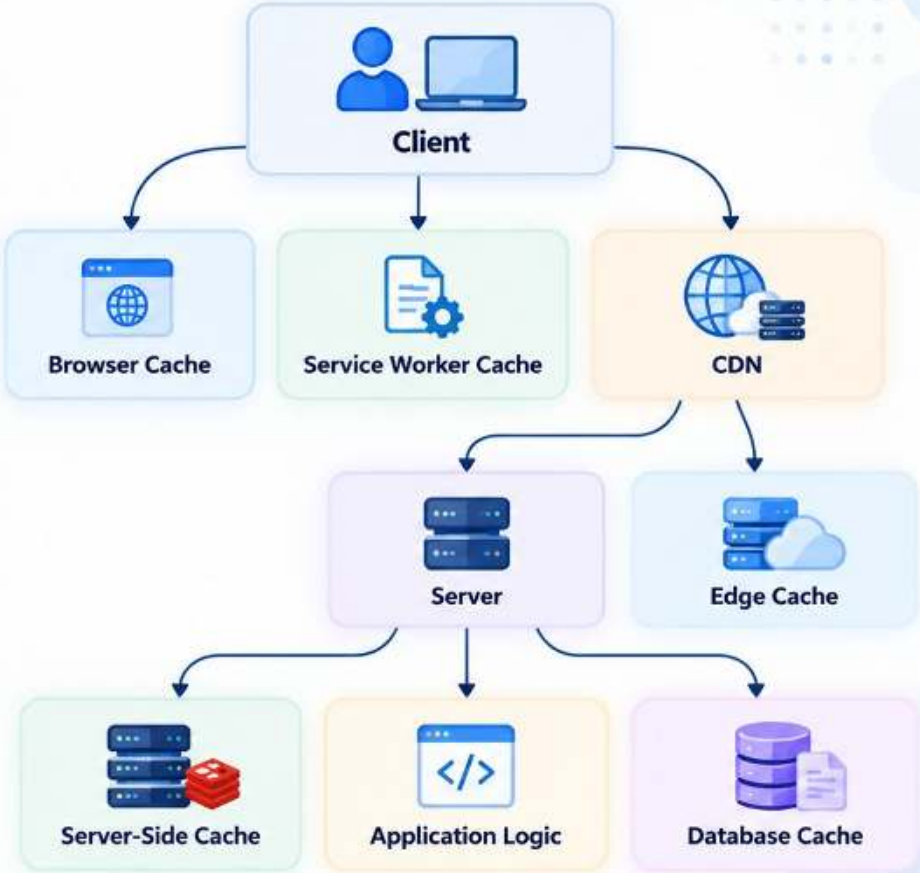
CDN caching

Static content stored closer to users; examples: Cloudflare, Akamai

4. 

Database caching

Cached query results and materialized views



Caching Strategies

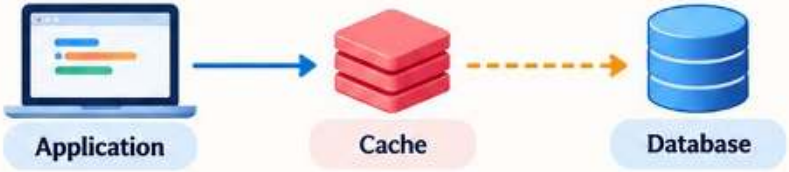
1 Write-through

Write to cache and database at the same time.



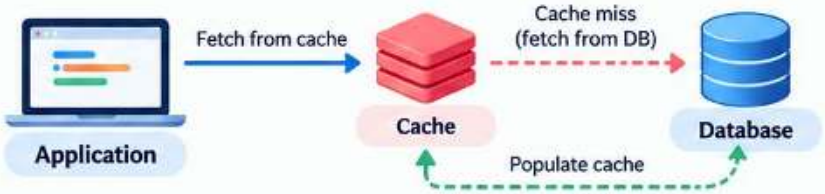
2 Write-back (write-behind)

Write to cache first; update the database asynchronously.



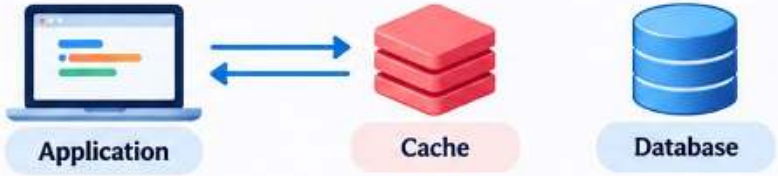
3 Cache-aside (lazy loading)

Fetch from the database on a cache miss, then populate the cache.



4 Explicit / manual caching

The application decides when to cache or evict it.



Cache Eviction Policies

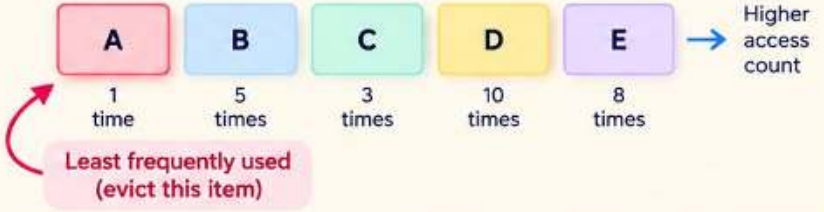
1 LRU (Least Recently Used)

Evict the item that has not been accessed for the longest time.



2 LFU (Least Frequently Used)

Evict the item with the fewest accesses.



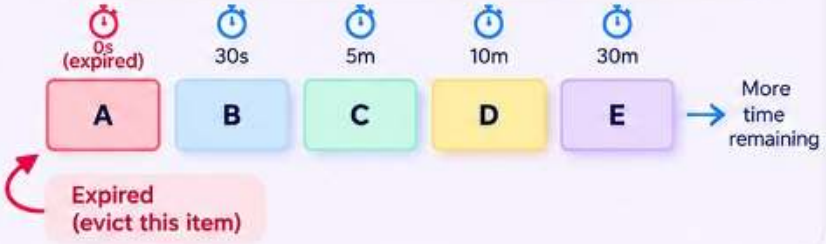
3 FIFO (First In, First Out)

Evict the item that entered the cache first.



4 TTL (Time To Live)

Expire items automatically after a configured duration.

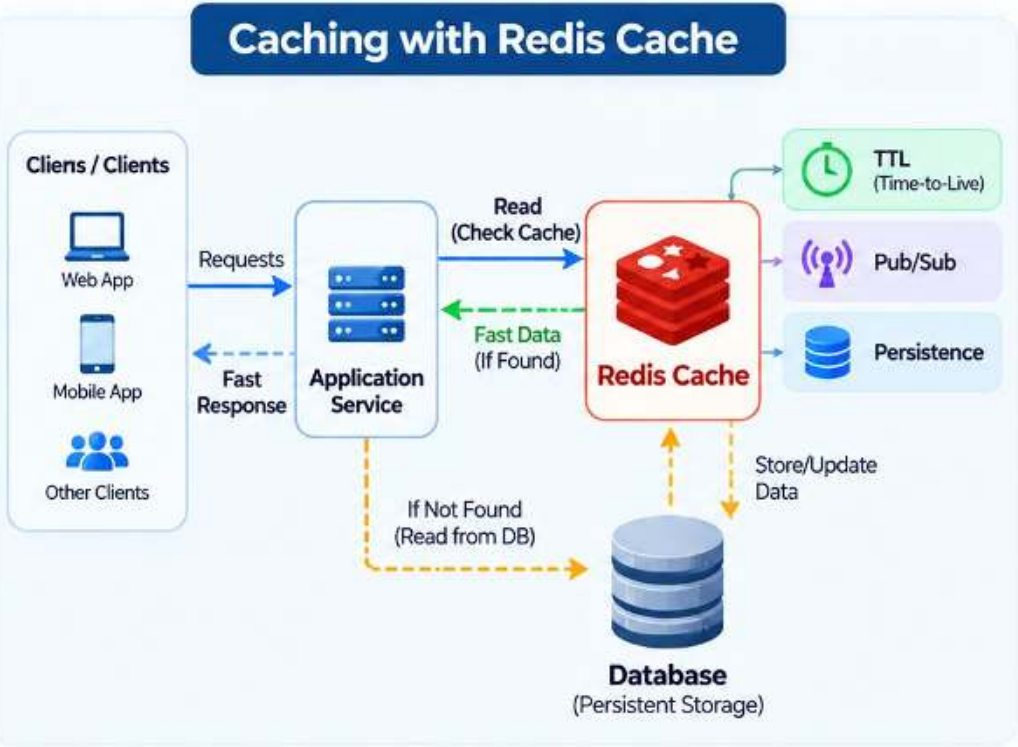


Redis Overview

1. **In-memory key-value store** for ultra-fast access.
2. Supports **TTLs (time-to-live)**, pub/sub, and persistence.
3. Used for **caching, queues, sessions, leaderboards, etc.**
4. **Open-source** and widely adopted.

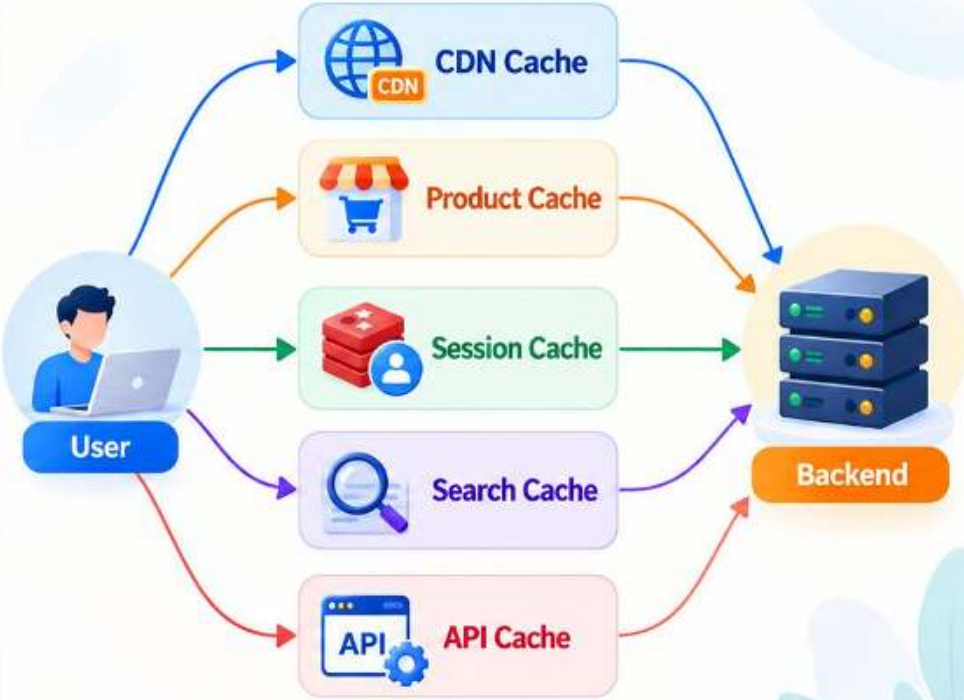
Common Use Cases

- Caching
- Sessions
- Queues
- Leaderboards



Real-World Caching Examples

1. **CDN**
   Cache static assets (images, JS, CSS).
2. **Product page data**
   Cache popular catalog queries.
3. **User sessions**
   Stored in Redis for fast access.
4. **Search results**
   Frequently repeated queries cached.
5. **API response caching**
   Microservices avoid recomputation.



Instructor: Rahul Rajat Singh

Interview Questions – Caching in System Design

Core Conceptual Questions

1. What is caching and why is it important in system design?
2. Explain different types of caching and where they are used.
3. What are write-through vs. write-back caching strategies?
4. What is lazy loading (cache-aside pattern) and when would you use it?

Scenario-Based Questions

1. How would you use caching to optimize a product details page?
2. What eviction strategy would you choose for a memory-limited system?
3. How would you keep cache and database in sync?
4. What are the potential downsides or risks of aggressive caching?

Practical Implementation

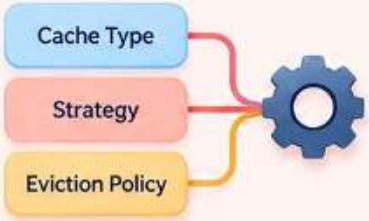
1. How would you implement Redis caching in a web application?
2. How can you prevent cache stampedes or thundering herd problems?
3. What tools can be used for distributed caching?

Instructor: Rahul Rajat Singh

Summary & What's Next



Caching improves **response time**, **reduces load**, and **scales better**.



Choose **cache type**, **strategy**, and **eviction policy** wisely.



Redis is a powerful tool in the caching toolkit.



Caching is foundational in **high-performance systems**.



WHAT'S NEXT

Messaging & Queues for Decoupling



Messaging & Queues for Decoupling

Performance – Concepts, Tools & Techniques

Mastering System Design: From Basics to Cracking Interviews



Instructor: Rahul Rajat Singh

Why Use Asynchronous Messaging?

Loose Coupling
Decouple producers from consumers.

Improved Performance
Producers don't block waiting for consumers.

Scalability
Consumers can scale independently.

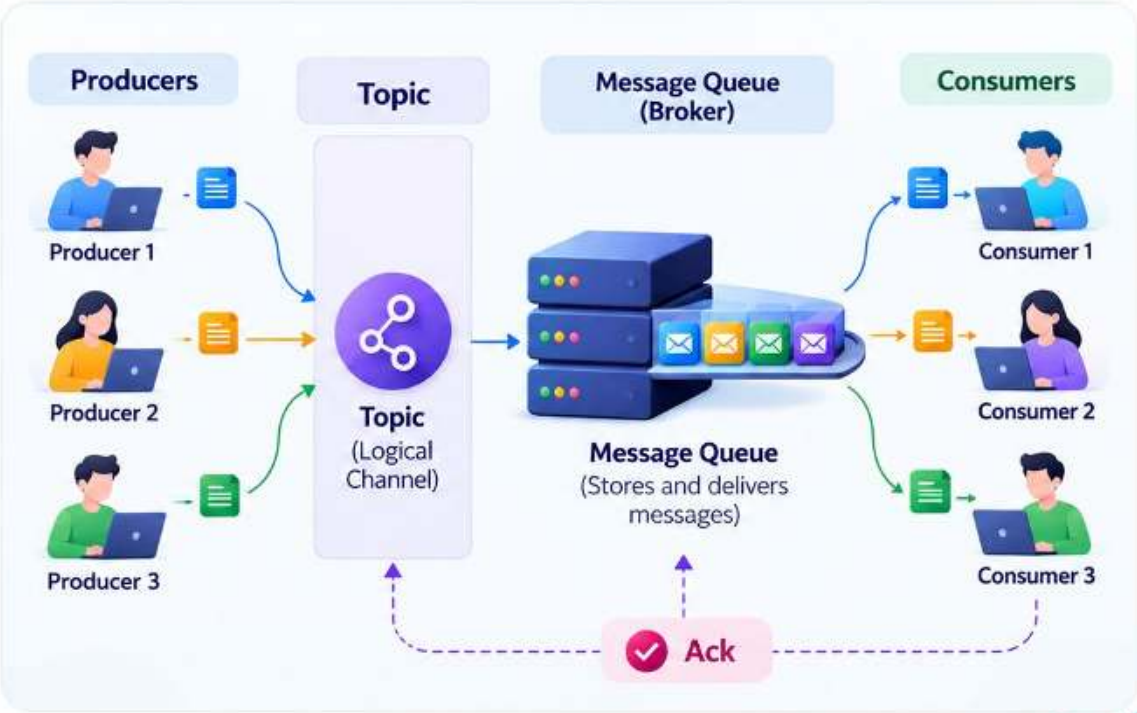
Resilience
Message durability adds fault tolerance.

Flexibility
Easily add new consumers without changing producers.

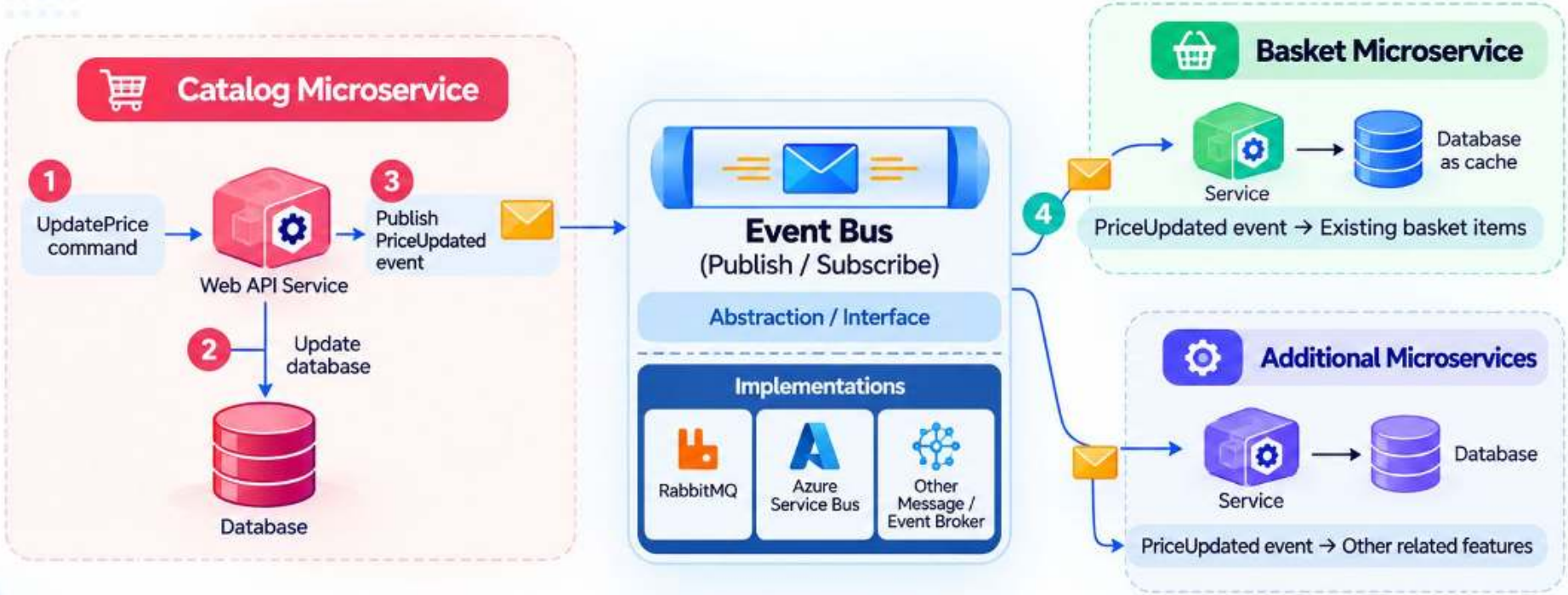


Key Concepts of Messaging Systems

1. **Message**
   A data packet sent from producer to consumer.
2. **Producer**
   Sends the message.
3. **Consumer**
   Receives and processes the message.
4. **Broker/Queue**
   Stores and delivers messages.
5. **Topic/Queue**
   Logical channel for message delivery.
6. **Ack**
   Acknowledgement of successful processing.



Visualizing a Decoupled Architecture



When to Use Queues in Architecture

1. **Burst Workloads**
   Absorb sudden demand spikes
2. **Service Decoupling**
   Let services work independently
3. **Background Jobs**
   Email, processing, exports
4. **Rate-Limited or Expensive Operations**
   Control when work is performed
5. **Traffic Buffering**
   Smooth peaks before workers process tasks



Delivery Guarantees

1 At-least-once
(default in many systems)



- **Message is retried** until acknowledged.
- **May lead to duplicates.**
- **Consumers must be** idempotent.

2 At-most-once



- **Message sent** only once.
- **No retries** → may result in message loss.

3 Exactly-once



- **Guaranteed single delivery** without duplicates.
- **Complex and** more resource-heavy.
- **Kafka supports it under** specific constraints.

Common Use Cases of Messaging Queues

1 Order Processing

Decouple frontend from inventory, payments, and shipping.



2 Logging & Monitoring

Centralize logs for analysis (e.g., ELK, Kafka).



3 Rate Limiting & Traffic Shaping

Queue incoming requests and process them at a safe pace.





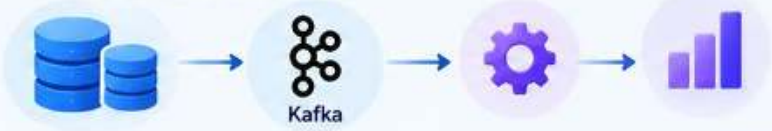
4 Email & SMS Notifications

Queue notifications for asynchronous delivery.



5 ETL Pipelines & Stream Processing

Kafka enables real-time data transformation.



Best Practices for Using Messaging Queues

1 Use idempotent consumers

Ensure that processing the same message multiple times produces the same result.



2 Implement dead-letter queues (DLQs)

Send failed or unprocesable messages to a dead-letter queue (DLQ) for later inspection and handling.



3 Monitor queue length & processing time

Track key metrics like queue length, processing time, and consumer lag to detect issues early.



4 Handle retries & failures gracefully

Implement retry mechanisms with backoff and handle failures without duplicating work.



5 Choose delivery semantics based on need

Select the right delivery semantics (e.g., at-most-once, at-least-once, exactly-once) based on your application requirements.



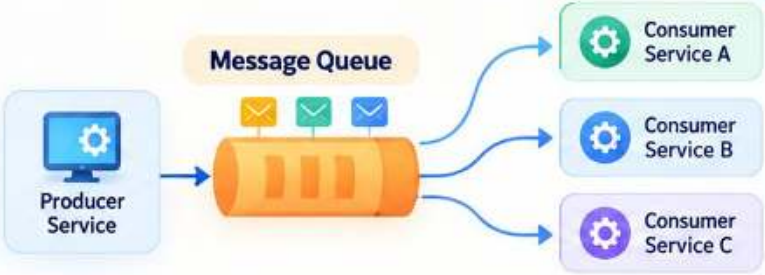
6 Secure your message brokers

Use authentication, encryption, and proper access controls to protect your message brokers.



Instructor: Rahul Rajat Singh

Interview Questions –
Messaging & Queues for Decoupling



FOUNDATIONS

1. Why would you use asynchronous messaging in a system?
2. What are the differences between RabbitMQ and Kafka? When would you use one over the other?

DELIVERY & RELIABILITY

3. Explain the different message delivery guarantees: At-least-once vs. Exactly-once vs. At-most-once – what are they and where do you apply them?
4. How do queues help improve system scalability and fault tolerance?

DESIGN UNDER LOAD

5. What problems might arise in a queue-based system under heavy load? How would you mitigate them?
6. How would you design an order processing system using a message queue?
7. How would you ensure idempotency in the consumers?

Instructor: Rahul Rajat Singh

Summary & Key Takeaways

1



Asynchronous messaging helps build **scalable, decoupled systems.**

2



Use queues when **real-time response isn't required.**

3



RabbitMQ is great for **traditional message queues.**

4



Kafka excels at **high-throughput event streaming.**

5



Understand your **delivery guarantees** to match each use case.

What's Next?



Concurrency & Parallelism

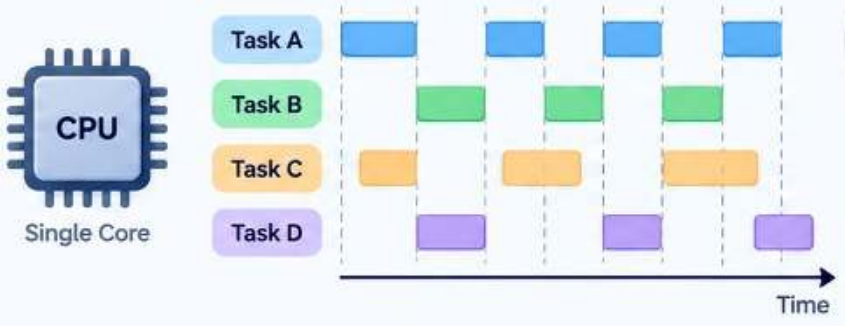
Concurrency & Parallelism

Performance – Concepts, Tools & Techniques

Mastering System Design: From Basics to Cracking Interviews

Concurrency

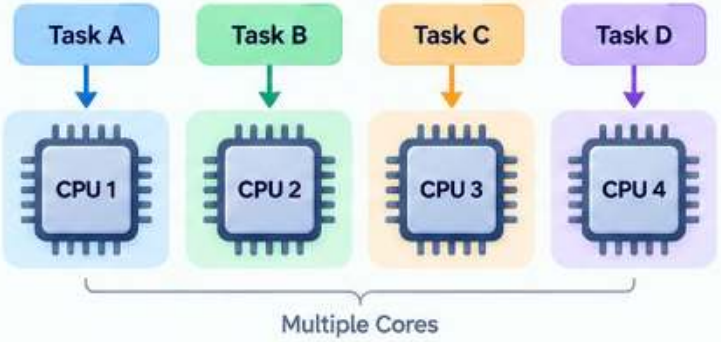
Multiple tasks make progress by interleaving on a **single core** (time sharing).



VS

Parallelism

Multiple tasks run simultaneously on **multiple cores**.



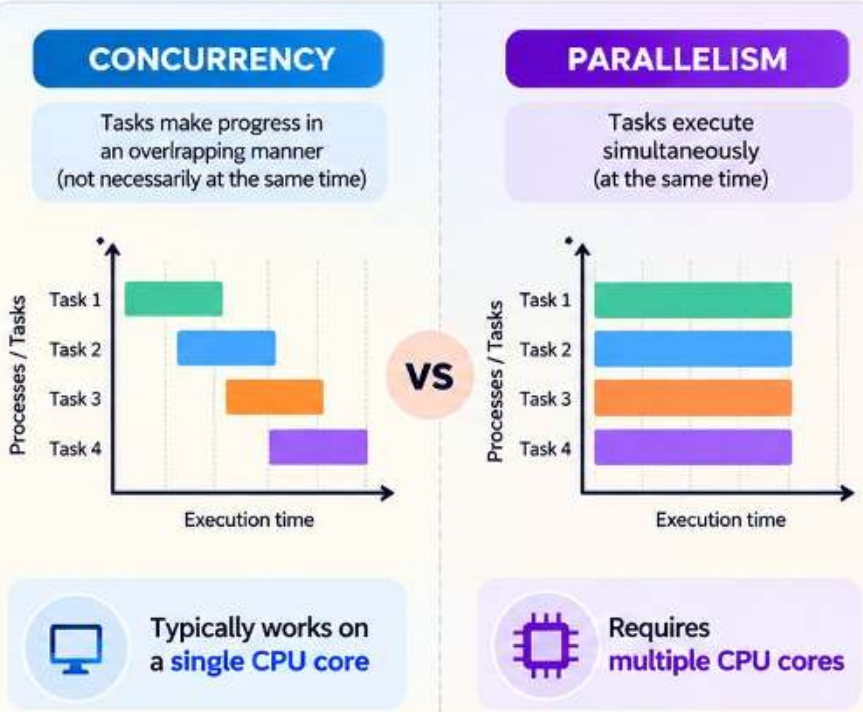
Concurrency and Parallelism

What is Concurrency?

- **Definition:** Concurrency is when multiple tasks start, run, and complete in overlapping time periods – not necessarily simultaneously.
- **Key Points:**
  - It's about managing multiple tasks efficiently, especially on a single CPU core.
  - It's all about task management.
  - Can happen on a single-core.
  - Goal is responsiveness.
- **Example:** Web server handling multiple incoming HTTP requests using asynchronous I/O.

What is Parallelism?

- **Definition:** Parallelism is when multiple tasks are executed simultaneously, typically on multiple CPU cores.
- **Key Points:**
  - It's about actual simultaneous execution to speed up performance.
  - It's all about task execution.
  - Needs multi-core for true parallelism.
  - Goal is speed/throughput.
- **Example:** Matrix computation where different parts are calculated in parallel on multiple threads.



Processes vs. Threads

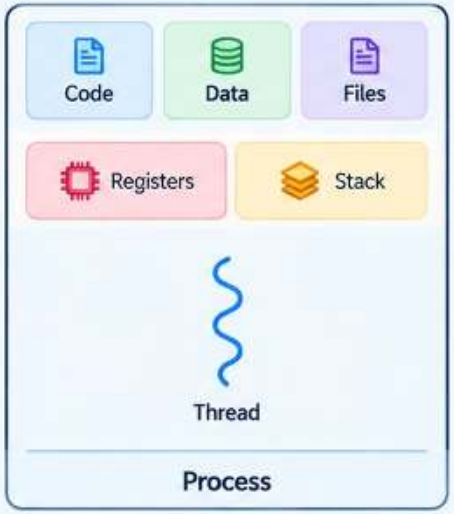
Processes

- Have their own **memory space**
- **Heavier** to create and switch
- **Isolated** and **safer**

Threads

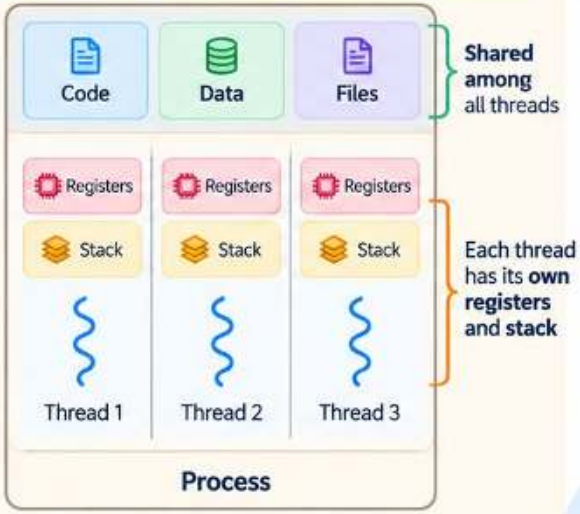
- **Share memory** within a process
- **Lightweight** and **faster**
- Can lead to **complex bugs** (e.g., race conditions)

Single-threaded Process
(One Process, One Thread)



Each process has its **own memory space.**

Multithreaded Process
(Multiple Threads in One Process)



Threads **share memory** within a process (code, data, files).

Thread Pools & Worker Models

1

Thread Pools

- Pre-created threads reused for multiple tasks
- Avoid overhead of creating/ destroying threads

2

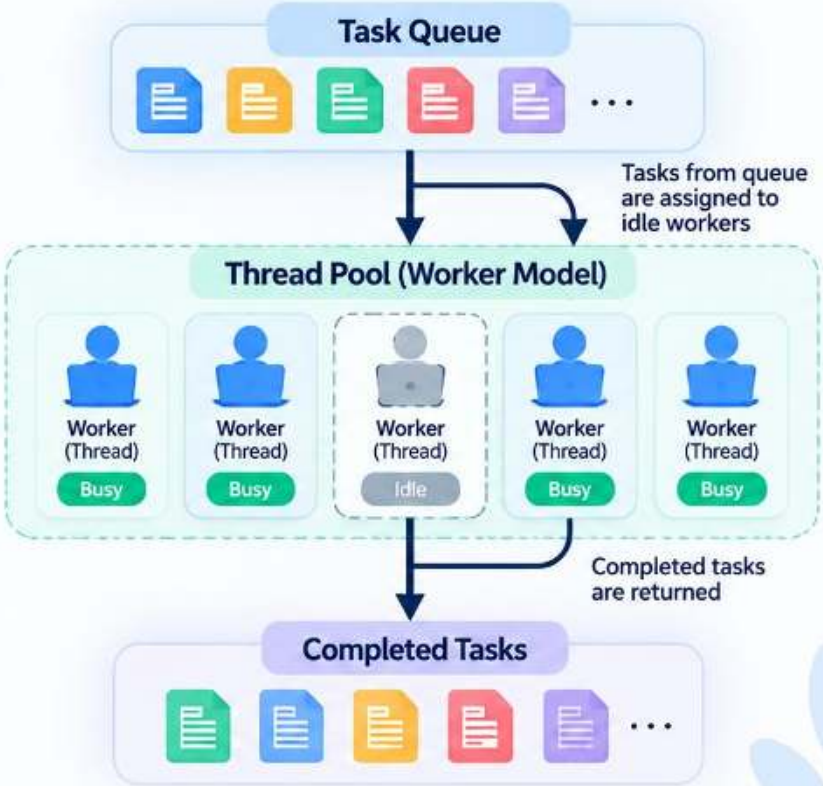
Worker Models

- Tasks are distributed to idle workers from a shared queue
- Improves scalability and CPU utilization

3

Example

- ASP.NET Core uses thread pool to handle requests efficiently.



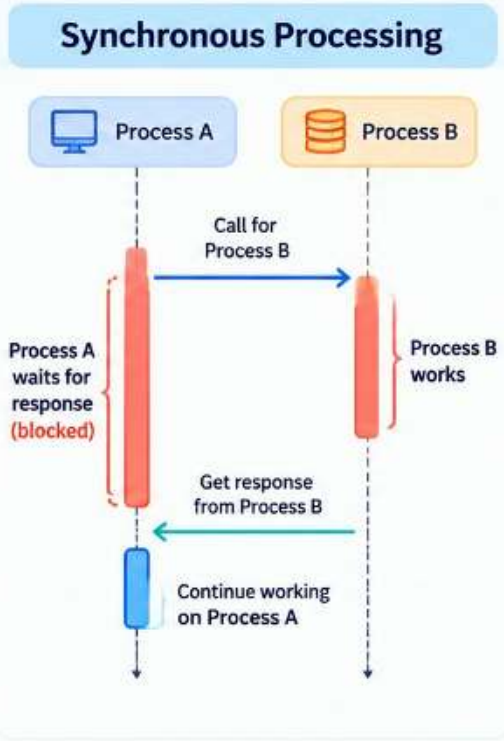
Asynchronous Processing

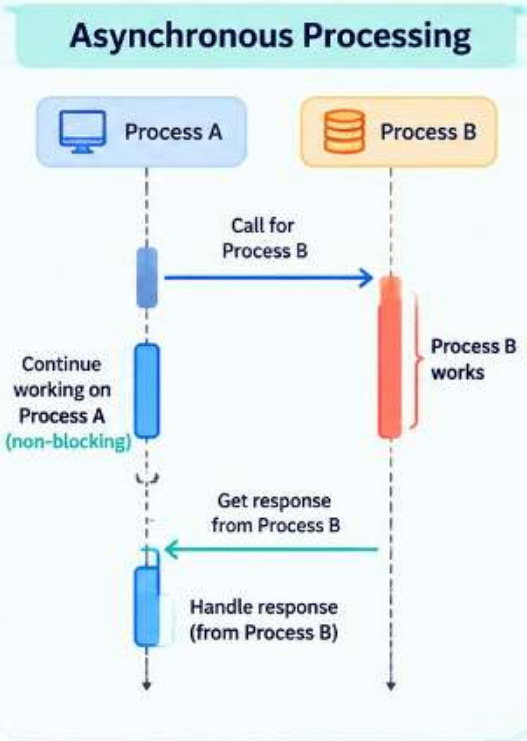
WHY ASYNC?

- Avoid blocking threads on I/O
- Improve throughput

TECHNIQUES

- Async/Await (C#, JavaScript)
- Promises and Futures
- Message queues (RabbitMQ, Kafka) for background work





Concurrency in Web Servers

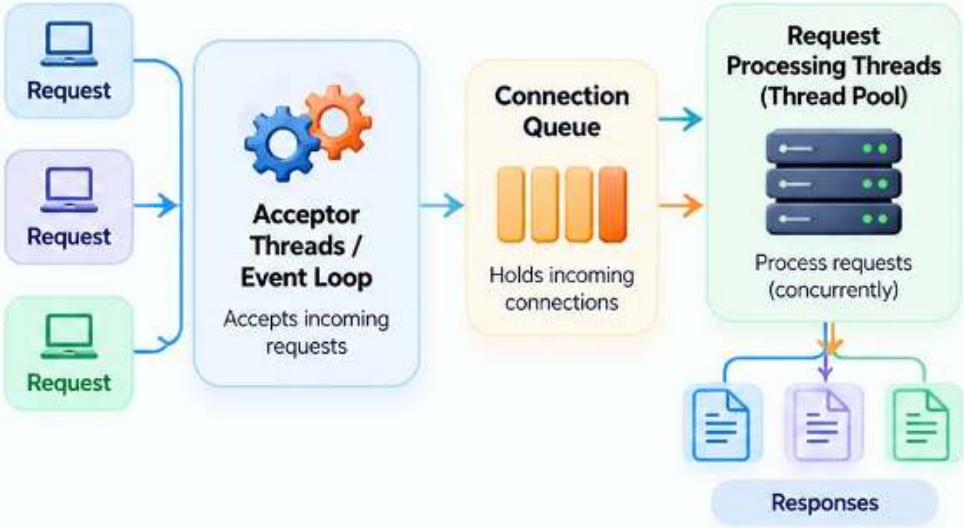
Traditional Servers
(e.g., Apache)

- Spawn a new **thread/process** per request.
- Not scalable for **high traffic.**

Modern Servers
(e.g., Node.js, ASP.NET Core, Nginx)

- Use **async/ non-blocking I/O.**
- Event-loop or thread-pool models for **scalability.**

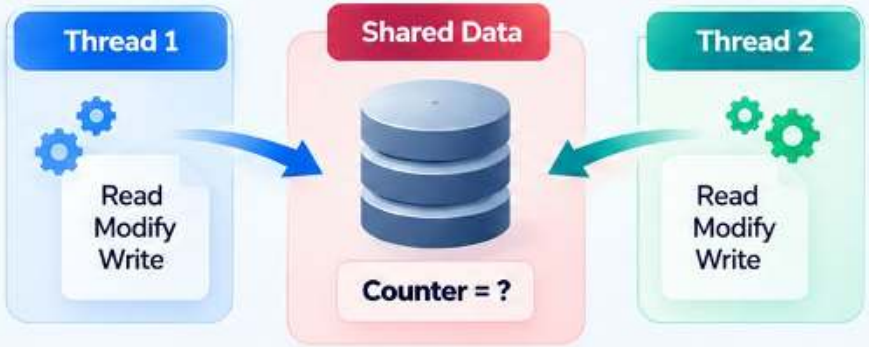
How Concurrency Works in a Web Server



Common Pitfalls

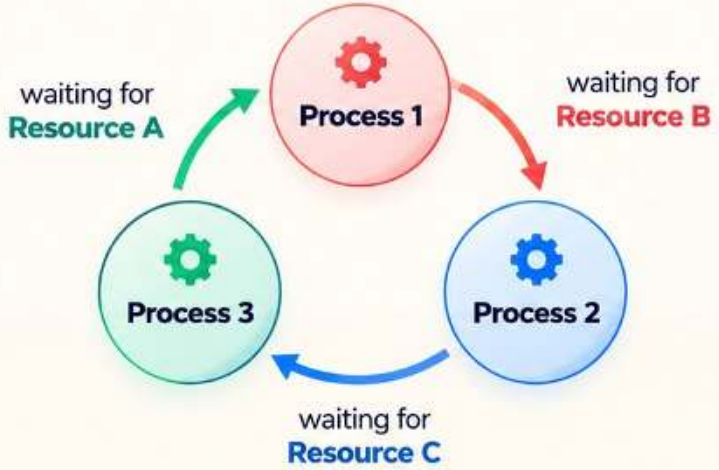
1 Race Conditions

- Multiple threads access and modify shared data concurrently.
- Causes unexpected behavior or data corruption.



2 Deadlocks

- Two or more threads wait for each other to release resources.
- System gets stuck indefinitely.



Best Practices & Real-World Examples

Best Practices

1 Async / non-blocking I/O

Prefer async/non-blocking I/O for I/O-bound tasks.



2 Thread pools

Use thread pools instead of creating raw threads.



3 Synchronize shared data

Always synchronize access to shared data (locks, mutexes, etc.).



Locks
Mutexes
Synchronized Access

4 Prevent deadlocks

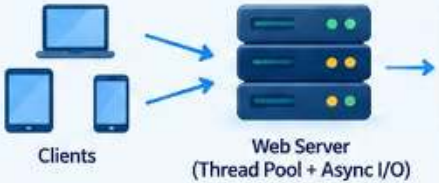
Detect and avoid deadlocks (lock ordering, timeout strategies).



Real-World Examples

1 Web Server Handling 1000s of Requests

Uses thread pool + async I/O to handle thousands of requests.



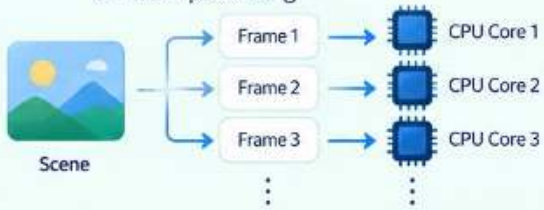
2 Background Job Processing

Worker model with RabbitMQ for email queues.

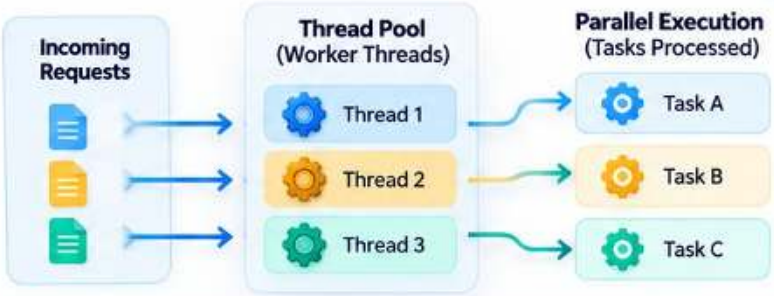


3 Parallel Image Rendering

Each frame rendered on a different core for faster processing.



Interview Questions – Concurrency & Parallelism



CONCEPTUAL QUESTIONS

1. What is the difference between concurrency and parallelism?
2. How do threads differ from processes?
3. What is a thread pool, and why is it preferred over creating new threads?

PRACTICAL SCENARIOS

1. How would you design a web server to handle thousands of concurrent requests?
2. Describe how you would implement background job processing in a scalable system.
3. How would you debug and resolve a deadlock in a multithreaded application?

PITFALL AWARENESS

1. What is a race condition, and how can you prevent it?
2. How do you ensure thread-safe operations in a shared-memory environment?

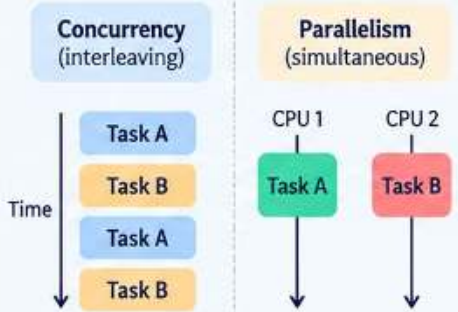
BONUS (ADVANCED)

1. How does the event loop work in Node.js or similar environments?
2. What's the difference between parallelism using threads vs. async I/O?

Summary & Key Takeaways

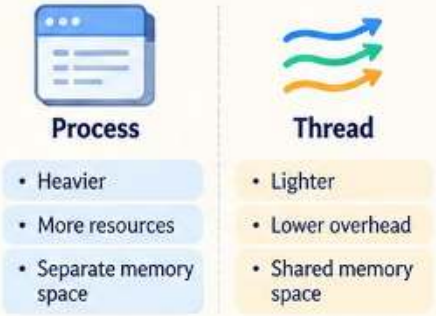
1 Concurrency ≠ Parallelism

Both are tools for performance and scalability.



2 Threads are lighter than processes

Threads are lighter than processes but need careful handling.



3 Web servers rely on thread pools and async models



4 Watch out for race conditions and deadlocks!



WHAT'S NEXT?

Database Performance Optimization Techniques

Instructor: Rahul Rajat Singh

Database Performance Optimization Techniques

Performance – Concepts, Tools & Techniques

Mastering System Design: From Basics to Cracking Interviews



Instructor: Rahul Rajat Singh

Replication Recap

01

What is Replication?

Replication is the process of copying and maintaining database objects across multiple databases.

02

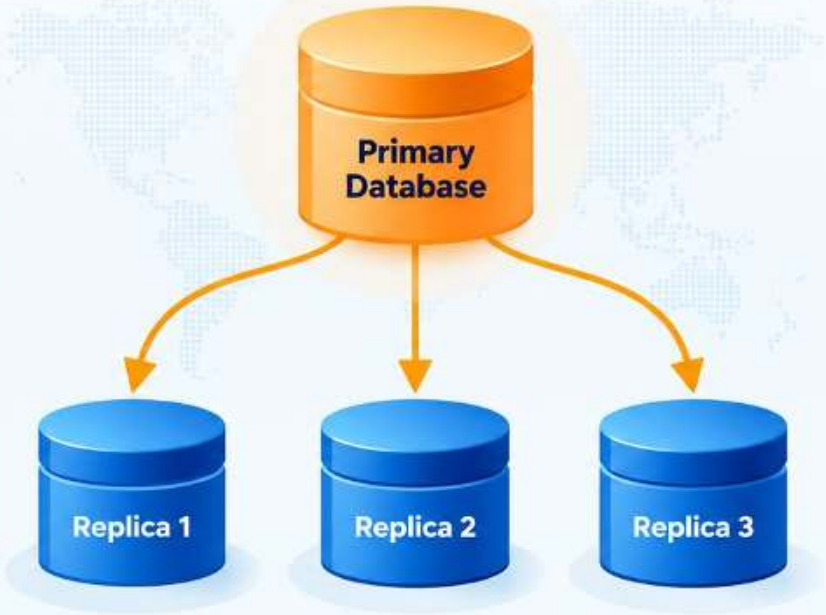
Why Replicate?

- **High Availability:** Ensure data availability in case of failure (data remains accessible).
- **Load Balancing:** Distribute read traffic across multiple servers.
- **Disaster Recovery:** Maintain copies of data across different locations.

03

Types of Replication

- **Primary-Replica Replication:** One primary database, multiple replicas for reads.
- **Multi-Primary Replication:** Multiple primary databases, often used for writes and redundancy.



Sharding & Partitioning Strategies Recap

1 Sharding

Definition: Splitting large datasets into smaller, more manageable pieces (shards).

Goal: Distribute data across multiple servers for better performance and scalability.

Example: User data across multiple shards based on geographical region.



2 Partitioning

Definition: Dividing data within a single database into separate tables or partitions.

Types of Partitioning:

Range Partitioning
Split data by a range (e.g., by date).
Example: Sales data by year/month.

Hash Partitioning
Split data by hashing on a key.
Example: User data by user_id (hash).

Vertical Partitioning (Split Columns)

Split columns/fields into separate tables.

Original Table (All Fields)

CustomerID	Name	Email	Total Purchases
1	ABC	abc@gmail.com	2000
2	PQR	pqr@gmail.com	3000
3	XYZ	xyz@gmail.com	4000
4	LMN	lmn@gmail.com	5000
5	JKL	jkl@gmail.com	6000

Customer Info (Basic Fields)

CustomerID	Name	Email
1	ABC	abc@gmail.com
2	PQR	pqr@gmail.com
3	XYZ	xyz@gmail.com
4	LMN	lmn@gmail.com
5	JKL	jkl@gmail.com

Purchase Info (Separate Table)

CustomerID	Total Purchases
1	2000
2	3000
3	4000
4	5000
5	6000

Horizontal Partition / Database Sharding (Split Rows)

Split rows/records across multiple shards or partitions.

Original Table (All Records)

CustomerID	Name	Email	Total Purchases
1	ABC	abc@gmail.com	2000
2	PQR	pqr@gmail.com	3000
3	XYZ	xyz@gmail.com	4000
4	LMN	lmn@gmail.com	5000
5	JKL	jkl@gmail.com	6000

Shard 1 (e.g., Users from A–M)

CustomerID	Name	Email	Total Purchases
1	ABC	abc@gmail.com	2000
2	PQR	pqr@gmail.com	3000
3	XYZ	xyz@gmail.com	4000

Shard 2 (e.g., Users from N–Z)

CustomerID	Name	Email	Total Purchases
4	LMN	lmn@gmail.com	5000
5	JKL	jkl@gmail.com	6000

CAP Theorem Recap

Performance Focus & Distributed-System Trade-offs

CONSISTENCY

Reads reflect a single, up-to-date view of data.

AVAILABILITY

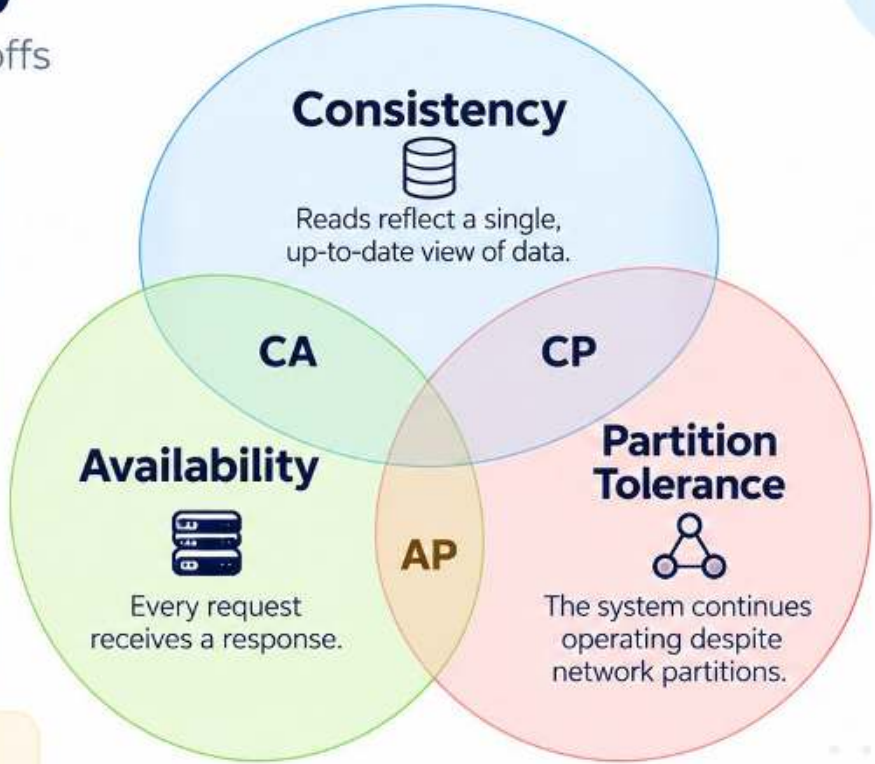
Every request receives a response.

PARTITION TOLERANCE

The system continues operating despite network partitions.

The practical trade-off

When a network partition occurs, a distributed system must prioritize consistency or availability for affected operations.



CAP trade-offs matter during partitions.

Indexes: Types & Use Cases

01

What are Indexes?

Data structures that speed up queries by reducing the amount of data scanned.

02

Types of Indexes

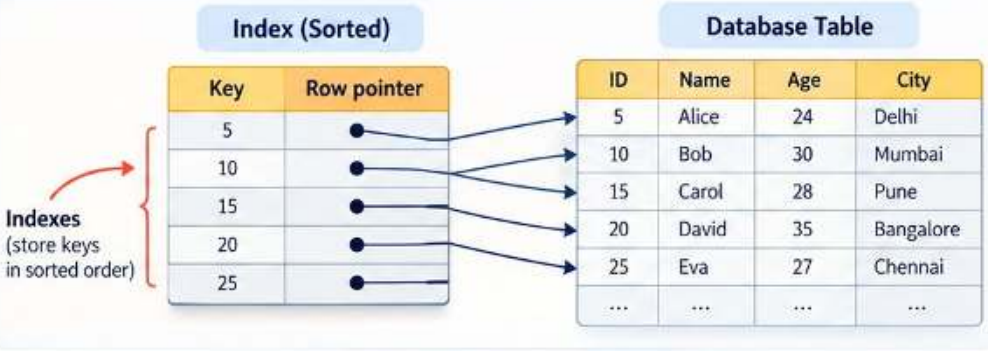
B-Tree	Common for exact matches and range queries.
Hash	Best for equality lookups.
Full-Text	Used for searching large textual data.
Bitmap	Suitable for low-cardinality columns (e.g., gender).

03

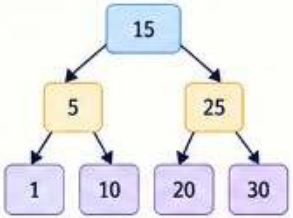
When to Use Indexes

Read-heavy operations:
Indexes speed up query performance.

Write-heavy systems:
Be cautious, as indexes can slow down inserts and updates.

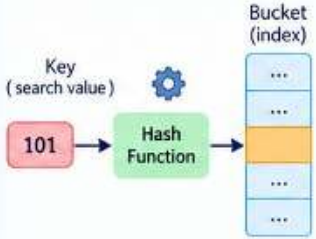


B-Tree Index



Balanced tree structure.
Good for exact matches
and range queries.

Hash Index



Uses a hash function
to find the row.
Best for equality lookups.

Bitmap Index

ID	Gender (Bitmap)	
	Male	Female
5	1	0
10	0	1
15	1	0
20	1	0
25	0	1
...	...	...

Uses bitmaps for each value.
Suitable for low-cardinality
columns (e.g., gender).

Instructor: Rahul Rajat Singh

Normalization vs. Denormalization

Normalization

Organize data into tables to reduce redundancy

Goal	Reduce data redundancy by organizing data into tables.
Benefits	Minimizes storage costs, eliminates anomalies.
Drawback	Can lead to complex joins and slower read performance.

Denormalization

Introduce redundancy to reduce join operations

Goal	Introduce redundancy to reduce join operations and speed up reads.
Benefits	Faster read performance.
Drawback	Increased storage and potential data anomalies.

When to Use Each

- **Normalization:** For transactional systems (OLTP).
- **Denormalization:** For reporting systems or read-heavy workloads.

Normalized



VS

Denormalized



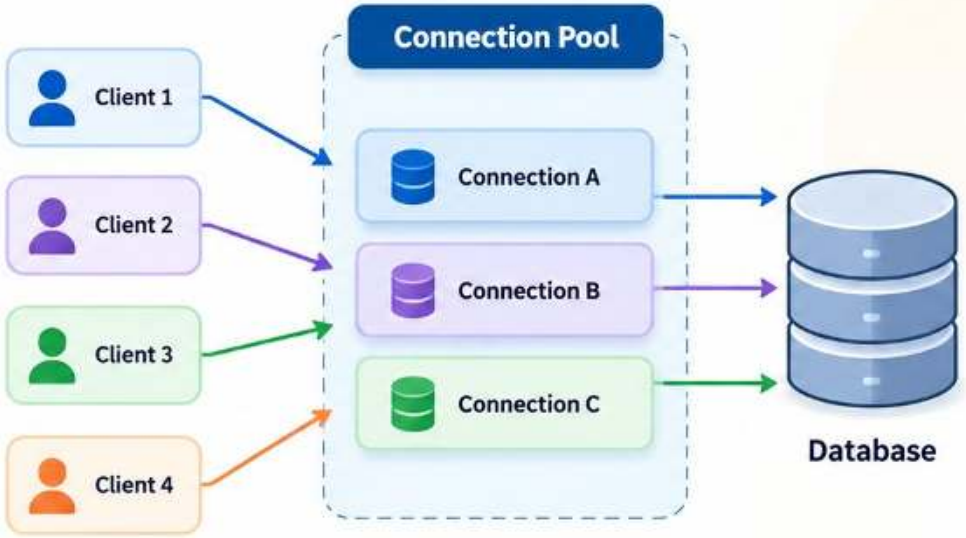
Additional Techniques – Connection Pooling

What is Connection Pooling?

A technique used to manage database connections efficiently by reusing established connections instead of creating new ones.

Why Use It?

- Reduces overhead caused by frequent connection creation and teardown.
- Helps handle a large number of concurrent connections effectively.



Additional Techniques - Query Optimization



Definition

The process of improving the performance of SQL queries.

Techniques

1 Use of Indexes

Leverage indexes to speed up search operations.



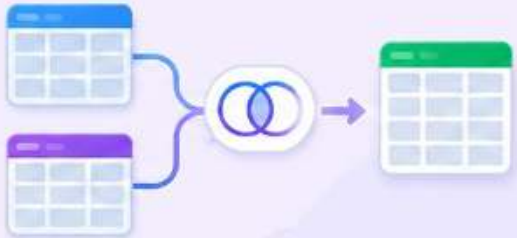
2 Avoiding N+1 Queries

Reduce unnecessary database hits by using joins or batching queries.



3 Using Proper Joins

Minimize complex joins when possible.



Additional Techniques - Materialized Views

Definition

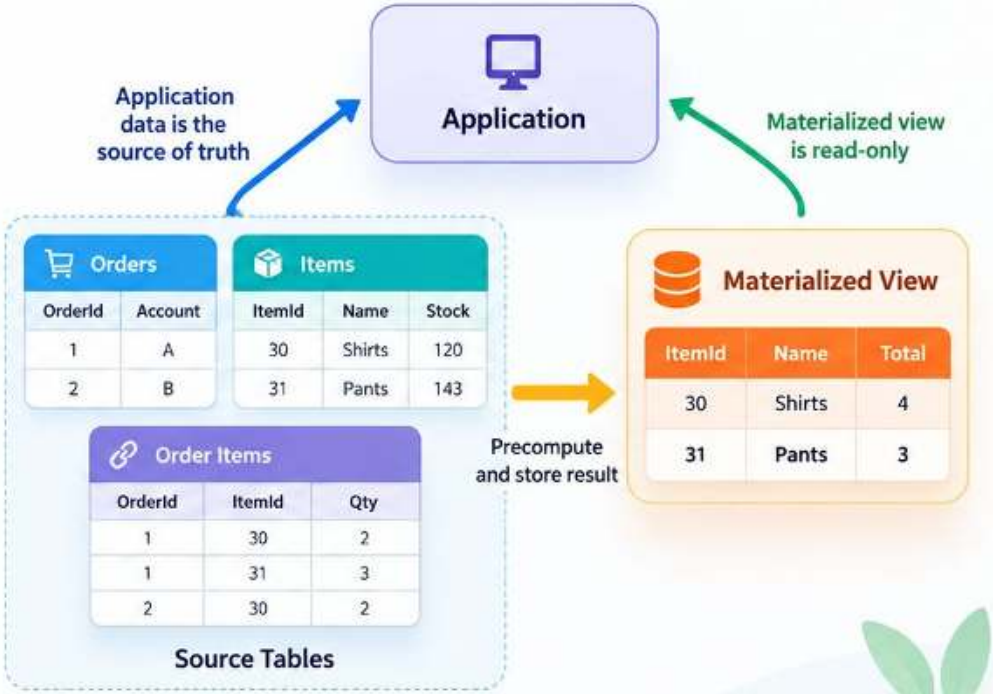
A precomputed query result stored as a table.

Benefits

- Speeds up query performance by avoiding real-time computation.
- Useful in reporting and data warehousing.

Use Cases

- Data aggregation or summary data that doesn't change frequently.
- Reporting systems where fast retrieval is critical.



Additional Techniques – Batching & Pagination

Batching

Definition: Sending multiple operations in a single request or transaction to reduce overhead.

Use Case:
Bulk inserts or updates.



Pagination

Definition: Breaking large sets of data into smaller chunks for efficient retrieval.

Prevents large queries
that could lead to timeouts or memory issues.

Ensures responsive UI
by fetching data incrementally.



Interview Questions

Database Performance Optimization Techniques



1. **What is database replication,** and why is it important for performance?
2. **Can you explain the difference** between sharding and partitioning in databases?
3. **How does the CAP theorem impact** database performance decisions?
4. **What are some common types of indexes,** and when would you use them?
5. **What is the difference between normalization and denormalization,** and how do they affect performance?
6. **How does connection pooling improve** database performance?
7. **What is query optimization,** and how would you approach optimizing a slow query?
8. **What are materialized views,** and how do they enhance performance?
9. **How would you handle large datasets** in your application?
10. **What are the trade-offs between consistency and performance** in distributed databases?

Instructor: Rahul Rajat Singh

Summary and Key Takeaways

1 Replication & Sharding

Key techniques for scaling databases across multiple servers.



2 CAP Theorem

Understanding trade-offs between consistency, availability, and partition tolerance.



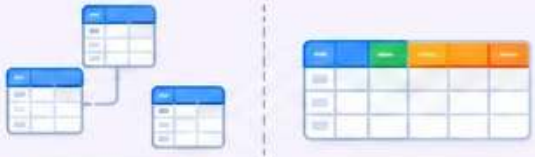
3 Indexes

Crucial for improving query performance, but need to be managed properly.



4 Normalization vs. Denormalization

Choose based on workload (transactions vs. reporting).



Normalization

Denormalization

5 Other Techniques

Connection pooling, query optimization, materialized views, and batching for efficient database operations.



Connection Pooling

Query Optimization

Materialized Views

Batching

6 What's Next?

Summary and Recap - Performance



Instructor: Rahul Rajat Singh

Section Summary - Performance

1



Introduction to System Performance

2



Caching for Speed Optimization

3



Messaging & Queues for Decoupling

4



Concurrency & Parallelism

5



Database Performance Optimization Techniques

WHAT'S IN NEXT SECTION?

Reliability – Availability, Failover & Recovery



Interview questions - **Caching for Speed Optimization**

✅ Core Conceptual Questions

🔹 What is caching and why is it important in system design?

Answer:
Caching is the technique of storing frequently accessed data in a faster storage layer (e.g., in-memory) to reduce data retrieval time. It helps:

- Minimize latency
- Reduce backend/database load
- Improve system scalability and user experience Caching is critical in high-traffic systems where performance and responsiveness are essential.

🔹 Explain different types of caching and where they are used.

Answer:

- **Client-side cache**: Browser cache, service workers — used for static assets, localStorage.
- **Server-side cache**: In-memory (e.g., Redis, Memcached) — used in API responses or session storage.
- **CDN cache**: Distributed edge servers cache static content like images, JS, CSS.
- **Database cache**: Materialized views or query result caching — reduces repeated complex DB queries.

🔹 What are write-through vs. write-back caching strategies?

Answer:

- **Write-through**: Data is written to cache and database simultaneously. Ensures consistency but slower write performance.
- **Write-back (write-behind)**: Data is written to cache first; the DB is updated later asynchronously. Faster writes but at risk of data loss on cache failure.

🔹 What is lazy loading (cache-aside) and when would you use it?

Answer: Lazy loading (cache-aside) loads data into the cache only when needed:

- On a cache miss, fetch from DB → populate cache → return data.
- Used when not all data is frequently accessed or when cache storage is limited.
- Gives fine-grained control but requires cache invalidation management.

🧠 Scenario-Based Questions

🔹 How would you use caching to optimize a product details page?

Answer:

- Cache frequently viewed product data in Redis with a TTL.
- Use lazy loading so only requested products are cached.
- Use CDN caching for images.
- Invalidate or refresh cache on product updates. This reduces DB hits and improves page load speed significantly.

🔹 What eviction strategy would you choose for a memory-limited system?

Answer:

- **LRU (Least Recently Used)** is ideal when recently accessed items are more likely to be used again.
- **LFU (Least Frequently Used)** if certain items are accessed more often than others.
- Eviction strategy should match access patterns to avoid cache misses.

🔹 How would you keep cache and database in sync?

Answer:

- Use **write-through** for strong consistency.
- Use **cache invalidation** on data updates (manually or via message queues).
- Set appropriate TTLs to auto-refresh stale data.
- Optionally, use change data capture (CDC) mechanisms or event-driven updates.

🔹 What are the potential downsides or risks of aggressive caching?

Answer:

- **Stale data**: Cached values may be outdated if not invalidated properly.
- **Cache stampede**: Multiple requests for uncached data can hit the backend simultaneously.
- **Overuse of memory**: Poor eviction strategy can lead to inefficient memory usage.
- **Complexity**: Cache invalidation and consistency handling can increase system complexity.

🔧 Practical Implementation

🔹 How would you implement Redis caching in a web application?

Answer:

- Use Redis as a key-value store (e.g., `product:123 → productData`).
- On cache miss, fetch from DB, store in Redis with TTL.
- On cache hit, return directly from Redis.
- Use libraries like StackExchange.Redis (C#), `redis-py` (Python), or `ioredis` (Node.js).

🔹 How can you prevent cache stampedes or thundering herd problems?

Answer:

- Use **lock or mutex** during cache miss to ensure only one backend call populates the cache.
- Use **stale-while-revalidate** strategies to serve old data while refreshing in the background.
- Implement **randomized TTLs** to spread out cache expiry.

🔹 What tools can be used for distributed caching?

Answer:

- **Redis Cluster**: High availability and partitioning.
- **Memcached**: Lightweight in-memory store.
- **Hazelcast, Apache Ignite**: In-JVM distributed cache for enterprise use.
- Use consistent hashing and replication to manage cache distribution across nodes.